

Министерство образования и молодёжной политики Свердловской области
государственное автономное профессиональное
образовательное учреждение Свердловской области
«Уральский радиотехнический колледж им. А. С. Попова»

ДОПУСТИТЬ К ЗАЩИТЕ

Зав. Отделением

_____ О. В. Алферьева

«__» _____ 2025 г.

РАЗРАБОТКА МАКЕТА ИТ-ИНФРАСТРУКТУРЫ СОВРЕМЕННОЙ КОМПАНИИ

Пояснительная записка к дипломному проекту

РК 09.02.06 401 09 ПЗ

Рецензент

_____ М. А. Мокшанцев

«__» _____ 2025 г.

Руководитель

_____ Д. С. Апататьев

«__» _____ 2025 г.

Нормоконтролер

_____ Н. Г. Васильев

«__» _____ 2025 г.

Разработчик

_____ Т. И. Куваев

«__» _____ 2025 г.

Содержание

Введение	4
1 Системный раздел	6
1.1 Значение IT-инфраструктуры в современном мире.....	6
1.2 Компоненты и составляющие IT-инфраструктуры.....	7
1.3 Основные принципы проектирования.....	10
1.4 Безопасность инфраструктуры	21
1.5 Требования для построения инфраструктуры	29
1.6 Выбор лабораторной среды для создания макета	30
2 Техничко-технологический раздел	36
2.1 Определение ПО, сервисов и производителей, подходящих под требования	36
2.2 Планирование и подготовка к реализации будущей инфраструктуры.....	41
2.3 Настройка сетевого оборудования.....	47
2.3 Настройка серверных ОС	61
2.4 Развертывание сервисов.....	67
2.6 Настройка пользовательских ОС.....	86
2.7 Проверка соответствия представленным и законодательным требованиям	92
3 Охрана труда, ТБ и производственная санитария при эксплуатации инфраструктуры	94
Заключение	96
Перечень сокращений и условных обозначений	97
Список используемых источников.....	100
Приложение А (обязательное) Прайс-лист с ценами	101
Приложение Б (обязательное) Акт апробации	102

					РК 09.02.06 401 09 ПЗ			
Изм	Лист	№ докум.	Подп.	Дата				
Разраб.		Куваев Т. И.			Разработка макета IT-инфраструктуры современной компании Пояснительная записка	Лит.	Лист	Листов
Пров.		Апататьев Д. С.					3	102
						ГАПОУ СО УРТК им. А. С. Попова, гр. Са-401		
Н. контр.		Васильев Н. Г.						
Рецензент		Мокшанцев М. А.						

Введение

В условиях активного развития цифровых технологий важной задачей для большинства организаций становится построение устойчивой, масштабируемой и управляемой ИТ-инфраструктуры. Современная компания нуждается не только в средствах вычислений и хранения данных, но и в логично организованной, безопасной, отказоустойчивой среде, способной поддерживать внутренние процессы, обеспечивать сотрудников необходимыми ресурсами и соответствовать внутренним и внешним требованиям.

Формирование такой инфраструктуры базируется на ряде факторов, включая:

- внутренние регламенты организации, определяющие требования к информационной безопасности и доступу к данным;
- структуру подразделений и характер их взаимодействия;
- специфику рабочих процессов и особенности внутренних ИТ-сервисов;
- практические ограничения, связанные с бюджетом, кадровыми ресурсами и физической инфраструктурой;
- необходимость демонстрации, тестирования и отработки решений до их внедрения в финальной среде.

Создание полноценной ИТ-инфраструктуры с нуля требует значительных временных и финансовых затрат. В связи с этим особую актуальность приобретает разработка макета – логически завершённой, упрощённой модели, отражающей ключевые функции и архитектурные принципы будущей инфраструктуры. Макет позволяет заранее оценить работоспособность решений, выявить потенциальные проблемы и проверить соответствие проектируемой системы заявленным требованиям.

Объектом исследования в данной работе является процесс проектирования корпоративной ИТ-инфраструктуры, предметом – этапы построения логической модели и её реализация в виртуальной среде. Цель проекта заключается в разработке макета ИТ-инфраструктуры условной современной компании, включающего ключевые сетевые, сервисные и организационные компоненты.

В рамках дипломной работы решаются следующие задачи:

- сбор и структурирование требований к инфраструктуре;
- выбор архитектурной модели и обоснование проектных решений;
- построение сетевой логики и выделение функциональных зон;
- развёртывание основных сервисов в виртуальной среде;
- обеспечение отказоустойчивости и управляемости компонентов;

- организация доступа, сегментации и фильтрации трафика;
- документирование и проверка соответствия техническому заданию.

Методологическую основу проекта составляет совокупность теоретического анализа, инженерного проектирования и практической реализации инфраструктурных решений. Актуальность работы определяется необходимостью создания наглядного, воспроизводимого макета, который может использоваться для предварительного тестирования, обучения и демонстрации архитектурных подходов без задействования физической инфраструктуры предприятия.

					РК 09.02.06 401 09 ПЗ	Лист
Изм	Лист	№ докум.	Подп.	Дата		5

1 Системный раздел

1.1 Значение IT-инфраструктуры в современном мире

ИТ-инфраструктура – это взаимосвязанные компоненты, из которых состоит информационная структура организации. В состав такой инфраструктуры входит техническое оборудование для выхода в интернет, выполнения рабочих задач и комфортной работы в офисе.

По мере распространения интернета и его доступности стали создаваться единые цифровые системы. Они необходимы для бесперебойной работы организации на всех уровнях.

Главная задача ИТ-инфраструктуры – обеспечить стабильную и безопасную работу организации, а значит, она решает множество актуальных вопросов. В первую очередь, важнейшим аспектом является защита информации. Сегодня, когда данные становятся наиболее ценным активом, грамотное управление ими критично важно. Организации разрабатывают системы, которые позволяют защищать конфиденциальную информацию от кибер-атак и утечек.

Еще одна важнейшая задача ИТ-инфраструктуры – это обеспечение отказоустойчивости. Никакой бизнес не может позволить себе длительные простои, и здесь на помощь приходят резервирование и автоматические решения для восстановления. Когда случаются сбои, возможность быстро вернуть работу системы в нормальный режим становится критически важной.

Оптимизация рабочих процессов – это значительное преимущество, которое дает современная ИТ-инфраструктура. Интеграция различных программных решений позволяет работникам получать доступ к нужной информации в любое время, что значительно облегчает выполнение задач. Поработав в такой системе, сотрудники быстро замечают, как снижается время, затрачиваемое на рутинные операции, и как повышается общая продуктивность.

По мере роста бизнеса возникает необходимость в гибкости и способности адаптироваться. ИТ-инфраструктура предоставляет возможность масштабировать ресурсы в зависимости от текущих потребностей. Компании могут увеличивать мощность своих систем, чтобы справляться с пиком загрузки, и, наоборот, снижать её в спокойные времена, что снижает затраты организации.

Кроме того, сегодня все чаще обсуждается такая тема, как мобильность и удаленная работа. ИТ-инфраструктура должна отвечать требованиям этого нового времени, позволяя сотрудникам иметь доступ к данным и приложениям независимо от места их нахождения.

Наконец, ИТ-инфраструктура не стоит на месте. Она требует постоянного обновления и адаптации к новым технологиям. Внедрение облачных сервисов, инструментов для анализа

данных и средств, которые помогают обеспечить безопасность и конфиденциальность информации становится необходимостью.

1.2 Компоненты и составляющие IT-инфраструктуры

Любая IT-инфраструктура – это комплекс взаимосвязанных компонентов, где каждый элемент выполняет строго определенную роли. Без их совместной работы невозможны ни обработка данных, ни коммуникация, ни даже базовое функционирование организации. Условно компоненты делятся на несколько типов:

- аппаратные;
- программные;
- сетевые.

Аппаратные компоненты можно подразделить на несколько подтипов:

- системы хранения данных – обязательный элемент относительно продвинутых организаций и выше. От обычного сервера с жесткими дисками этот элемент отличается наличием специализированных решений, по типу NAS для файлового доступа или SAN, где данные передаются блоками через iSCSI/Fibre Channel. RAID-массивы в хранении данных являются абсолютным стандартом. Если один диск «умрет», данные не потеряются;

- автоматизированное рабочее место – это всё, чем пользуются сотрудники: офисные ПК, инженерные станции, принтеры, телефоны. Главное правило – соответствие задачам. Для бухгалтерии хватит слабого ПК с офисным пакетом ПО, для специфических задач с обработкой графики – рабочая станция с GPU невероятной мощности;

- сервер. В любой организации подобное оборудование можно грубо назвать «моторами» инфраструктуры. Бывают стоечные, блейд-сервера или Tower-сервера (по виду – обычный ПК). Их задача – не выключаться, поэтому всё идет с запасом: блоки питания, диски с горячей заменой и много других функций.

Аппаратные компоненты в инфраструктуре лишь некоторая часть. Поверх них работают программные компоненты, которые выполняют большую часть функций, необходимых организации.

Операционная система – фундамент, который превращает набор плат в компьютере в рабочую среду. Без неё сервера, ПК и смартфоны останутся бесполезными устройствами. ОС управляет ресурсами, распределяет память между программами, выделяет процессорное время, обеспечивает доступ к дискам и сети. Даже встроенные устройства (роутеры, например) имеют свою операционную систему – обычно сильно урезанные дистрибутивы Linux. Ключевая задача

любой ОС – быть посредником, то есть принимать команды от программ, преобразовывать их в инструкции для процессора и гарантировать, что сбой одного приложения не сможет помешать остальным.

Системное программное обеспечение – это инструментарий для управления инфраструктурой. Оно не решает бизнес-задачи напрямую, но обеспечивает стабильность и контроль над аппаратными ресурсами. Если брать в пример гипервизоры, то увидим, что позволяют запускать на одном сервере десятки виртуальных машин, изолируя их друг от друга. Утилиты резервного копирования создают копии данных даже во время работы систем. Инструменты мониторинга следят за нагрузкой на процессор, диски и сеть, отправляя предупреждения при критических значениях. Отдельная категория – драйвера. Микропрограммы, которые «объясняют» аппаратной части, как взаимодействовать с ОС. Без них видеокарта не будет выводить изображение. Системное ПО работает в фоне, но его отсутствие парализует инфраструктуру, примерно, как отключение электричества на предприятии.

Прикладное программное обеспечение – это программы, которые непосредственно решают задачи бизнеса или пользователя. Например, офисные пакеты программ (P7, Мой Офис) позволяет бухгалтеру формировать отчеты, CAD-программы помогают инженеру проектировать детали, а Paint дает возможность дизайнеру редактировать изображения. В корпоративной среде востребованы CRM-системы (Bitrix24, Salesforce) для учета клиентов, ERP-платформы (1C, SAP) для управления финансами и логистикой, ну и конечно, специализированные решения вроде САПР для машиностроения или медицинских информационных систем для больниц. Даже веб-браузер – часть прикладного ПО: без него сотрудник не откроет корпоративный портал. Эти программы зависят от системного ПО и ОС, но именно они создают ценность для бизнеса.

Ограничивается программное обеспечение последним типом – промежуточным ПО. Промежуточное программное обеспечение – это что-то вроде клея, который соединяет приложения друг с другом и с инфраструктурой, но при этом может выступать и в роли прикладного ПО, конкретное определение зависит от контекста эксплуатации. Используется подобное как правило в серверной части инфраструктуры, где имеется какое-то нагромождение сервисов. В качестве примера можно привести СУБД (PostgreSQL, MySQL и прочие), которая хранит данные для интернет-магазина, веб-сервер (Nginx, Apache) принимает запросы от покупателей и может передавать какую-то информацию дальше по системе. Без промежуточного ПО приложения не смогут взаимодействовать.

Программные компоненты обеспечивают логику работы инфраструктуры, но без сети, в большинстве случаев – мало полезны. Сервер не передаст данные в СУБД на другом сервере,

сотрудник не отправит отчет коллеге. Сетевые компоненты – это что-то вроде кровеносной системы ИТ-инфраструктуры, которая соединяет устройства, маршрутизирует трафик.

Физический уровень – основа любой сети, по которой движутся данные: кабели, разъемы, сетевые карты и оборудование, преобразующее сигналы в электрические импульсы или световые сигналы. Самый распространенный вариант – Ethernet. Витая пара CAT5E дешевая, проста в монтаже и подходит для офисов, где расстояние между устройствами не превышает 100 метров. Для больших дистанций или высокой скорости используют витую пару с категорией выше или оптоволокно – световые сигналы в таких кабелях почти не затухают, всегда используется в глобальных магистралях.

Канальный уровень обеспечивает передачу данных между устройствами в рамках локальной сети, организуя их в структурированные блоки – кадры. На этом уровне к исходной информации добавляется служебная информация, MAC-адреса отправителя и получателя, контрольные суммы для проверки целостности и управляющие флаги. Без этой структуризации данные не могут быть корректно интерпретированы принимающей стороной.

Основная задача уровня – точная доставка кадров, где эту задачу решает MAC-адресация. Например, при отправке данных с компьютера на принтер коммутатор анализирует MAC-адрес получателя и перенаправляет фрейм только в соответствующий порт. Это исключает избыточную рассылку трафика, характерную для устаревших сетевых концентраторов.

Канальный уровень также отвечает за обнаружение и коррекцию ошибок передачи. Контрольная сумма (CRC), включенная в каждый кадр, выявляет искажения данных из-за помех или сбоев. При обнаружении ошибки фрейм отклоняется, что инициирует повторную передачу со стороны отправителя.

На канальном уровне реализованы:

- Ethernet (IEEE 802.3) – стандарт для проводных сетей, использующий MAC-адресацию и фреймы до 1500 байт;
- Wi-Fi (IEEE 802.11) – беспроводная передача данных с механизмами минимизации помех;
- VLAN (IEEE 802.1Q) – логическое разделение физической сети на изолированные сегменты (например, для отделов компании);
- PPPoE – инкапсуляция данных для широкополосных подключений;
- LLDP – обмен служебной информацией между сетевыми устройствами.

Сетевой уровень обеспечивает передачу данных между устройствами, находящимися в разных сетях, определяя оптимальные маршруты для их доставки. Его ключевая задача – преодолеть ограничения локальной коммуникации, организовав взаимодействие в масштабах

глобальных сетей, включая глобальные сети. На этом уровне кадры инкапсулируются в пакеты, содержащие IP-адреса отправителя и получателя и прочую служебную информацию для управления маршрутизации.

Основной функцией уровня является маршрутизация – процесс выбора пути для пакетов через маршрутизаторы. Для этого нужны таблицы маршрутизации, где указаны направления к сетевым сегментам. Например, пакет, отправленный из офиса в Москве в филиал в Берлине, проходит через несколько маршрутизаторов, каждый из которых анализирует IP-адрес назначения и перенаправляет его дальше по оптимальному маршруту.

Сетевой уровень также отвечает за:

- фрагментацию пакетов – разделение данных на части, если их размер превышает MTU (максимальный размер блока, поддерживаемый канальным уровнем);
- логическую адресацию – использование IP-адресов (IPv4/IPv6) вместо MAC-адресов, что позволяет работать в гетерогенных сетях (Ethernet, Wi-Fi, спутниковая связь);
- обработку ошибок – протоколы вроде ICMP отправляют уведомления о проблемах доставки (например, если узел недоступен).

На сетевом уровне работают:

- IP (Internet Protocol) – основа интернета, обеспечивающая адресацию и маршрутизацию;
- ICMP (Internet Control Message Protocol) – диагностика и уведомления об ошибках (например, ping);
- NAT (Network Address Translation) – преобразование частных IP-адресов в публичные для выхода в интернет;
- VPN (Virtual Private Network) – создание защищенных туннелей между сетями;
- OSPF, BGP – протоколы динамической маршрутизации.

1.3 Основные принципы проектирования

Проектирование ИТ-инфраструктуры – комплексный процесс, направленный на создание технологической основы для поддержки бизнес-процессов организации. Современные принципы проектирования информационных систем учитывают не только текущие потребности, но и обеспечивают возможность масштабирования и отказоустойчивости.

Современная инфраструктура должна проектироваться с учетом концепции "пирамиды надежности", где каждый уровень (аппаратный, программный, сетевой) обеспечивает резервирование и отказоустойчивость. Рассматривая этот момент чуть подробнее, стоит отметить

– полная отказоустойчивость в большинстве случаев требует достаточно больших затрат, ведь сделать резерв всего иногда попросту невозможно, поэтому иногда приходится где-то сэкономить, опираясь на надежность имеющихся аппаратных и программных средств.

Выделяют несколько основных типов, которые выбираются в зависимости от задач и внешних требований организации:

- традиционная;
- облачная;
- гибридная.

Традиционная ИТ-инфраструктура, основанная на локальных дата-центрах и физических серверах, на протяжении десятилетий оставалась преобладающей моделью для большинства предприятий. Несмотря на растущие тенденции к облачным решениям и виртуализации, эта модель по-прежнему сохраняет свою актуальность и востребованность в ряде секторов.

В основе традиционной инфраструктуры лежит концепция полного контроля организации над собственными информационными ресурсами. Выделенные серверные помещения, оснащенные надежными системами электропитания, охлаждения и физической безопасности, выступают фундаментом этой архитектуры. Оборудование приобретается в собственность компании и размещается на ее территории. За счет этого обеспечивается прямое управление аппаратными мощностями и данными. Данный подход обусловлен жесткими требованиями к безопасности информации, которые характерны, например, для сфер государственного управления, финансов, энергетики и прочих отраслей, которые могут содержать в себе объекты КИИ.

При проектировании традиционной инфраструктуры ключевыми аспектами являются:

- обеспечение избыточности и отказоустойчивости с помощью кластерных решений, резервного оборудования и систем бесперебойного питания;
- иерархическая структура сетевой инфраструктуры с ядром, распределительными коммутаторами и точками подключения клиентских устройств;
- интеграция с унаследованными системами и обеспечение совместимости с существующими ИТ-ресурсами.

Классическая сетевая инфраструктура опирается на жесткую иерархию физических компонентов с жестко закрепленными функциями для каждого элемента. Основу формируют проводные соединения, создающие фиксированную топологию с централизованными управляющими узлами. Коммутационные устройства, размещенные в защищенных серверных помещениях, выполняют роль системного каркаса, обеспечивая взаимодействие между

серверами, рабочими станциями и периферийными устройствами. Архитектура подразумевает строгое разделение на зоны: внутренние сегменты сети изолированы от внешних подключений физическими барьерами, а межзональный доступ регулируется через централизованные контрольные точки.

В отличие от гибких современных решений, управление трафиком здесь базируется на заранее заданных правилах, жестко прописанных в конфигурациях оборудования. Данные передаются по статическим маршрутам, оптимизированным для стандартных нагрузочных сценариев. Подобная схема гарантирует стабильность работы, однако плохо адаптируется к динамичным изменениям – резким скачкам числа подключений или неравномерному распределению запросов.

Система безопасности реализуется через многоуровневую изоляцию. Внешний периметр контролируется аппаратными фильтрами, блокирующими несанкционированные подключения на уровне сетевых пакетов. Внутренние сегменты дополнительно разделяются по функциональному принципу для ограничения горизонтального перемещения угроз. Серверы баз данных, к примеру, могут располагаться в отдельном физическом контуре с ограниченным доступом для определенных групп пользователей. Централизованная реализация аутентификации упрощает администрирование, одновременно создавая риски единых точек отказа.

Масштабирование инфраструктуры связано с физическим расширением – установкой новых коммутаторов, прокладкой кабельных трасс, настройкой дополнительных портов. Подобные операции усложняют топологию и повышают эксплуатационные расходы. Интеграция новых узлов требует проверки совместимости оборудования, обновления таблиц маршрутизации и тестирования на предмет возникновения узких мест. Подключение облачных сервисов осуществляется через выделенные шлюзы, что увеличивает задержки передачи данных.

Надежность сети обеспечивается дублированием критических элементов: резервных линий связи, избыточных источников питания и зеркальных устройств. Несмотря на резервирование, часть мощностей часто остается невостребованной в штатных режимах, а перенастройка под меняющиеся условия требует остановки работы для физического переподключения компонентов.

Облачная инфраструктура представляет собой современный подход к организации вычислительных ресурсов, где все компоненты – серверы, хранилища и сети, предоставляются как сервис через интернет. В отличие от традиционных локальных решений, она исключает необходимость закупки и обслуживания собственного оборудования, перекладывая эти задачи на облачного провайдера.

Современные облачные системы строятся на принципе абстракции аппаратных ресурсов, преобразуя их в гибкие логические сервисы. Фундаментом этой модели выступает гипервизор – технология, разделяющая вычислительные мощности сервера на изолированные виртуальные среды. Каждая виртуальная машина функционирует независимо, получая выделенные параметры CPU, RAM и хранилища, что устраняет зависимость между ОС и физическим «железом». В отличие от традиционных дата-центров с фиксированным распределением ресурсов, облака позволяют динамически перераспределять мощности: незадействованная память одной VM может быть мгновенно передана соседнему экземпляру без прерывания работы сервисов.

Эластичность инфраструктуры усиливается за счет программно-определяемых сетей (SDN). Их ключевая инновация – декомпозиция управления: контроллер верхнего уровня (control plane) централизованно задает правила маршрутизации, тогда как коммутаторы (data plane) выполняют только транспортировку пакетов. Используя открытые протоколы вроде OpenFlow, система адаптирует топологию под текущие задачи – от автоматического создания изолированных сегментов для клиентов до глобальной балансировки нагрузки между дата-центрами. Такой подход не только минимизирует зависимость от вендорного оборудования, но и позволяет мгновенно блокировать кибератаки на уровне всей сети.

Следующий уровень абстракции – виртуализация хранилищ. Технологии распределенных файловых систем (Ceph, GlusterFS) объединяют дисковые массивы серверов в единый пул с автоматическим восстановлением данных. Информация дробится на блоки, которые реплицируются между узлами, а метаданные управляются кластерными службами. При отказе диска система перенаправляет запросы к другим копиям, сохраняя доступность данных.

Сегодня существует два основных варианта построения облачной инфраструктуры:

- частные облака – выделенные ресурсы для одной организации;
- публичные облака – общие ресурсы у провайдера.

Частное облако – это модель локальных облачных вычислений, которая предоставляет выделенные ресурсы, включая вычислительную мощность, хранилище и сеть, ограниченному числу пользователей в рамках одной организации. Большинство компаний выбирают частное облако в тех случаях, когда они хотят иметь огромный контроль над своей информацией и быть в безопасности. Это могут быть виртуальные облака, такие как VMware Cloud и OpenStack, а также облачные решения, предоставляемые поставщиками частных платформ.

Публичное облако – тип облака, которое провайдер предоставляет в аренду на время. Используется для создания пул ресурсов, выделенного под конкретный проект или задачу. Можно создавать несколько виртуальных серверов и управлять ими, однако физического доступа к оборудованию у компании не будет. Оставшиеся ресурсы, достаточные, чтобы выделить железо,

будут использоваться уже другими компаниями. То есть, вашими виртуальными соседями будут сервисы других организаций.

Ресурсы в публичном облаке будут использоваться по модели pay-as-you-go – по мере потребления. В случае пиковых нагрузок (акции, распродажи) облако будет расходовать больше ресурсов и плата за них возрастает. Но когда нагрузка вновь стабилизируется, облако продолжит работать на обычных мощностях и плата снизится. Публичные облака выросли из частных облаков провайдеров, которые накопили экспертизу и поняли, что ее можно предлагать в форме продукта. Так же, как и частное облако, публичное представляет собой инфраструктуру по типу IaaS.

Рассматривая российский рынок облачных услуг, самыми крупными провайдерами являются:

- Timeweb;
- VK Cloud;
- Yandex Cloud;
- MTS Web Services;
- Cloud.ru.

Чтобы детальнее понять разницу между этими двумя типами, нужно чуть углубиться в документации различных провайдеров.

Термин «частное облако» был введен, чтобы провести различие между этими внутренними облачными средами и публичными облачными сервисами сторонних производителей. Пользователи как публичных, так и частных облачных сервисов имеют определенные сходства.

И публичные, и частные облака абстрагируют и совместно используют вычислительные ресурсы, такие как аппаратное обеспечение, сети, программное обеспечение, обычные серверы и серверы хранения данных.

Пользователи могут выделять и освобождать ресурсы по мере необходимости и управлять конфигурацией инфраструктуры.

Оба типа облачных сред используют схожие базовые технологии. Они используют виртуализацию для абстрагирования базового оборудования и его предоставления через API. Они также обеспечивают автоматическое масштабирование, автоматическую оркестровку, отказоустойчивость и улучшенные системы резервного копирования.

Как публичные, так и частные облака обеспечивают операционную эффективность ИТ-инфраструктуры компании. Компании могут сократить расходы за счет централизованного управления инфраструктурой. Быстрее масштабируются и быстрее выводят на рынок новые

продукты. Существующие мощности используются более эффективно, а затраты снижаются. Опыт малых и средних предприятий показывает, что во многих случаях публичные облака значительно эффективнее частных.

Гибридное облако – смесь публичного и частного. Кратко говоря – берутся две инфраструктуры, связываются через VPN или прямые каналы связи (что не получило сильного распространения) и организация по итогу этих действий получает фиксированные мощности с возможностью расширения.

При таком подходе также важно учесть совместимость API, одинаковые сетевые настройки. Иногда ставят шлюзы для синхронизации – чтобы приложения с обеих сторон видели друг друга как одну систему.

Также играет роль финансовая составляющая – локальная инфраструктура все также требует вложений, но в меньшем количестве, но затраты на облачную инфраструктуру заметно сокращаются из-за модели оплаты по факту использования, вплоть до почти нулевых значений, когда нет сильной нагрузки и локальная инфраструктура справляется сама.

Если рассматривать такой подход с точки зрения безопасности, то появляется возможность обрабатывать критичные данные – внутри, а различные функции и сервисы, не требующие специфических условий – перенести в публичное облако. Аудит доступа в таких случаях проводится везде, даже если части облака физически в разных местах. Итоговое сравнение всех моделей приведено в таблице 1.1.

Таблица. 1.1 – Сравнение моделей

Параметр	Частная	Публичная	Гибридная
1	2	3	4
Уровень контроля	Высокий	Ограниченный	Высокий
Гибкость и масштабируемость	Ограниченная	Высокая	Очень высокая
Теоретическая стоимость инфраструктуры	Высокая	Низкая	Высокая
Уровень безопасности данных	Высокий	Зависит от поставщика. Потенциально – низкий	Зависит от реализации и поставщика. Потенциально – высокий

Продолжение таблицы 1.1

1	2	3	4
Возможность соблюдения требований регуляторов	Высокая	Зависит от поставщика. Потенциально – низкая	Зависит от реализации и поставщика. Потенциально – высокая
Производительность	Низкая	Количество возможных ресурсов зависит от поставщика. Потенциально – очень высокая	Очень высокая

Развивая тему облачных моделей, стоит отметить модель управления инфраструктурой под названием IaC (инфраструктура как код). Условно говоря, это подход, с помощью которого инфраструктура управляется через конфигурационные файлы и код. Вместо ручной настройки серверов, виртуальных машин и прочих необходимых сервисов, системный администратор может написать несколько файлов, в которых опишет что, в каком формате и на каких устройствах хочет получить. Почти та же ручная настройка – но фактически не выходя из консоли.

К самым распространенным средствам эксплуатации такого подхода относят:

- Ansible. При привычном подходе нужно вручную прописывать каждую команду или скрипт и по кругу запускать их на серверах. Когда серверов много, это процесс становится сложным и трудоёмким, а вот уже с помощью Ansible всю настройку можно прописать в одном конфигурационном файле, который программа разошлёт на большое количество машин. Пример файла конфигурации можно увидеть на рисунке 1.1.

- Terraform. Этот продукт уже не занимается как таковой настройкой инфраструктуры, он ее только разворачивает в определенной среде. Если у организации имеется какое-то облако или несколько серверов, выделенных под виртуализацию – развернуть и настроить все виртуальные машины можно ровно также через несколько файлов.

Есть еще много другого ПО, которое позволяет автоматизировать управление инфраструктурой организации: Puppet, Chef, Vagrant, SaltStack и многие другие. Но как правило, все они используются в связке с первыми двумя средствами, чтобы как можно меньше лезть в сами сервера и минимизировать ручную настройку.

```

1  - become: yes
2  name: bootstrap
3  hosts: control
4  vars_prompt: []
11 tasks:
12   - name: full system upgrade []
17   - name: install zsh []
28   - name: create aur_builder user []
39   - name: install fonts []
59   - name: install cli apps []
80   - name: install gui apps []
99   - name: install xorg
100  become: yes
101  pacman:
102    name:
103      - mesa
104      - xorg-server
105      - xorg-xinit
106      - xorg-xrandr
107      - xorg-xbacklight
108  - name: install de []
114  - name: install themes []
121  - name: install vmware []
149  - name: link configs []
158  - name: link xorg configs []
168  - name: install window manager and desk env []
177  - name: install i3-style []
182  - name: config vim
183  block:
184    - file:
185      src: /home/user/dev/arch/config/{{ item }}
186      dest: /home/user/{{ item }}
187      state: link
188      with_items:
189        - .vimrc
190    - file:
191      path: /home/user/.vim/bundle
192      state: directory
193    - git:
194      repo: 'https://github.com/VundleVim/Vundle.vim.git'
195      dest: /home/user/.vim/bundle/Vundle.vim
196  - expect:

```

Рисунок 1.1 – Пример Ansible-плейбука

Самым важным моментом при проектировании инфраструктуры являются сервисы, ПО и задачи, которые они будут выполнять. Эффективность итоговой инфраструктуры в большей части определяется не новизной и новыми технологиями, а наличием отказоустойчивых базовых элементов, которые обеспечивают жизнеспособность всего ПО и иногда аппаратной части инфраструктуры. Ключевая особенность такой архитектуры заключается в способности обеспечивать непрерывность процессов при сохранении гибкости для адаптации к изменяющимся условиям.

К обязательным компонентам любой инфраструктуры в первую очередь относят, конечно, IP-адресацию и организацию сети. Обосновывать такое, в целом, бессмысленно. Без нее ничего в ИТ-инфраструктуре работать никогда не будет.

Организация сети начинается с проектирования её логической структуры. Основой служит разделение на подсети – либо по VLAN, либо через физическую изоляцию. Например, отдельная подсеть для серверов, клиентских устройств и управляющего оборудования. Маршрутизация между ними настраивается на L3-коммутаторах или маршрутизаторах, где прописываются правила доступа (ACL). Для защиты периметра используются межсетевые

экраны, блокирующие нежелательный трафик по портам или IP-адресам. Внутри сети часто применяется 802.1X для аутентификации устройств: если компьютер не прошел проверку – доступ в VLAN ограничивается. Отдельное внимание уделяется резервированию: два независимых маршрутизатора с VRRP или HSRP гарантируют, что выход одного из строя не парализует инфраструктуру. Для Wi-Fi сетей аналогично – несколько точек доступа с общим SSID и быстрым роумингом. Всё это требует строгой документации: схемы адресации, таблицы VLAN, политики безопасности. Без этого даже незначительное изменение конфигурации может привести к коллизиям или уязвимостям.

Продолжая тему IP-адресации и организации сети, нельзя упустить DHCP – протокол динамической настройки хостов. Позволяет автоматически присваивать настройки сети хостам, которые отправляют запросы на получение адреса. В основном используются для устройств, которые по большей части принимают трафик, а не получают. Обуславливается это тем, что подобным устройствам в сети нужно чего-то получать, делать что-то и после куда-то опять отсылать данные. Под такую роль подходят именно различные клиентские ПК. Также для связи с каким-то устройством надо знать его IP-адрес (или доменное имя, которое тоже преобразуется в IP-адрес), что при получении адреса через DHCP достаточно затруднительно. Эту проблему могут решать DDNS и статические адреса, выдаваемые по MAC-адресам.

На этом моменте инфраструктура, пускай и откровенно плохая – но будет работать. Хорошим тоном всегда будет настроить систему доменных имен. Задача простая – переводить доменное имя в IP-адрес, чтобы никаких лишних чисел с точками работникам запоминать не приходилось. Есть много других функций, по типу DNSSEC, TXT-записей, которые передают метаданные, но они играют роль обеспечения работоспособности других сервисов, например – почтовых серверов (MX и SPF-записи). Как правило, имена присваиваются только сервисам и серверам, к которым часто приходится обращаться – локальное ПО для конференций, сервисы, взаимодействие с которыми происходит через веб-интерфейс. Для обычных рабочих станций такое обычно не настраивается, они только получают различные записи с DNS-серверов. Но в случае, если такое вдруг понадобится – DDNS снова в помощь. Это функция, которая позволяет DHCP серверам при очередной аренде вносить на один или несколько DNS-серверов изменения. Составляются они из имени устройства и обслуживаемой зоны.

Иногда администратору не приходится вручную настраивать DNS – этим занимается контроллер домена. Это как раз тот сервис, который не может работать без доменных имен. Добавляются служебные SRV записи, которые по большей части необходимы для Kerberos-аутентификации. Сам домен строится в большинстве случаев из LDAP (места, где хранятся все объекты домена), DNS и Kerberos. В случае использования AD-совместимого домена, все это

разворачивается автоматически. И записи для всех клиентов, и в том числе, что немаловажно – централизованная аутентификация. Существуют и другие реализации: FreeIPA и ее производные (ALD PRO). Эти системы в основном ориентированы на Linux и Unix, но и Windows поддерживают. Также контроллеры домена реализуют общие сетевые папки (netlogon, например), политики доступа к ПК, сетевым папкам и почти всему, что есть в используемом домене. Доменов может быть несколько – корневой, дочерний, несколько дочерних, у которых в свою очередь еще пару дочерних доменов. Задачу аутентификации в таких решают доверительные отношения. Администратор вручную указывает, какие домены могут передавать данные от клиента на другие контроллеры домена.

Вслед за доменом и сетевыми папками приходит проблема износа комплектующих в серверах и в особенности – накопителей. Эту проблему успешно решают RAID-массивы и резервные копии на другие сервера. Проблему, когда из строя выходит сервер с резервными копиями, решают незамысловато – третьим сервером. Продуктов, которые предоставляют функции резервного копирования – несметное множество. От rsync до проприетарного ПО с немыслимым функционалом.

С вышеописанным функционалом работает огромное большинство организаций, будь то облачная инфраструктура или традиционная. Все что выстраивается дополнительно необходимо для обеспечения работы ПО специфичного для определенной отрасли организации или для ускорения каких-то процессов (любое ПО для этого и служит). Но даже резервные копии и RAID не спасут от проблем, которые не видны.

Решения вроде Zabbix или связки Prometheus с Grafana – обязательный минимум для организаций покрупнее. Они следят за нагрузкой на процессор, диски, сеть, предупреждают о критических сбоях или аномалиях. За логами тоже следят – через Elasticsearch или Graylog. Без этого админ узнает о проблеме только когда сотрудники начнут негодовать.

Следующий шаг для оптимизации инфраструктуры – уменьшить количество физических серверов. Не полностью, но большую часть можно заменить виртуальными машинами. Виртуализация дает несколько плюсов: можно быстро масштабировать ресурсы, мигрировать VM между хостами, изолировать сервисы. Да и с точки зрения энергопотребления такой подход гораздо выгоднее.

Если уж говорить о виртуализации – нельзя обойти гипервизоры. Proxmox здесь вне конкуренции для небольших и средних инфраструктур. Весь функционал – live-миграция, распределенное хранилище (CERN), HA-кластеры и прочее идут в составе по умолчанию. Достаточно настроить ноды, прописать общее хранилище – и падение одного хоста останется незамеченным для пользователей. Также имеется встроенная сетевая организация: мосты, VLAN,

SDN. Конечно, SDN в Proxmox не может конкурировать с Cisco ACI, но для базовых сценариев функционала хватает. Например, разделить трафик виртуальных машин от управляющего трафика гипервизоров или создать изолированные подсети для тестовых стендов.

Когда инфраструктура начинает достаточно разрастаться, появляются проблемы масштабирования и организации сети. Обычно эти проблемы решаются самыми первыми – еще на этапе создания примерного макета сети, путем выбора одной из распространенных сетевых архитектур.

Стандартная трехуровневая архитектура (также известная как иерархическая модель) – базовая модель для большинства корпоративных сетей. Выделяют уровни ядра, распределения и доступа. Ядро отвечает за маршрутизацию между подсетями, распределение – за агрегацию трафика и применение политик (ACL, QoS), доступ – за подключение конечных устройств. Такая архитектура масштабируется вертикально: добавление новых коммутаторов на уровне доступа не затрагивает ядро. Однако при росте устройств возникают узкие места – например, STP-блокировки или перегрузка L3-интерфейсов. Пример такой архитектуры представлен на рисунке 1.2.

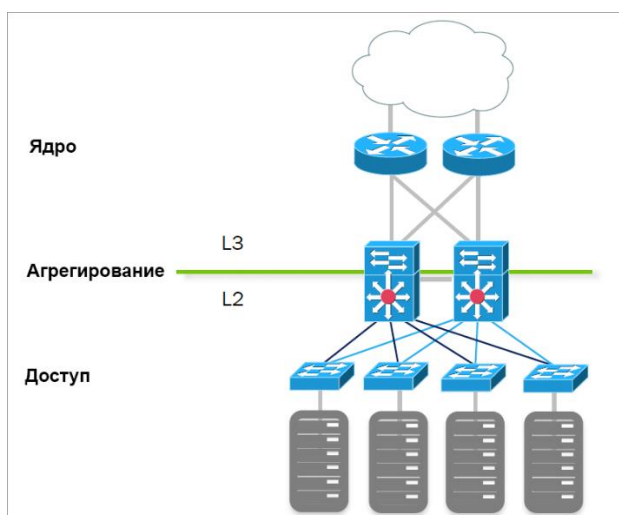


Рисунок 1.2 – Иерархическая модель сети

Leaf-spine – архитектура для ЦОД и высоконагруженных сред. Все leaf-коммутаторы (доступ) соединены с каждым spine-коммутатором (ядро), образуя сетку с фиксированной задержкой. Маршрутизация – только L3, что исключает петли и STP. Плюсы: горизонтальное масштабирование, балансировка нагрузки через ECMP, отказоустойчивость (выход из строя spine-узла не влияет на связность). Из минусов такого подхода выделяют стоимость каждого порта на коммутаторах и невероятную сложность миграции с устаревших сетей. Подходит для кластеров и распределенных СХД. Пример такой архитектуры представлен на рисунке 1.3 [6].

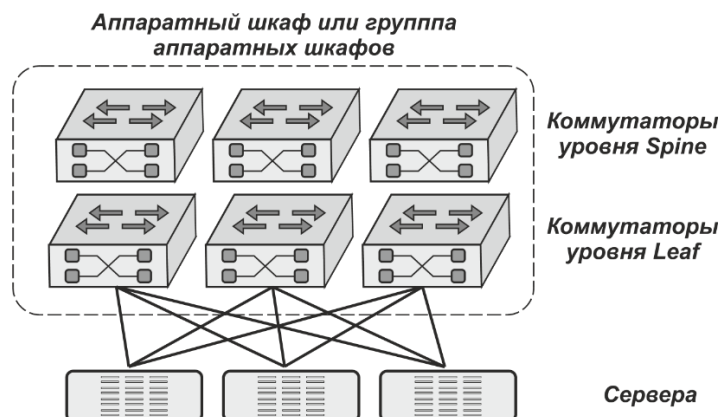


Рисунок 1.3 – Архитектура leaf-spine

Гибридные модели – комбинации подходов. Например, трехуровневая архитектура в офисе и leaf-spine в ЦОД. Или spine-уровень на основе VXLAN, что позволяет расширить L2-домены поверх L3-сети. Пример гибридной модели представлен на рисунке 1.4.

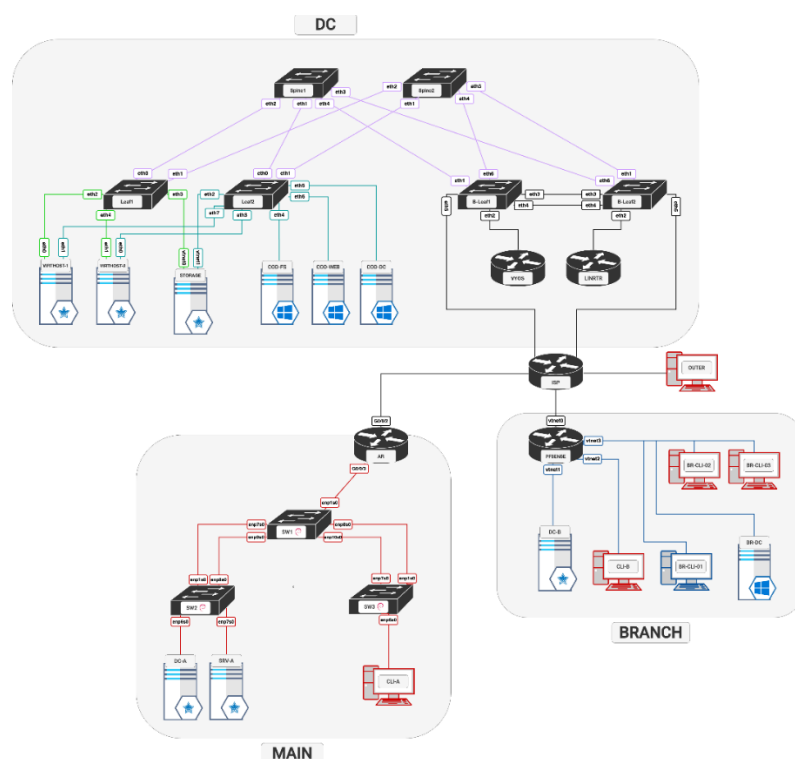


Рисунок 1.4 – Гибридная архитектура

Такие решения часто встречаются в гибридных облаках, где часть инфраструктуры размещена локально, часть – у облачного провайдера.

1.4 Безопасность инфраструктуры

Информационная безопасность направлена на обеспечение конфиденциальности, целостности и доступности данных. Безопасность ИТ-инфраструктуры предполагает комплексную защиту всех её элементов: сетей, серверов, центров обработки данных, рабочих

станций и клиентских устройств, где каждый компонент инфраструктуры должен быть изолирован и контролируем, поскольку сбой или компрометация одной части могут вызвать цепную реакцию по всем элементам сети. Ключевыми принципами являются разграничение прав доступа, многоуровневая защита и постоянный мониторинг.

Угрозы безопасности ИТ-инфраструктуры разнообразны и включают в себя как внешние, так и внутренние факторы. Основными источниками угроз являются киберпреступники, стремящиеся нарушить работу систем и похитить данные, а также сотрудники организации, умышленно или по неосторожности использующие свои привилегии во вред компании. В совокупности именно действия персонала являются причиной примерно 66% всех инцидентов безопасности, причём для 14% инцидентов сотрудники становятся «точкой входа» для внешних атак. Среди классических видов угроз выделяются:

- несанкционированный доступ к конфиденциальной информации (например, через взлом пароля или уязвимость системы);
- вредоносное ПО (вирусы, шпионские программы, программы-вымогатели);
- атаки на отказ в обслуживании, фишинг и социальная инженерия;
- аппаратные и программные сбои (отказы серверов, сбои накопителей, сбои электропитания и т.п.).

Целевые атаки могут быть направлены на нарушение конфиденциальности данных, нарушение их целостности (например, удаление или искажение информации) или доступности ресурсов и служб предприятия. Полноценная защита инфраструктуры требует анализа всех перечисленных типов угроз и своевременного реагирования на инциденты.

Современные системы безопасности инфраструктуры включают круглосуточный мониторинг и анализ событий специалистами по ИБ. К основным техническим средствам защиты ИТ-инфраструктуры относятся [5]:

- сегментация сети. Локальная сеть разбивается на изолированные сегменты с ограниченным обменом данными между ними. При таком подходе компрометация одного сегмента слабо затрагивает другие, что существенно снижает риски распространения атаки. Например, отдельные VLAN или подсети для административного, пользовательского и гостевого трафика. Такой многоуровневый доступ не обеспечивает полного разделения ресурсов, но все-таки позволяет внедрить другие инструменты для проверки и разграничения трафика;
- межсетевые экраны и фильтрация трафика. На границах периметра и между сегментами устанавливаются МЭ, анализирующие пакеты на основе установленных правил. Новое поколение фаерволов (NGFW) может фильтровать трафик на уровне приложений, применять DPI и интегрироваться с IDS или IPS. МЭ ограничивает входящий/исходящий трафик

по источнику, назначению, протоколу и содержимому, предотвращая неавторизованные соединения. В сочетании с VPN он защищает удалённый доступ сотрудников к ресурсам организации;

- антивирусы и EDR. На рабочих местах и серверах устанавливаются антивирусные и EDR-системы, обнаруживающие известные вирусы, сложные вредоносные программы, а также отслеживающие подозрительное поведение;

- системы обнаружения и предотвращения вторжений (IDS/IPS). IDS/IPS анализируют сетевой трафик в режиме реального времени, выявляя атакующие активности и аномалии. По сравнению с «традиционными» средствами (антивирус, межсетевые экраны), IDS/IPS обеспечивают более высокий уровень защиты сети. Важна комбинированная установка: сети внутреннего периметра, периметра ЦОД, конечных машин, что позволит обнаруживать сложные угрозы, даже если злоумышленник уже прошёл начальный рубеж;

- шифрование данных. Данные важно защищать как «в покое», так и «в движении». Для передачи информации следует применять защищённые каналы (TLS/SSL или VPN, или все вместе) с современными алгоритмами (в приоритете, конечно, отечественные стандарты и алгоритмы). Хранение конфиденциальных данных должно выполняться в зашифрованном виде. Также все больше распространяется использование сертифицированных криптографических модулей (по типу «КриптоПро») для обеспечения криптографической защиты, даже в организациях без требований к защите такого уровня;

- системы контроля доступа и аутентификации. Всегда необходимо жёстко ограничивать привилегии пользователей по принципу наименьших прав. Для этого используют централизованные решения для управления идентификацией (например, отечественные IdM/IGA-системы – Solar inRights или аналогичные), которые позволяют реализовать ролевой подход и автоматизировать выдачу и отзыв прав. Обязательны надёжные политики паролей, двухфакторная аутентификация для критичных ресурсов и журналы аудита. Также в совокупности с предыдущими практиками приветствуется регулярный анализ логов (с применением SIEM) для выявления попыток НСД и нарушения политик безопасности.

При проектировании инфраструктуры предпочтительно использовать отечественное или открытое программное обеспечение. Например, ОС Astra Linux SE имеет сертификат соответствия ФСТЭК России (запись №2557 в реестре) и соответствует профильной защите ОС первого класса. Также стоит упомянуть операционную систему Alt Linux СП, которая также имеет встроенные средства защиты информации, сертифицирована ФСТЭК (№ 3866). Вместо проприетарного ПО можно внедрять Zabbix для мониторинга сети и серверов (имеет открытый

исходный код), OpenVPN для построения VPN и так далее, вообще все решения во всех сферах имеют отечественные или открытые аналоги.

Говоря об операционных системах, с 2017 введены стандарты, разработанные ФСТЭК, которые касаются операционных систем, предназначенных для функционирования в условиях, где необходима повышенная безопасность обрабатываемых данных. Это означает, что теперь любая ОС, если она претендует на использование в государственных или критически важных системах, должна соответствовать конкретным требованиям, которые описаны в нормативной базе. Собственно, с этого момента в реестре сертифицированных средств защиты информации стали появляться операционные системы, соответствующие этим требованиям. На сегодняшний день в перечне таких продуктов насчитывается 18 ОС, причём 15 из них получили сертификаты по схеме, которая допускает серийное производство и реализацию.

ФСТЭК предложил следующую классификацию операционных систем, разделяя их на три ключевых типа, исходя из характера задач и особенностей среды применения:

- тип «А» включает в себя так называемые универсальные или, проще говоря, операционные системы общего назначения. Под этим термином скрываются привычные системы, которые используются в рабочих станциях, серверном оборудовании, а также на мобильных устройствах. Их основное назначение – обеспечить гибкое выполнение широкого спектра пользовательских задач;

- тип «Б» охватывает встраиваемые операционные системы. Эти ОС проектируются не для повседневного взаимодействия пользователя с техникой, а для жёстко регламентированных функций в составе какого-либо оборудования. То есть они «живут» внутри систем видеонаблюдения, промышленных контроллеров, систем управления на транспорте и так далее. Грубо говоря, везде, где от системы требуется стабильность и предсказуемость в условиях ограниченных ресурсов;

- тип «В», в свою очередь, представляет собой операционные системы реального времени (ОСРВ). Их особенность в том, что они должны гарантированно реагировать на события в строго заданный промежуток времени. Зачастую подобные системы применяются в авиации, оборонной сфере или в робототехнике, где задержка даже в миллисекунду может привести к сбоям или авариям.

Каждая операционная система в рамках требований ФСТЭК получает определённый класс защищённости. Эти классы, которых всего шесть, представляют собой своего рода «рейтинговую шкалу», где шестой класс – это минимально допустимый уровень защиты, а первый – максимальный:

– 6 класс предназначен для информационных систем, не предполагающих хранения или обработки особо чувствительных данных. Такие ОС могут использоваться в государственных системах 3 или 4 классов защищённости, в АСУ ТП 3 класса, а также в информационных системах персональных данных с необходимостью 3 и 4 уровней защищённости;

– 5 класс применяется в более ответственных системах, например, во 2 классе защищённости государственных информационных систем, аналогичных по уровню АСУ ТП, и в ИСПДн с 2 уровнем защищённости;

– 4 класс относится к системам, где обрабатываются особенно важные данные. Это может быть 1 класс защищённости для государственных ИС, автоматизированные системы критических объектов, а также ИСПДн с максимальным, 1 уровнем защищённости;

– классы 1–3 уже предполагают работу со сведениями, составляющими государственную тайну. Их применение требует соответствующего допуска и особого порядка работы с такими системами.

Технических средств защиты недостаточно без адекватных организационных мер. Необходимо разработать и внедрить политику информационной безопасности, определяющую правила работы с ресурсами, а именно требования к учётным записям, классификация информации, процедуру инцидент-менеджмента, ответственность сотрудников. Политику необходимо регулярно пересматривать и актуализировать. Важные положения нужно закрепить во внутренних документах (регламентах, инструкциях) и доводить до персонала.

Не менее важна обученность сотрудников. Из статистики, приведенной ранее, нужно вспомнить, что 66% всех инцидентов ИБ вызваны непреднамеренными действиями сотрудников. Это как раз так и подчёркивает, что без осведомленности пользователей в вопросах защиты информации обеспечить ее невозможно. Например, регулярное обучение и повышение осведомлённости о киберугрозах (фишинг, инженерия, безопасная работа в сети) существенно снизит долю ошибок из-за человеческого фактора. В программу обучения включают изучение корпоративных процедур ИБ, правил работы с удалёнными подключениями и средствами обмена информацией. Как правило, процедуры обучения рабочего персонала по темам безопасности информации проводятся регулярно и иногда сопровождаются работами по проверке знаний.

Также хорошей, но весьма трудной практикой является определение прав доступа и ролевой подход. Сотрудникам назначают только необходимые для работы полномочия, исключая одновременное владение конфликтующими правами (также называют SoD – разделение обязанностей). Желательно наличие периодических аудитов учётных записей и ревизия выданных прав. Важной практикой является ведение журналов действий и их анализ. Только

таким образом возможно выявить злоупотребление привилегиями и своевременно отреагировать на подозрительные события.

Наряду с организационными и физическими мерами защиты в ИТ-инфраструктуре критически важно соблюдать законодательные требования по защите информации. В РФ действует множество нормативных актов, определяющих права и обязанности участников информационных процессов, способы защиты данных и ответственности за их нарушение, но рассмотрены будут только некоторые основные.

Федеральный закон №152-ФЗ «О персональных данных» - этот закон регулирует сбор, хранение и обработку персональных данных граждан. Под персональными данными понимается любая информация, непосредственно или косвенно относящаяся к конкретному физическому лицу (субъекту персональных данных). Закон вводит понятие оператора – лица (госоргана, компании или ИП), самостоятельно или совместно обрабатывающего персональные данные и определяющего цели такой обработки. Оператор обязан обрабатывать ПД только законным образом, в частности – на основании явного согласия субъекта или других установленных законом оснований

Перед сбором данных оператор должен информировать субъектов персональных данных о целях и условиях обработки, указывать своё наименование и адрес, перечень собираемых данных и права субъекта (например, право отозвать согласие). При необходимости получения согласия оператор обязан разъяснить гражданину последствия отказа. Если же персональные данные получены не от самого субъекта, оператор до начала обработки должен уведомить его обо всех указанных сведениях. Таким образом обеспечивается принцип прозрачности и информированности.

Закон закрепляет строгие требования к защите персональных данных. Оператор при обработке обязан принимать необходимые правовые, организационные и технические меры для защиты ПД от несанкционированного доступа, утраты, уничтожения и иных неправомерных действий. В частности, требуется выявлять угрозы безопасности в информационных системах, применять сертифицированные средства защиты, оценивать эффективность мер до ввода системы в эксплуатацию и контролировать любые инциденты. Оператор также обязан обеспечить быстрое восстановление данных в случае нарушения их целостности. Если обработка персональных данных ведётся в рамках государственных функций или оказания услуг (например, базы клиентов или работников), Базы данных должны располагаться на территории России, а передача данных за рубеж допускается лишь при соблюдении особых условий и защиты.

Его влияние на ИТ-инфраструктуру особенно заметно в части локальных систем хранения и обработки данных. Закон требует, чтобы информационные системы, обрабатывающие

персональные данные граждан РФ, находились на территории Российской Федерации, что ограничивает использование зарубежных серверов, хостинга и средств телекоммуникации. Это делает обязательным развёртывание собственных серверов или аренду сертифицированных дата-центров внутри страны. Дополнительно необходимо реализовать физическую и программную изоляцию сегментов сети, где производится обработка ПДн, от других элементов ИТ-инфраструктуры.

Закон также влечёт за собой необходимость закупки и внедрения сертифицированного ПО и технических средств защиты информации, соответствующих требованиям ФСТЭК и ФСБ. Это касается, например, межсетевых экранов, средств шифрования, антивирусной защиты, журналирования и резервного копирования. Всё это увеличивает общую стоимость сопровождения локальной ИТ-инфраструктуры и потребует отдельного учета при проектировании систем.

Федеральный закон №149-ФЗ «Об информации, информационных технологиях и о защите информации» – этот закон устанавливает общие принципы правового регулирования информационных отношений. Под информацией он понимает любые сведения (данные, сообщения) независимо от формы представления. Информационные технологии – это процессы поиска, сбора, хранения, обработки и передачи информации consultant.ru. Закон определяет понятие информационной системы как совокупность информации в базах данных и средств её обработки (программные и технические). Оператор информационной системы – это обычно её владелец или организация, эксплуатирующая оборудование и базы данных.

Закон вводит понятие конфиденциальности информации – обязательство лица, получившего доступ к данным, не передавать их третьим лицам без разрешения владельца. Он различает общедоступную и ограниченную информацию, устанавливает порядок ограничения доступа и обеспечения открытости там, где это необходимо (например, экологические сведения должны быть доступными). Также закон диктует основные требования к защите информации. Защита понимается как комплекс правовых, организационных и технических мер, направленных на:

- предотвращение несанкционированного доступа, искажения или уничтожения данных;
- соблюдение конфиденциальности ограниченной информации;
- реализацию права на доступ к сведениям.

Этот закон прямо требует, чтобы базы данных, содержащие персональные данные граждан РФ, находились на территории страны. Приказы ФСТЭК России (в частности, №17, №21, №235, №239, №76) конкретизируют эти общие требования, устанавливая правила

классификации информации, порядок организации защиты локальных ИТ систем, процедур аудита и санкции за несоблюдение.

Если рассматривать его роль в рамках только проектирования инфраструктуры, то он влияет на архитектуру и эксплуатацию, так как требует обеспечения целостности, конфиденциальности и доступности информации в любых ИС. В локальных ИС это требует детальной проработки механизмов разграничения прав доступа, шифрования данных, устойчивости к отказам оборудования и обеспечения физической безопасности серверов.

Также закон ограничивает использование неконтролируемого программного обеспечения. В рамках исполнения требований закона в государственных и корпоративных инфраструктурах запрещается использование несертифицированного ПО, что требует полной инвентаризации установленного программного обеспечения и соответствия требованиям ФСТЭК.

Федеральный закон №187-ФЗ «О безопасности критической информационной инфраструктуры РФ» – посвящён защите объектов критической информационной инфраструктуры (КИИ), то есть ИТ систем и сетей, имеющих особое значение для устойчивости общества и государства. Под КИИ понимаются совокупности объектов, информационных систем и сетей электросвязи, обеспечивающие функционирование жизненно важных сфер (энергетика, транспорт, связь, финансы, оборона). КИИ делится на значимые объекты – те, что имеют присвоенную категорию значимости и включены в реестр значимых объектов КИИ РФ.

Закон устанавливает категорирование объектов КИИ: каждому объекту присваивается одна из трёх категорий (первая, вторая, третья) по степени его критичности. Критерии значимости включают оценки потенциального ущерба здоровью людей, функционированию инфраструктуры, экономике, обороне и прочим важным факторам. Ответственность за проведение категорирования несут субъекты КИИ – государственные органы и организации, владеющие или эксплуатирующие такие объекты.

Субъекты КИИ обязаны соблюдать особые правила безопасности и информировать государство о происшествиях. В частности, они незамедлительно уведомляют уполномоченный федеральный орган о любых компьютерных инцидентах. Для значимых объектов вводятся ещё более жёсткие требования: необходимо выполнять предписания по безопасности, проводить регулярный аудит, обеспечивать восстановление работоспособности в случае инцидентов и допускать проверяющих на объекты в любой момент. В совокупности с остальными приведенными законами влияние на инфраструктуру остается аналогичным.

1.5 Требования для построения инфраструктуры

Главным специалистом отдела аналитики и архитектуры государственной единой облачной платформы и государственной информационной системы также был предоставлен перечень требований, на основании которых необходимо спроектировать и реализовать ИТ-инфраструктуру. Требования охватывают несколько ключевых направлений: общая структура, организация удалённого доступа, информационная безопасность, отказоустойчивость, сетевая архитектура, а также техническое наполнение макета.

Прежде всего, указано, что все базовые и необходимые сервисы, используемые исключительно внутри подразделения, должны быть развернуты локально. Это подчёркивает автономный характер разрабатываемой инфраструктуры и исключает необходимость интеграции с основными ресурсами организации.

В части удалённого доступа поставлена задача обеспечить отдельный канал подключения сотрудников, ограниченный только локальными ресурсами отдела. Отдельное внимание уделено требованию совместимости со всеми дистрибутивами, используемыми в компании, что предполагает кроссплатформенную реализацию соответствующих механизмов.

Что касается фильтрации трафика, в требованиях зафиксирована необходимость использования NGFW для контроля клиентского трафика. Подчёркнута важность блокировки запрещённого и нежелательного контента. Подход к формированию правил должен быть основан на принципе запрета по умолчанию: разрешается только явно определённый трафик, всё остальное блокируется.

Отказоустойчивость – ещё один важный аспект. В требованиях обозначена необходимость обеспечить устойчивость хранения данных и систем виртуализации. При этом все специализированные сервисы должны быть развернуты в виртуальной среде, что позволяет добиться гибкости и масштабируемости. В качестве службы единого подключения пользователей (ЕПП) предписано использовать ALD PRO.

Сетевые требования предполагают использование трёхуровневой архитектуры либо её упрощённой вариации без уровня распределения. Обязательно должна быть предусмотрена возможность горизонтального масштабирования, что особенно важно при дальнейшем расширении инфраструктуры. Для сотрудников, задействованных в разработке, должны быть развёрнуты сервисы для ведения документации. Также предполагается создание центра сертификации, необходимого для выпуска сертификатов, используемых при разработке и тестировании программных решений.

Сеть необходимо сегментировать на функциональные зоны, включая сегменты для клиентского трафика, административных нужд, репликации и серверного взаимодействия. Трафик виртуальных машин рассматривается как часть серверного сегмента. Оборудование для подключения конечных устройств должно иметь избыточный запас портов, что обеспечит гибкость при эксплуатации. Архитектура сети в целом должна следовать иерархическому принципу построения.

Отдельные указания касаются наполнения макета: предполагается минимизация количества виртуальных машин с целью снижения нагрузки. Допускается абстрагирование от фактического числа рабочих мест, а также игнорирование будущего физического размещения сетевого оборудования – как активного, так и пассивного.

Подводя итог всем представленным требованиям, можно сказать, что они задают общую концепцию для построения локальной ИТ-инфраструктуры, ориентированной на демонстрацию ключевых технических решений в области сетевой архитектуры. При этом важно понимать, что разрабатываемый проект носит характер макета, основная цель которого – отработка архитектурных подходов и проверка работоспособности выбранных решений в контролируемой среде.

Многие аспекты, такие как фактическое количество рабочих мест, физическое размещение оборудования, а также полное соответствие корпоративным стандартам, сознательно опущены. Именно такой подход позволяет сосредоточиться на логике построения и взаимодействия компонентов, а не на их физической реализации. Предусматривается, что отдельные элементы инфраструктуры могут быть реализованы в упрощённом или условном виде, поскольку проект не предполагает немедленного переноса в физическую среду. Основной акцент нужно сделать на моделировании, тестировании и демонстрации потенциальных решений, которые в реальных условиях могут быть как реализованы, так и адаптированы или отклонены.

1.6 Выбор лабораторной среды для создания макета

Для создания и тестирования сетевых макетов в рамках проекта необходима лабораторная среда, обеспечивающая реалистичную эмуляцию или симуляцию сетевой инфраструктуры. Выбор подходящей платформы зависит от множества факторов, включая функциональные возможности, требования к ресурсам, совместимость с реальными устройствами, уровень сложности настройки и поддержки, наличие встроенных инструментов для анализа сетевого трафика и диагностики. Основные инструменты, примерно подходящие под требования описаны в этом разделе.

Cisco Packet Tracer – Это симулятор, разработанный Cisco для обучения базовым сетевым концепциям. Он имитирует поведение устройств Cisco (роутеры, коммутаторы) через упрощенные алгоритмы. Внутренняя архитектура основана на визуальном моделировании логических и физических слоев сети, включая автоматическую генерацию кабелей и упрощенную эмуляцию протоколов (RIP, OSPF, DHCP и прочие протоколы). Каждое устройство – программная модель с ограниченным набором команд, которые не включают полный функционал аналогичного физического устройства.

Из явных плюсов выделяются:

- относительная простота использования;
- поддержка лабораторных работ внутри интерфейса;
- не требователен к ресурсам.

Минусов слегка больше:

- изолированность в экосистеме Cisco;
- отсутствие возможности взаимодействия со сторонними программами;
- ограничения на количество узлов в одном проекте;
- не бесплатен;
- только достаточно устаревшие модели оборудования.

Переходя к следующим симуляторам, можно упомянуть малоизвестный Mininet. Это эмулятор сетей, использующий встроенные средства Linux и OpenFlow для создания виртуальных устройств. На всех устройствах в рамках проекта можно полноценно пользоваться утилитами из ядра.

С основной задачей в построении сети этот инструмент справится, но с нюансами. Для сетевых устройств используются OpenFlow-контроллеры, что встречается только в SDN и особой популярностью не пользуются и получается, что и пользы в реализации данного проекта – не несет.

Следующий инструмент имеет подобный функционал и гораздо большее применение. ContainerLab – инструмент, который также используется для создания и тестирования сетей. Топология строится не в графическом интерфейсе, а описывается в yaml файле, почти идентичному файлу docker-compose, а после управляется через терминал.

Инструмент воссоздает нужную сетевую инфраструктуру используя docker-контейнеры и стандартные средства по типу виртуальных пар интерфейсов и мостов в Linux. Так как в docker'е можно запустить практически все что угодно, за исключением специфических устройств, то и в этом инструменте их тоже можно использовать. В поддержке имеются все самые используемые linux-дистрибутивы, также есть возможность запуска сетевого оборудования Cisco,

Huawei (сильно урезанные подобию VRP), MikroTik, Arista и многих других. Пример использования представлен на рисунке 1.5.

```
antonr@clab:~/clab/Habr$ clab deploy --topo ros.clab.yml
INFO[0000] Containerlab v0.31.0 started
INFO[0000] Parsing & checking topology file: ros.clab.yml
INFO[0000] Creating lab directory: /home/antonr/clab/Habr/clab-lab
INFO[0000] Creating docker network: Name="statics", IPv4Subnet="172.20.20.0/24", IPv6Subnet="", MTU="1500"
INFO[0000] Creating container: "ubuntu"
INFO[0000] Creating container: "ros1"
INFO[0000] Creating container: "alpine1"
INFO[0000] Creating virtual wire: ros1:eth2 <--> alpine1:eth1
INFO[0000] Creating virtual wire: ubuntu:eth1 <--> ros1:eth1
INFO[0001] Adding containerlab host entries to /etc/hosts file
INFO[0064] Executed command 'bash /tmp/setup.sh' on ubuntu. stdout:
* Starting OpenBSD Secure Shell server sshd
...done.
INFO[0064] Executed command 'bash /tmp/setup.sh' on ubuntu. stderr:
debconf: delaying package configuration, since apt-utils is not installed
INFO[0072] Executed command 'sh /tmp/setup.sh' on alpine1. stdout:
ssh-keygen: generating new host keys: RSA DSA ECDSA ED25519
INFO[0084] Executed command 'sh /tmp/setup.sh 192.168.122.1 10.2.2 10.2.3' on ros1. stdout:
Waiting for ssh: . SSH UP/ip dns set servers=192.168.122.1;
/ip address add address=10.2.2.1/24 interface=ether2 network=10.2.2.0;
/ip address add address=10.2.3.1/24 interface=ether3 network=10.2.3.0;
ip route add distance=1 gateway=172.31.255.29;
INFO[0084] Executed command 'sh /tmp/setup.sh 192.168.122.1 10.2.2 10.2.3' on ros1. stderr:
debconf: delaying package configuration, since apt-utils is not installed
Warning: Permanently added '172.31.255.30' (RSA) to the list of known hosts.
connection to 172.31.255.30 closed.
```

#	Name	Container ID	Image	Kind	State	IPv4 Address	IPv6 Address
1	alpine1	f7144234a70d	alpine:latest	linux	running	172.20.20.31/24	N/A
2	ros1	02bca5d68ab9	iparchitects/chr:long-term	vr-ros	running	172.20.20.11/24	N/A
3	ubuntu	0acaf4792b7b	ubuntu:latest	linux	running	172.20.20.21/24	N/A

Рисунок 1.5 – Пример использования ContainerLab

Из интересных моментов также выделяется возможность запуска Windows, но только сильно странных редакций для интернета вещей. Ресурсов оно расходует очень мало, так как не требуется полная виртуализация устройств, они работают просто как отдельный процесс в системе. В рамках проекта следующий пункт роли не играет, но также имеется возможность интеграции в процесс автоматического развертывания при разработке какого-либо продукта в команде.

Подводя итог, инструмент имеет потенциал, в виде:

- быстрого развертывания проекта;
- отличной интеграции с облачными технологиями;
- ресурсоемкости;
- большой поддержки числа производителей.

Минусов слегка больше:

- непрактичное управление;
- сложность управления и построения больших сетей;
- малая функциональность в сравнении с другими продуктами.

Следующий эмулятор – EVE-NG. Представляет из себя набор инструментов для работы с виртуальными устройствами, построением сетей, коммутацией с реальным оборудованием. Возможности данного продукта позволяют легко использовать, управлять, коммутировать моделируемое сетевое оборудование. Имеет две редакции – бесплатную, с ограничениями в виде 64 узлов в проекте (что в целом и немало) и платную – без ограничений, с поддержкой

контейнеров в одном проекте с виртуальными машинами. В отличие от прошлых инструментов – используется полная виртуализация используемых устройств, что позволяет производителям в образах устройств вместить все функции, которые используются в реальном оборудовании. Пример топологии представлен на рисунке 1.6.

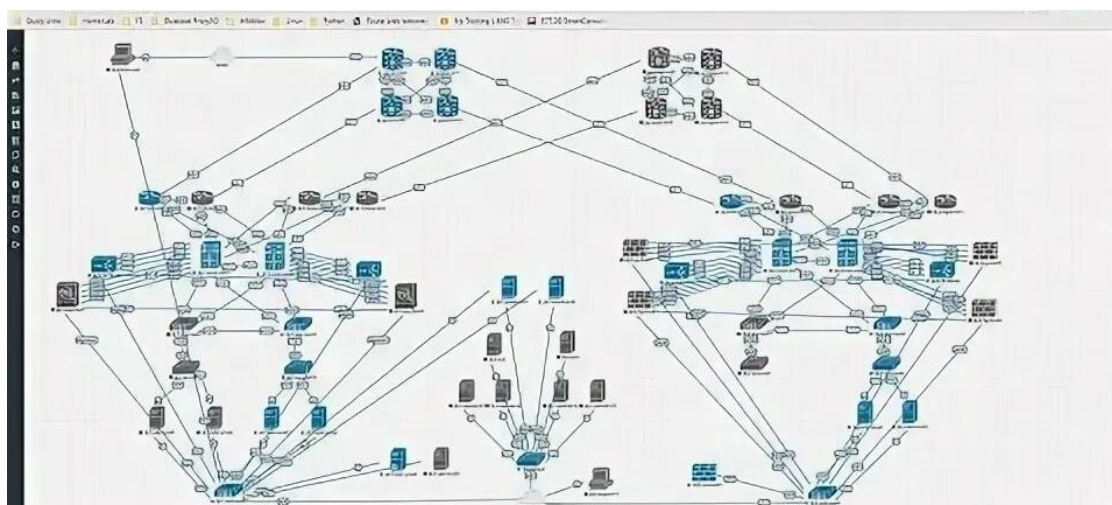


Рисунок 1.6 – Пример топологии в EVE-NG

Продукт сильно серьезный, так как до определенного момента использовался в исследованиях ФСТЭК в рамках исследований и тестирований уязвимостей. Имеет поддержку вообще всех производителей, которые представляют программные образы своих устройств, используется через веб-интерфейс, а управляется – через терминал. Также позволяет прослушивать весь трафик на абсолютно всех соединениях, которые имеются в проекте, через WireShark. Интересный нюанс - при должном понимании работы EVE-NG можно прекрасно интегрировать проект с физической инфраструктурой, даже в случае, когда просто требуется проверить совместимость. Развертывается данный инструмент может в качестве виртуальной машины или сразу на физическое оборудование и поставляется в виде установщика, по типу обычной ОС.

Последний инструмент, и по мнению сотен тысяч людей – самый перспективный из всех вышеописанных – GNS3. Если дословно – графический симулятор сети, но говоря правильно – эмулятор. Инструмент абсолютно бесплатен, не имеет ограничений в сравнении с прошлым инструментом. Так как проект открытый, то и технической поддержки не предоставляется, что заставляет различные организации посмотреть в сторону EVE-NG.

Развертываться он может в качестве виртуальной машины и также в качестве хостовой ОС, но первое, что его отличает от остальных инструментов – это возможность запуска еще и на любой ОС, без необходимости развертывания виртуальной машины. Имеет полную поддержку всех функций остальных инструментов. Также выделяется возможность запуска машин в проекте на разных серверах, чем не может порадовать EVE-NG. Получается что-то вроде объединения

мощностей серверов как в облачной инфраструктуре, но только для запуска ВМ в рамках проекта. Среди всех этих преимуществ есть и минус – для построения топологий больше, чем пара устройств, потребуется неплохое количество ресурсов.

GNS3 имеет веб-интерфейс, представленный на рисунке 1.7, но достаточно скучный.

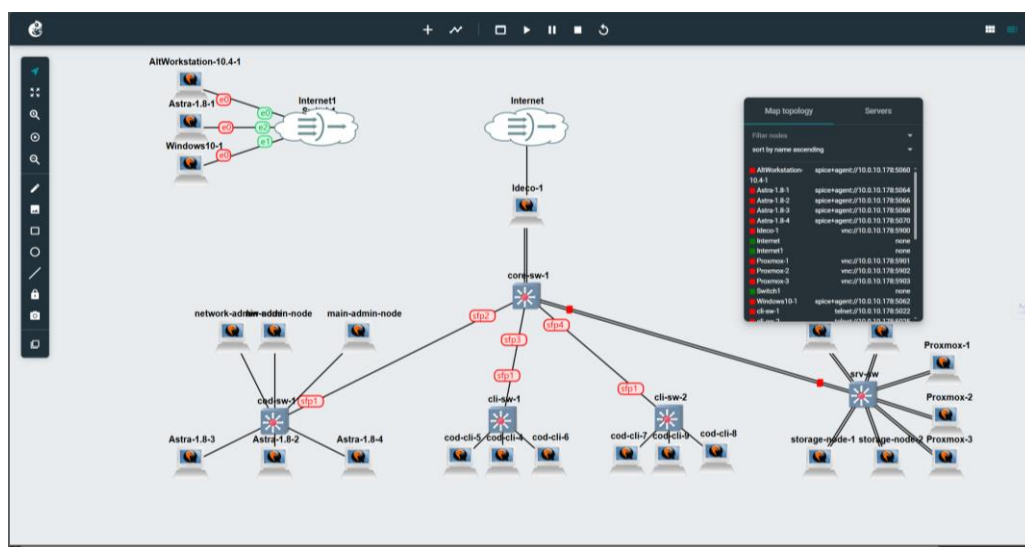


Рисунок 1.7 – Внешний вид веб-интерфейса

Лучшим вариантом использования является клиент, который включает в себя сразу и сервер. Внешний вид клиента представлен на рисунке 1.8. В одном проекте возможна совместная работа сразу нескольких людей, все изменения в проекте отображаются с минимальной задержкой.

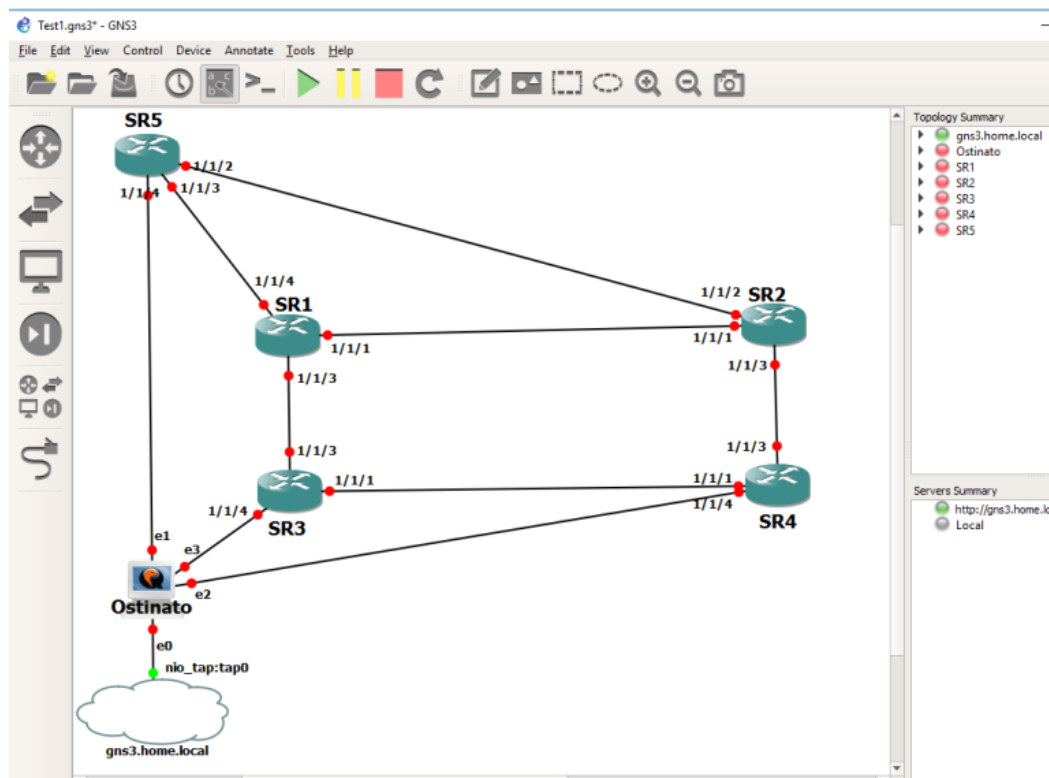


Рисунок 1.8 – Внешний вид клиента GNS3

Подводя итог – GNS3 показал существенные преимущества, что и делает его фаворитом во многих сферах. Каждый инструмент имеет свои уникальные особенности, подходящие для разных целей и условий работы. Однако, исходя из потребностей, предъявляемых к будущему проекту и характеристик рассматриваемых инструментов, можно сделать вывод, что GNS3 является наиболее подходящей платформой для создания сложных и детализированных сетевых топологий. Большое количество функциональных возможностей, поддержка полноценных ОС и возможность настройки делают его идеальной лабораторной средой.

					РК 09.02.06 401 09 ПЗ	Лист
Изм	Лист	№ докум.	Подп.	Дата		35

2 Технико-технологический раздел

2.1 Определение ПО, сервисов и производителей, подходящих под требования

Компьютеры в АРМ пользователей в этом проекте выбираются заказчиком, исходя из требований к специализированным решениям, которое не были рассмотрены в ТЗ. Выбор итогового сетевого оборудования рассматривается далее, но на стороне заказчика также остается возможность замены, с единственным требованием к поддержке технологий, которые реализуются в данном проекте. Но перед выбором сетевого оборудования стоит точнее определить сетевую архитектуру и требования к функциональности устройств.

За основу проектируемой сети будет взята архитектура Collapsed Core, несколько измененная стандартная иерархическая модель. Данное решение обусловлено следующими ключевыми факторами, делающими стандартную трехуровневую модель неоптимальной для данного проекта:

- прогнозируемый масштаб сети и объем межсегментного трафика не достигают значений, требующих выделения специализированного, высокопроизводительного и дорогостоящего уровня ядра. Использование отдельного уровня ядра при текущих требованиях привело бы к значительным неоправданным капитальным затратам на оборудование с избыточной мощностью и усложнению физической инфраструктуры;

- Collapsed Core обеспечивает существенное снижение совокупной стоимости владения – исключается закупка дорогих коммутаторов ядра, уменьшается количество необходимых устройств, оптимизируется СКС и энергопотребление, упрощается лицензирование;

- архитектура централизует критически важные функции маршрутизации между VLAN, ACL, QoS и фильтрации трафика на меньшем количестве устройств уровня распределения (функционально совмещающего роль ядра). Это значительно упрощает конфигурирование, мониторинг, администрирование сети и повышает предсказуемость ее работы.

- современные управляемые L3 коммутаторы уровня агрегации обладают достаточной производительностью на L2 и L3, портовой плотностью (включая uplink-порты по 10G), а также поддержкой необходимых технологий (маршрутизация, QoS, резервирование протоколами типа STP/RSTP/MSTP, Stacking) для эффективного выполнения функций объединенного уровня Distribution/Core в рамках заданных требований проекта.

В итоге Collapsed Core является практически единственной допустимой архитектурой для данного проекта, обеспечивающей соответствие требованиям при оптимальных затратах и управляемости, и уже на ее основе будет производиться выбор оборудования и настройка.

В качестве коммутаторов на уровень доступа в первую очередь нужно рассмотреть решения от Eltex - MES1124M (очень бюджетный вариант) и MES2324. Сравнительная таблица коммутаторов представлена в таблице 2.1.

Таблица. 2.1 – Сравнительная таблица коммутаторов

Наименование характеристики	MES1124M	MES2324	Единица измерения
Производитель	Eltex		
Общая пропускная способность коммутатора	12,8	128	Гбит/сек
Количество портов 1G (RJ-45)	0	24	Шт
Количество портов 100M (RJ-45)	24	0	Шт
Количество портов 1G (SFP)	4	0	Шт
Количество портов 10G (SFP)	0	4	Шт
Объем ОЗУ	128	512	МБ
Поддержка L3-функций	Нет	Есть	-
Исполнение	19", 1U		

Внешний вид коммутаторов представлен на рисунках 2.1–2.2.



Рисунок 2.1 – Коммутатор MES1124M



Рисунок 2.2 – Коммутатор MES2324

Так как четких данных о планируемых нагрузках представлено не было, в макете на уровне доступа будет использоваться виртуальная версия коммутатора MES1124M, без поддержки L3 функционала.

В случае изменения архитектуры с объединенным уровнем агрегации и ядра на полноценную трехуровневую модель – на уровень ядра подходит MES5324, для агрегации - MES2324F. Основное требование к уровню ядра – скорость маршрутизации и поддержка протоколов динамической маршрутизации.

Рассматривая модель с Collapsed Core дальше – ввиду того, что трафик в сети по большей части будет направлен только к остальным офисам организации или в Интернет, то решения с uplink-портами по 10 Гбит/сек и более (которые больше подходят к центрам обработки данных, например - MES5324) будут чересчур избыточными. Лучшим вариантом будет переносить не уровень агрегации в ядро, а ядро в агрегацию, что позволит сэкономить большое количество бюджета и избежать простоя мощностей оборудования. Поэтому в качестве устройств на Collapsed Core уровне будет использоваться виртуальный образ, похожий на MES2324F.

Говоря о безопасности трафика, нужно отметить, что в мире МСЭ появилось много различных решений. Основные, которые стоит внести в рассмотрение:

- Ideco NGFW;
- UserGate;
- Континент 4;
- xFirewall.

Из предложенного списка стоит сразу отменить Континент, так как действительно серьезных законодательных требований и требований к безопасности представлено не было, что и делает это решение сильно избыточным. Также это решение очень специфично в настройке, чем стоит заниматься квалифицированным специалистам, имеющим опыт в работе с таким оборудованием.

UserGate – самый многофункциональный NGFW из всех представленных. Данное решение закрывает практически все потребности, которые может закрывать МСЭ. Также существует реализация для ЦОД – UserGate DCFW. За огромный функционал придется платить стабильностью и очень квалифицированными специалистами, которые будут заниматься в организации только администрированием UserGate.

Ideco – МСЭ, особо не выделяющийся из множества остальных. Для организаций плюсом будет то, что продукт до невероятного прост в настройке, с этим справится абсолютно каждый человек с небольшой базой теории. Также отличается огромной стабильностью, но за эти плюсы придется пожертвовать функциональностью. Предоставляется возможность контроля

трафика на уровнях до L7, но не так гибко, как в остальных продуктах. Имеет только базовые возможности от маршрутизаторов – например, нет того же VRRP и его иных реализаций.

xFirewall является также очень специфичным продуктом, даже больше, чем Континент. Ключевым моментом является то, что продукт направлен в первую очередь на внедрение в сети с VipNet. Обуславливается это тем, что для администрирования xFirewall нужен Coordinator, который тоже сам по себе является МСЭ, но без IPS, но с функциями криптографии, которых лишен xFirewall.

Исходя из сравнения, выбор в данном проекте стоит отдать Idesco NGFW. Решение закрывает необходимый функционал по фильтрации и защите трафика, но отсутствие функциональности не влияет в данном проекте.

Подводя итог проектирования, необходимо подытожить планируемые технологии и протоколы, их местоположение в инфраструктуре.

На границе сети существует вариант сделать или один МСЭ, или кластер из двух МСЭ. Для экономии бюджета, с предположительными нагрузками сможет справиться и один NGFW. При росте нагрузки и трафика самым простым решением будет добавить еще один межсетевой экран и настроить балансировку. Решение достаточно простое и лаконичное, потому что никаких изменений в основу сети вносить не придется, не будет даже простоя.

Далее – сердце сети. Использоваться будет именно Collapsed Core, так что на уровне ядра изначально можно оставить два коммутатора. Так как сеть не высоконагруженная, а также не планируется рост числа пользователей, на двух коммутаторах останется запас и по ресурсам, и по общей пропускной способности. Масштабируется такое решение будет аналогично – необходимо добавить еще один коммутатор, добавить все интерфейсы в VRRP группу и настроить анонс маршрутов. С подключениями все немного сложнее – каждый коммутатор доступа должен иметь минимум два подключения к вышестоящим, но при добавлении третьего это решение не играет особой роли. В таком случае будет необходимо проводить подключения не ко всем коммутаторам доступа, а только к новым, которые добавляются при расширении количества пользователей.

На уровне доступа все немного проще. Так как не указано точное число пользователей, будет настроено лишь небольшое количество, для грубой демонстрации работоспособности решения.

Итого, при увеличении количества пользователей, лучшим вариантом будет возвращаться к полноценной трехуровневой модели или делать дополнительные каналы в агрегации и добавлять устройств на уровень доступа и ядра. При первом варианте огромной проблемой станет потребность в полной переработке сети, что приведет и к большим затратам, и

к долгому простоем сети. При втором варианте исчезает вероятность простоя сети, но возникают другие проблемы:

- капитальные затраты на новое оборудование;
- большое количество роста подключений.

На плакате 1 представлена итоговая сетевая архитектура. Все подключения к ядру дублированы, чтобы соблюсти особенности изначальной сетевой модели. Для максимальной экономии, чтобы избежать единой точки отказа был добавлен второй канал от провайдера, но без дублирования (как правило, очень дорогих) МСЭ, который заходит напрямую в ядро. Такое решение можно было бы назвать большим ударом по безопасности, но суть при таких размытых требованиях к безопасности все же остается в максимальной отказоустойчивости за наименьшие деньги.

Каналы от серверной части в макете планируются в виде LAG, но при использовании платформ с uplink-портами более чем в 1 Гбит/сек такое решение можно назвать избыточным. Планируется туннель до основного офиса, куда будут анонсироваться локальные сети отдела. Также динамическая маршрутизация не будет мешать между уровнем ядра и NGFW (в таком случае при масштабировании и ядра, и МСЭ на границе сети потребуется минимальное количество времени), даже при условии, что количество сетей слишком маленькое.

Вся локальная сеть сегментирована на:

- VLAN 201 – клиентский сегмент (если рассматривать его уже реализованным в физической инфраструктуре, то этот сегмент придется дробить для функционального разделения всех работников и их прав);
- VLAN 100 – сегмент администраторов (при рассмотрении макета в физическом виде, сюда будут относиться также устройства для СКУД и прочие сетевые решения, используемые для администрирования);
- VLAN 101 – общий серверный сегмент;
- VLAN 102 – сегмент для ООВ-интерфейсов, управления сетевым оборудованием, служебного трафика.

Количество серверов в макете можно назвать минимально необходимым, особенно для основных задач отдела. Решения ALD PRO вынесены на отдельные физические машины для обеспечения точной и полной работоспособности и изолированности от остальных сервисов организации. Для хранения используется две машины для обеспечения полной отказоустойчивости хранения данных.

Между машинами добавлено отдельное подключение, которое никак не функционирует и не задействуется в основной сети. Как правило, сервера, СХД поставляются с дополнительными

сетевыми картами (2.5 или 10 Гбит/сек). В случае без них – среднебюджетный вариант не будет стоить больших денег. Именно по этому подключению будет происходить синхронизация всех хранимых файлов, образов виртуальных машин между серверами хранения.

Планируется три сервера виртуализации, объединенных в кластер. Три – минимальное количество, при котором можно обеспечить абсолютную отказоустойчивость всех виртуальных машин, работающих на серверах. Подводя итог, можно сказать – при таких условиях будет обеспечена очень хорошая возможность масштабирования, что в условиях макета (постоянное изменение, добавление, тестирование новых решений в инфраструктуре) очень важно.

2.2 Планирование и подготовка к реализации будущей инфраструктуры

При создании макета с виртуальными машинами, сначала всегда требуется создать шаблоны VM, из которых все будет разворачиваться. Первым делом будет создан шаблон Astra Linux обеих версий (1.7 и 1.8). Конфигурации шаблонов показаны на рисунках 2.3–2.4, а сам процесс установки на рисунках 2.5–2.14. 4 ГБ и 4 vCPU были выбраны, так как данный объем считается избыточным для макета и тестирования инфраструктуры. Также играют роль минимальные требования и клиентских ПК, и серверов, ПО которых должно запустится и функционировать.

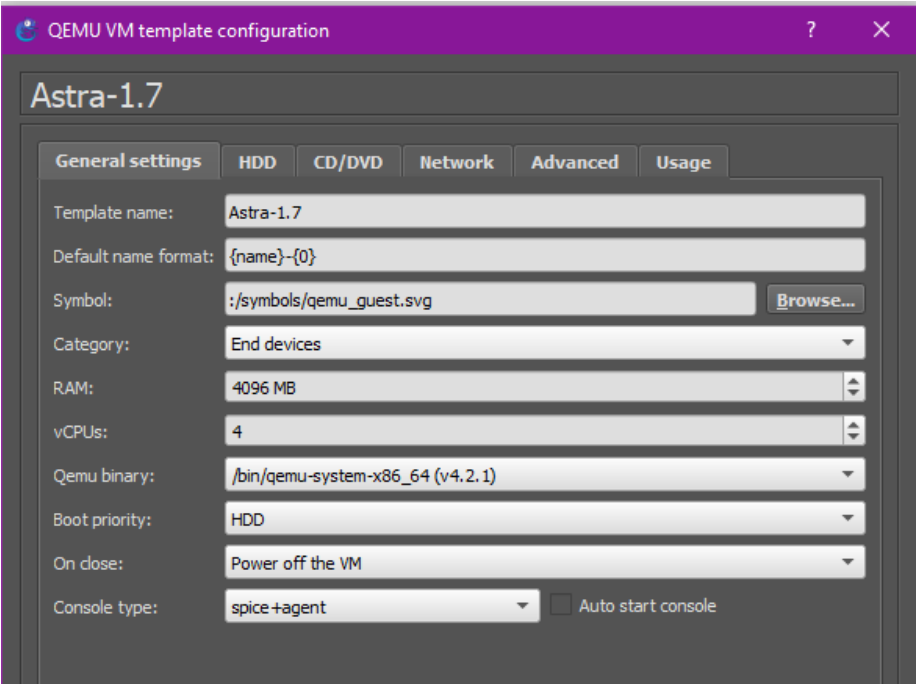


Рисунок 2.3 – Настройки шаблона Astra Linux 1.7

Настройки для версии 1.8 будут аналогичными.

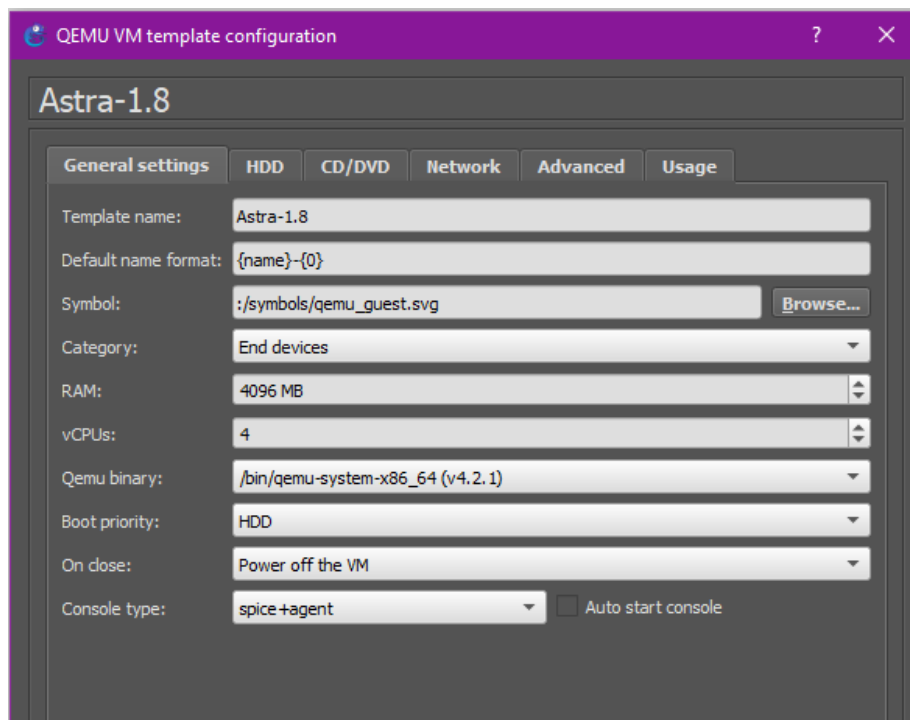


Рисунок 2.4 – Настройки шаблона Astra Linux 1.8

Далее для шаблонов необходимо создать диск и установить саму операционную систему. После успешной загрузки установочного образа необходимо придумать имя для учетной записи администратора.

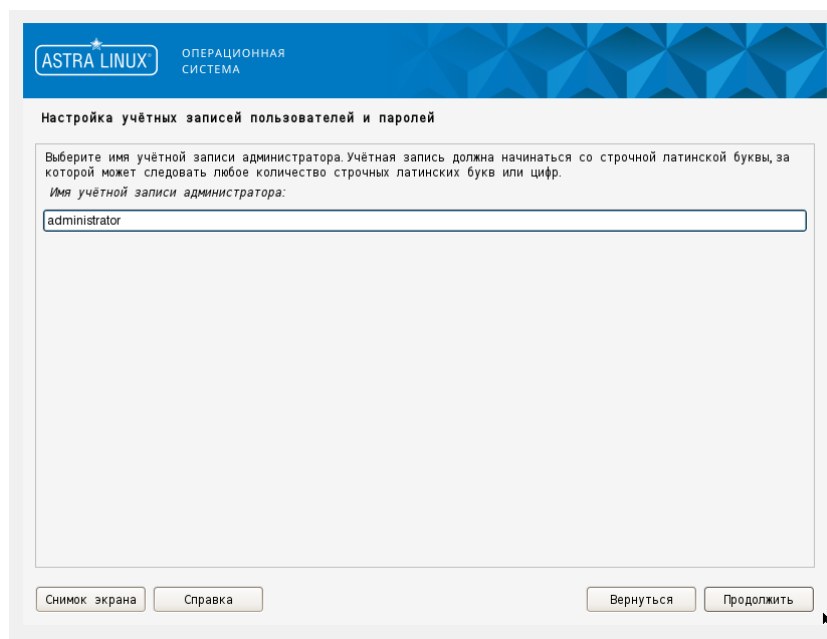


Рисунок 2.5 – Создание аккаунта администратора

Дальше необходимо два раза повторить пароль для нашего пользователя. Пароль в рамках макета выставляется одинаковым везде, ради ускорения развертывания. После ввода пароля и нескольких шагов по настройке системы появится этап разметки диска. Ввиду

отсутствия требований можно выбрать «Авто», что автоматически создаст загрузочный раздел и раздел подкачки.

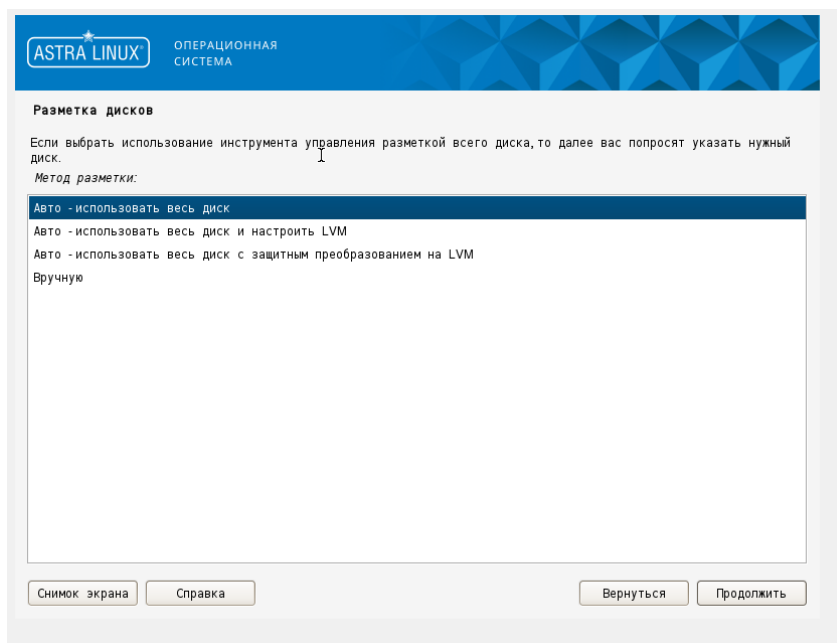


Рисунок 2.6 – Выбор метода разметки

Далее, если был выбран метод «Авто», появится окно выбора схемы. Необходимо просто выбрать «Все файлы в одном разделе», так как второй в данном случае никакой роли не играет.

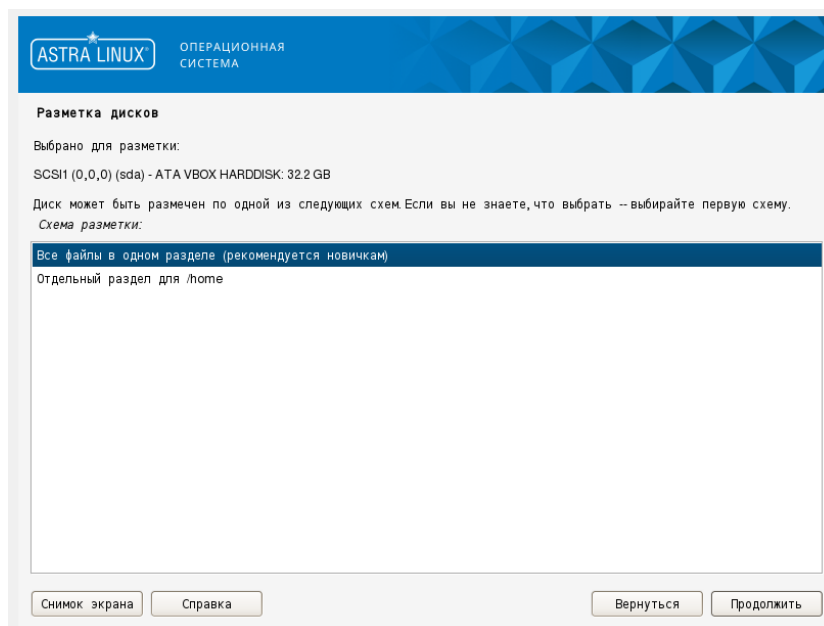


Рисунок 2.7 – Выбор метода разметки

После разметки идет следующий важный шаг в виде выбора ПО, которое будет установлено сразу же, без ручного вмешательства. Для полноты и упрощения последующей работы необходимо выбрать пункты:

- графический интерфейс;
- средства работы с Интернет;

- консольные утилиты;
- средства удаленного подключения SSH.

Пункт с ядром не выбирается ввиду отсутствия совместимости многого ПО с ядром «hardened».

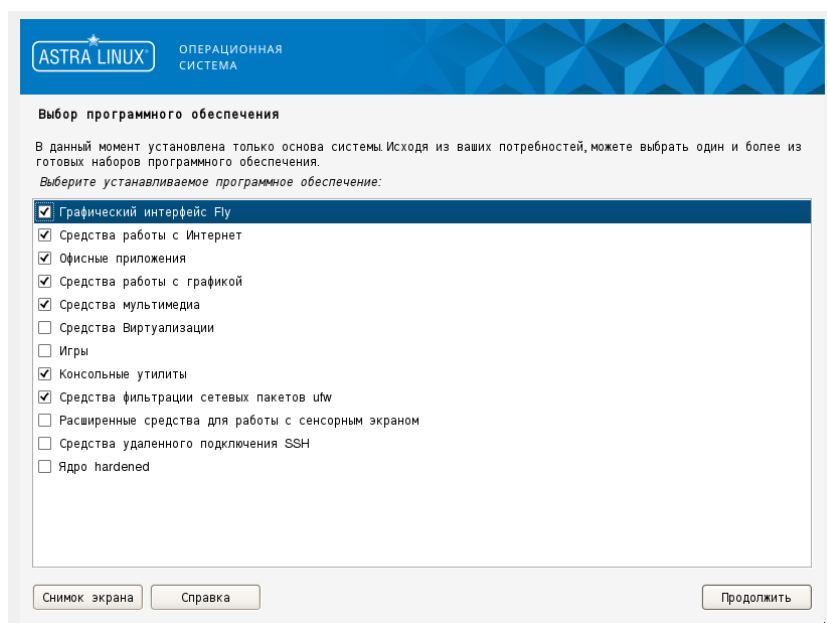


Рисунок 2.8 – Выбор программного обеспечения

После идет завершающий этап основных настроек – выбор режима работы ОС. Режимы отличаются функциональностью, которая отвечает за безопасность информации.

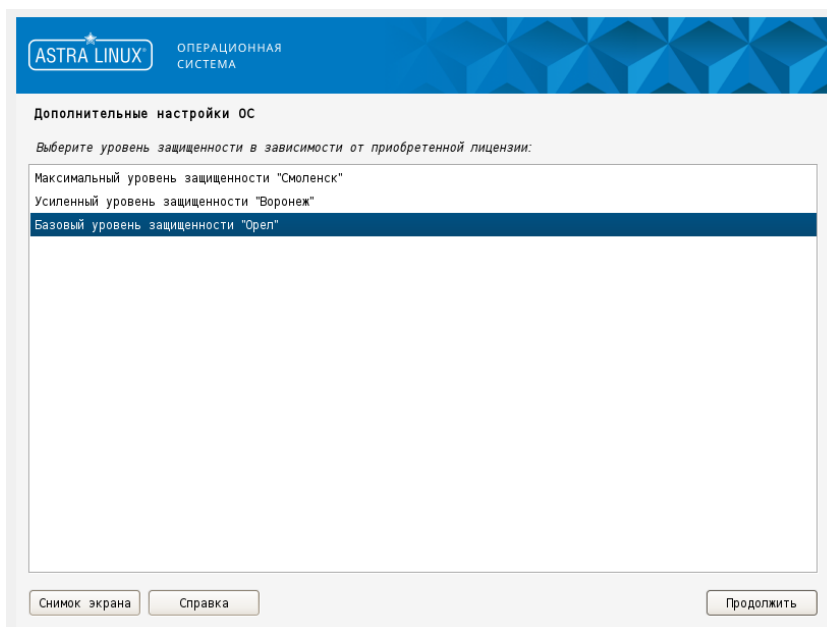


Рисунок 2.9 – Выбор режима работы ОС Astra Linux

Режим выбирается в соответствии с приобретенной лицензией, но в данном случае нужно выбрать базовый уровень защищенности, чтобы без особых проблем с функциями безопасность базово настроить ОС. После установки уровень защищенности также можно

сменить. Настройка Astra Linux SE 1.8 выглядит аналогично и требует абсолютно аналогичных настроек, поэтому освещение этого процесса не несет особо смысла.

После необходимо создать шаблон Ideco NGFW. Минимальные требования для установки – 150 ГБ места на накопителе и 16 ГБ оперативной памяти, после установки ресурсы будут снижены до 8ГБ ОЗУ для увеличения производительности самого сервера. Технические детали шаблона представлены на рисунке 2.10. Общий процесс установки, только для шагов играющих роль в работе в рамках макета, показан в рисунках 2.11–2.14.

The screenshot shows the 'Ideco' configuration window with the 'General settings' tab selected. The settings are as follows:

Setting	Value
Template name:	Ideco
Default name format:	{name}-{0}
Symbol:	./symbols/qemu_guest.svg
Category:	Routers
RAM:	8192 MB
vCPUs:	4
Qemu binary:	/bin/qemu-system-x86_64 (v4.2.1)
Boot priority:	HDD
On close:	Power off the VM

Рисунок 2.10 – Выбор режима работы ОС Astra Linux

После создания жесткого диска необходимо установить саму операционную систему. Необходимо загрузиться с указанного образа, в первом меню выбрать «Install Ideco NGFW»



Рисунок 2.11 – Меню загрузчика GRUB

```

Установка Ideco NGFW 19.0.496
-----
Для установки выбран диск '161 ГБ - QEMU HARDDISK (drive-scsi0)'.
ВНИМАНИЕ! Все данные на нём будут уничтожены!

Пожалуйста подтвердите ваш выбор.

Введите 'y' и нажмите Enter для подтверждения.
Введите 'c' и нажмите Enter для отмены.
# _
  
```

Рисунок 2.12 – Выбор накопителя для установки ОС

После выбора диска необходимо настроить временную зону, подтвердить настройки. После этого нужно загрузиться с уже установленной ОС и произвести начальную настройку.

Нужно отказаться от предложения сделать устройство второй нодой кластера и приступить к созданию аккаунта администратора.

```
Внимание! Аккаунт администратора отсутствует.  
Требуется предварительно его создать.  
  
Создание аккаунта администратора.  
  
Введите новый логин и нажмите Enter.  
# admin  
  
Введите новый пароль и нажмите Enter.  
  
Введите 'b' и нажмите Enter для возврата.  
#  
  
Повторите пароль и нажмите Enter.  
  
Введите 'b' и нажмите Enter для возврата.  
#  
Аккаунт администратора создан успешно.  
  
Нажмите любую клавишу для перехода к локальному меню.
```

Рисунок 2.13 – Создание аккаунта администратора

После этого нужно войти и приступить к настройке локального интерфейса, для доступа в веб-интерфейс. Idesco NGFW не поощряет настройку через командную строку и в случае конфигурирования именно таким образом – снимает с себя ответственность за работоспособность устройства. Важное замечание - при использовании сетевых карт одного производителя могут возникнуть трудности при идентификации сетевой карты для настройки сетевого интерфейса. Для корректной идентификации сетевой карты нужно использовать ее MAC-адрес.

```
Внимание! Не найдено ни одного настроенного локального  
сетевого интерфейса. Его необходимо настроить для доступа  
к web-интерфейсу управления сервером.  
  
Выберите сетевую карту.  
  
1. 00:15:5d:a9:ac:0f Microsoft Hyper-V Virtual Ethernet Adapter (Link N/A)  
2. 00:15:5d:a9:ac:10 Microsoft Hyper-V Virtual Ethernet Adapter (Link N/A)  
  
Введите номер пункта и нажмите Enter.  
Введите 'c' и нажмите Enter для отмены.  
#
```

Рисунок 2.14 – Настройки сетевых интерфейсов

После выбора интерфейса можно выбрать тип IP адреса – статический или динамический. В случае шаблона это не имеет значения, поэтому необходимо выбрать

динамический, чтобы при развертывании без особых проблем изменить на требуемый по топологии. На этом процесс установки Idec NGFW завершен.

Остальные ОС необходимо установить с заявленными минимальными требованиями, в процессе установки нужно выполнить аналогичные шаги с аналогичными значениями.

После установки ОС необходимо собрать топологию в проекте и подключить устройства в соответствии со схемой на плакате 1.

2.3 Настройка сетевого оборудования

После создания топологии в проекте стоит приступить к настройке сетевых устройств и адресации на серверах и клиентских ПК. Сначала нужно настроить межсетевой экран, только для обеспечения связности между устройствами – трансляция адресов, динамическая маршрутизация. Процесс настройки МСЭ представлен на рисунках 2.15–2.31. Первым делом надо перенастроить сетевые реквизиты на нужном интерфейсе.

```
Управление сервером
1. Консоль
2. Отключить все интерфейсы и настроить новый
3. Включить доступ к веб-интерфейсу из внешней сети
4. Включить доступ к серверу по SSH из Интернет
5. Включить доступ к серверу по SSH из локальных сетей
6. Включить режим 'Разрешить Интернет всем'
7. Сбросить блокировки по IP
8. Отключить пользовательский фаервол
9. Отключение VCE-интерфейсов
10. Создать новый бэкап
11. Восстановить из бэкапа
12. Мгновенно восстановить из бэкапа
13. Включить доступ Удаленного Помощника
14. Контакты технической поддержки
15. Управление кластером
16. Восстановиться на предыдущую версию
17. Перезагрузка сервера
18. Отключить сервер
19. Выход

Введите номер пункта и нажмите Enter.
# 2

Выберите сетевую карту.

1. 0c:83:ab:f3:00:00 Intel Corporation 82540EM Gigabit Ethernet Controller Link Up
2. 0c:83:ab:f3:00:01 Intel Corporation 82540EM Gigabit Ethernet Controller Link N/A
3. 0c:83:ab:f3:00:02 Intel Corporation 82540EM Gigabit Ethernet Controller Link N/A
4. 0c:83:ab:f3:00:03 Intel Corporation 82540EM Gigabit Ethernet Controller Link N/A
5. 0c:83:ab:f3:00:04 Intel Corporation 82540EM Gigabit Ethernet Controller Link N/A
6. 0c:83:ab:f3:00:05 Intel Corporation 82540EM Gigabit Ethernet Controller Link N/A

Введите номер пункта и нажмите Enter.
Введите 'с' и нажмите Enter для отмены.
#
```

Рисунок 2.15 – Новая настройка сетевых интерфейсов

Далее в интерактивном опросе, следует настроить автоматическое получение IP-адреса (или иначе, если того требует провайдер). VLAN-тег в данном случае – оставить пустым.

```

Введите номер пункта и нажмите Enter.
Введите 'с' и нажмите Enter для отмены.
# 1

Настроить локальную сеть автоматически через DHCP?

Введите 'у' и нажмите Enter для подтверждения.
Введите 'н' и нажмите Enter для отказа.
# у

Введите VLAN тэг (или оставьте пустым) и нажмите Enter.

Введите 'b' и нажмите Enter для возврата.
Введите 'с' и нажмите Enter для отмены.
#
Локальный интерфейс успешно настроен.

```

Рисунок 2.16 – Продолжение настройки сетевых интерфейсов

После следует перепроверить IP-адрес и найти его среди прочих системных интерфейсов.

```

[administrator@bez-nazvaniya-d76b6c6a-33d9-4c48-a41c-84d8db63b309 ~]# ip --br a
lo                UNKNOWN    127.0.0.1/8 169.254.254.254/32 fc00::1::1/128 ::1/128
lb_local_out@lb_local_in UP        169.254.1.6/29
lb_local_in@lb_local_out UP      169.254.1.1/29 169.254.1.2/29
FWtest           UNKNOWN
Leth3            UP        192.168.122.145/24
eth1             DOWN
eth2             DOWN
eth3             DOWN
eth4             DOWN
eth5             DOWN
CCwg_cl         UNKNOWN
Lvpn0@if12       UP        169.254.1.4/32
[administrator@bez-nazvaniya-d76b6c6a-33d9-4c48-a41c-84d8db63b309 ~]# _

```

Рисунок 2.17 – Проверка IP-адресов на интерфейсах

После надо зайти в веб-интерфейс по IP-адресу от провайдера, с портом 8443 и войти с учетными данными, созданными при установке ОС.

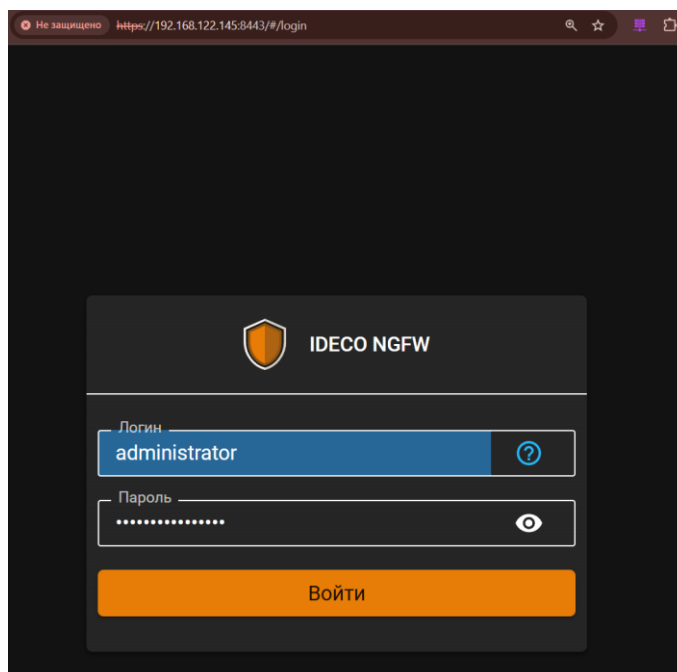


Рисунок 2.18 – Страница входа в интерфейс управления

На главной странице пользователь сможет увидеть управление модулями, графики о загрузке межсетевого экрана и статистику трафика от пользователей.

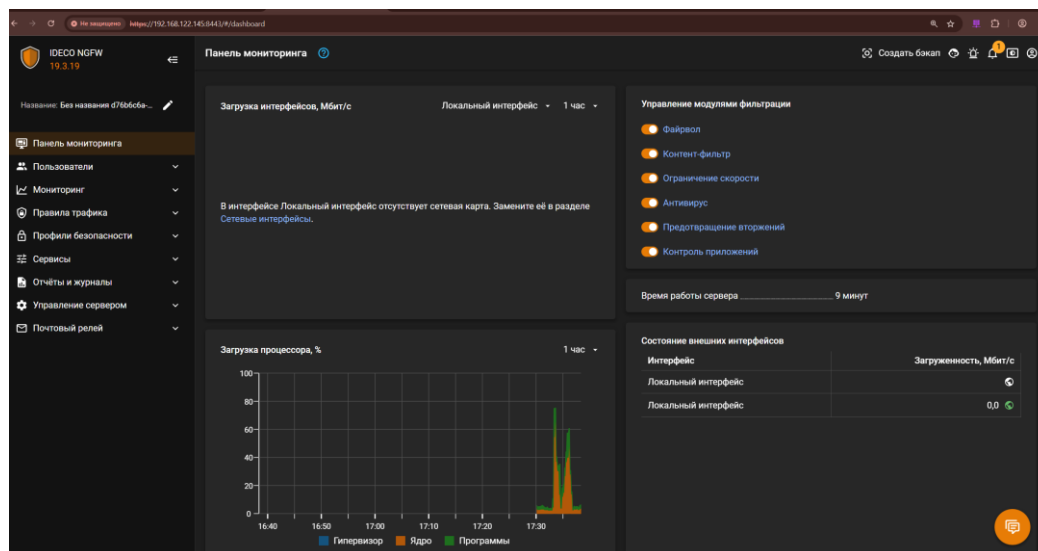


Рисунок 2.19 – Дашборд с модулями и графиками

После входа необходимо настроить адресацию на интерфейсах, создать пользователей и авторизацию по подсетям, прикрепив созданных пользователей к нужным подсетям. В итоге весь результат анализа трафика будет отсылаться на пользователей, которые были настроены до этого.

Сначала нужно зайти в раздел «Сетевые интерфейсы» и выбрать создание агрегированного канала. После дать название, выбрать интерфейсы, которые будут добавлены в LAG и сделать описание. Аналогично, в соответствии с созданной L1 схемой необходимо настроить второй LAG.

Создание агрегированного интерфейса (LAGP)

Название:

Выберите сетевые карты

- ☐ 0c:83:ab:f3:00:00 Intel Corporation 82540EM Gigabit Ethernet Controller
- ☒ 0c:83:ab:f3:00:01 Intel Corporation 82540EM Gigabit Ethernet Controller
- ☒ 0c:83:ab:f3:00:02 Intel Corporation 82540EM Gigabit Ethernet Controller
- ☐ 0c:83:ab:f3:00:03 Intel Corporation 82540EM Gigabit Ethernet Controller
- ☐ 0c:83:ab:f3:00:04 Intel Corporation 82540EM Gigabit Ethernet Controller
- ☐ 0c:83:ab:f3:00:05 Intel Corporation 82540EM Gigabit Ethernet Controller

Комментарий:

Рисунок 2.20 – Создание агрегированного канала

В результате, после создания в разделе агрегированных интерфейсов появятся новые. Здесь же доступно управление – включение, переназначение участников группы и так далее.

ВНЕШНИЕ И ЛОКАЛЬНЫЕ	АГРЕГИРОВАННЫЕ (LAG)	ТУННЕЛЬНЫЕ	VCE	SPAN
+ Добавить Отображение				
Название	Сетевая карта	Статусы соединения	Комментарий	Управление
BondToCore1	Выбраны 2 сетевые карты	—	Агрегированный кэ ↓	
BondToCore2	Выбраны 2 сетевые карты	—	Агрегированный кэ ↓	

Рисунок 2.21 – Результат создания двух LAG

Далее необходимо назначить IP-адреса Bond-интерфейсам. Для этого необходимо также в разделе «Сетевые интерфейсы» зайти во «Внешние и локальные» и начать создание. Это создаст некое «соответствие» физического и логического интерфейса. Необходим такой подход потому, что логических интерфейсов на физическом может гораздо больше одного (например, реализация Router-on-a-Stick). Необходимо задать название, физический интерфейс и IP-адрес, остальное оставить по умолчанию. Аналогично в соответствии с L3 схемой нужно задать остальные IP-адреса на МСЭ.

Создание локального Ethernet интерфейса

Название

ToCore1

Сетевая карта

BondToCore1

MAC-адрес

b2:2c:ed:26:03:89

Зона

ToCore

Поле необязательное

Тег VLAN

Число от 1 до 4094

☐ Автоматическая конфигурация через DHCP

IP-адрес/маска

172.16.0.1/30

+ Добавить IP-адрес с маской

Рисунок 2.22 – Назначение адресов агрегированным группам

По итогу настройки в разделе «Внешние и локальные» можно будет увидеть новые интерфейсы.

ВНЕШНИЕ И ЛОКАЛЬНЫЕ						
АГРЕГИРОВАННЫЕ (LACP) ТУННЕЛЬНЫЕ VCE SPAN						
+ Добавить Сетевые карты Отображение						
Тип	Название	Зона	IP-адрес/маска	Сетевая карта	Статусы соеди...	Управление
Локальная с...	Локальный ...	—	192.168.1	0с:83:a...	ETH	
Локальная с...	ToCore1	ToCore		BondToCore1	ETH	
Локальная с...	ToCore2	ToCore		BondToCore2	ETH	

Рисунок 2.23 – Все сетевые интерфейсы

После базовой адресации нужно позаботиться о том, чтобы трафик клиентов мог проходить через NGFW, по умолчанию весь трафик фильтруется и отбрасывается.

Добавить пользователя в группу «Все»

Основные настройки

Имя пользователя

ClientNetwork

Логин

client

Пароль

Qwerty123123123!

Повторите пароль

Qwerty123123123!

Скрыть пароль

Рекомендуется использовать комбинацию цифр и букв

Надежный

Телефон

Формат: знак «плюс» (+), код страны, код региона и номер телефона

Рисунок 2.24 – Создание пользователей

После создания необходимых пользователей должна получиться структура, как на [рисунке №.](#)

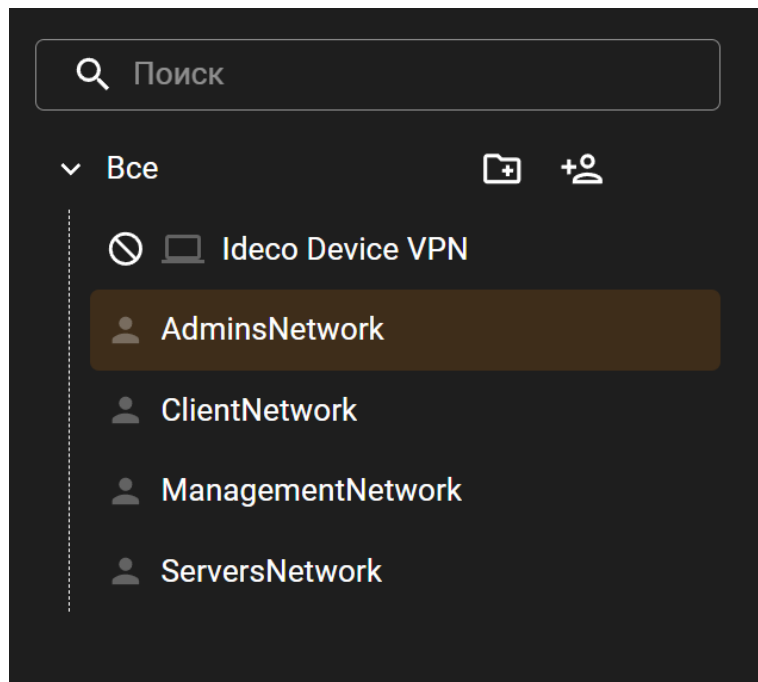


Рисунок 2.25 – Итоговое количество пользователей

Далее нужно зайти в раздел «Авторизация» в меню «Авторизация по подсетям» и присвоить каждому пользователю свою сеть.

Рисунок 2.26 – Назначение сетей пользователям для авторизации

В результате должна получиться структура, как на [рисунке 2.27](#).

ОСНОВНОЕ IP И MAC АВТОРИЗАЦИЯ АВТОРИЗАЦИЯ ПО ПОДСЕТЯМ ПОЛЬЗОВАТЕЛИ ТЕРМИНАЛЬНЫХ СЕРВЕРОВ AD			
+ Добавить		Отображение	Поиск
Подсеть	Пользователь	Комментарий	Управление
10.0.0.0/23	ClientNetwork	Авторизация для общего клиентского	🟢 ✎ 🗑
192.168.100.0/24	AdminsNetwork	Авторизация для сегмента администр	🟢 ✎ 🗑
10.0.2.0/24	ServersNetwork	Авторизация для общего серверного с	🟢 ✎ 🗑

Рисунок 2.27 – Результат настройки авторизации по подсетям

После необходимо настроить подключение к VPN организации и обеспечить динамическую маршрутизацию, чтобы обмениваться маршрутами с коммутаторами уровня ядра и остальными офисами организации.

Режим работы

Транспортный (GRE over IPsec)

Адрес удалённого устройства

10.255.255.1

Например, 198.168.32.10 или example.com

IP-адрес интерфейса туннеля

10.255.255.21/24

Пример: 10.100.0.1/16

Удалённый IP-адрес туннеля

10.255.255.1

Формат: 10.100.0.50

Заполните «IP-адрес интерфейса туннеля» и

Рисунок 2.28 – Создание IPsec туннеля до сети в главном офисе

После создания туннеля и указания необходимых настроек, согласованных с администратором второго конца туннеля, в разделе исходящих подключений можно будет увидеть новый, идущий к основному офису организации.

IPsec

125 %

Сбросить

ИСХОДЯЩИЕ ПОДКЛЮЧЕНИЯ

ВХОДЯЩИЕ ПОДКЛЮЧЕНИЯ

Конфигуратор подключения MikroTik

Конфигуратор подключения Cisco

+ Добавить

Отображение

Поиск

Название ...	Вх. скоро...	Исх. ...	Зона	Режим работы	IP-адрес интерфейс...	Интерфейс	Сре...	Пот...	Дж...	Адре...	Удалённый IP-адрес тунн...
ToOrg - 0	—	—	OfficeTun...	Транспортный ...	10.255.255.21/24	—	—	—	—	—	10.255.255.1

Рисунок 2.29 – Созданный IPsec-туннель

Чтобы основной офис и клиенты из нового отдела могли общаться между собой, необходимо настроить маршрутизацию. Офис небольшой, но предусматривая момент роста сети, необходимо настроить динамическую маршрутизацию. Для этого надо перейти в раздел «OSPF» настроить зону на интерфейсе туннеля, а после отдельно на двух локальных интерфейсах, чтобы получать основные локальные маршруты от ядра сети и передать их дальше.

Настройка OSPF на локальном интерфейсе

Интерфейс
ToOrg

Название зоны (в виде IP-адреса)
10.255.255.0 А/В

Целое число от 0 до 4 294 967 295

Вес (Cost)
30

Добавить

Отмена

Рисунок 2.30 – Настройка анонса локальных маршрутов в туннеле

После выполненных настроек, должны появиться все интерфейсы участвующие в маршрутизации и их зоны.

Локальные интерфейсы

+ Добавить Фильтры Отображение Поиск				
Интерфейс	Название зоны	Вес (Cost)	Управление	
ToOrg	10.255.255.0 (184549120)	30		
ToCore1	172.16.0.0 (2886729728)	10		
ToCore2	172.16.0.0 (2886729728)	10		

Рисунок 2.31 – Все интерфейсы, участвующие в маршрутизации

После настройки межсетевого экрана необходимо приступить к настройке коммутаторов уровня ядра. Нужно настроить IP-адреса, агрегацию каналов, динамическую маршрутизацию в сторону МСЭ и маршрутизацию между VLAN. Процесс выполнения описанных шагов показан на рисунках 2.32–2.44.

Настройка агрегации на коммутаторах Eltex очень схожа с Cisco. Нужно создать группу, добавить в нее необходимы интерфейсы. Так как Ideco NGFW поддерживает только LACP (active-backup), то стоит задать приоритет интерфейсов в LAG. Такие настройки надо выполнить до серверного коммутатора, на втором коммутаторе ядра – тоже аналогичные.

```

console#
console# configure
console(config)# interface port-channel 1
console(config-if)# exit
console(config)# interface gi1/0/1
console(config-if)# channel-group 1 mode auto
console(config-if)# lacp port-priority 100
console(config-if)# exit
console(config)# interface gi1/0/2
console(config-if)# channel-group 1 mode auto
console(config-if)# lacp port-priority 110
console(config-if)# exit
console(config)#
console(config)#

```

Рисунок 2.32 – LAG на CORE-1 к Ideco NGFW

После настройки агрегации нужно приступить к назначению IP-адресов на VLAN-интерфейсы на всех коммутаторах уровня ядра, чтобы маршрутизировать трафик между VLAN. Для этого надо просто создать VLAN, а после назначить им адреса. На первом коммутаторе все адреса будут иметь 253 адрес, на втором – 254. При масштабировании ядра адреса необходимо выставлять в убывающем порядке.

```

console#
console# configure
console(config)#
console(config)#
console(config)# interface vlan 100
console(config-if)# ip address 192.168.100.253/24
console(config-if)# exit
console(config)#
console(config)# interface vlan 201
console(config-if)# ip address 10.0.0.253/23
console(config-if)# exit
console(config)#
console(config)# interface vlan 101
console(config-if)# ip address 10.0.2.253/24
console(config-if)# exit
console(config)# interface vlan 102
console(config-if)# ip address 192.168.101.253/24
console(config-if)# exit
console(config)#

```

Рисунок 2.33 – Настройка VLAN-интерфейсов на CORE1

Дальше нужно назначить адреса на физические интерфейсы, идущие к МСЭ. По умолчанию включен Firewall и ICMP-запросы блокируются. Для этого везде надо их отключить.

```
console(config)# interface port-channel 1
console(config-if)# ip address 172.16.0.2/30
console(config-if)# ip firewall disable
console(config-if)# exit
console(config)# interface gi1/0/5
console(config-if)# ip address 172.16.2.1/30
console(config-if)# ip firewall disable
```

Рисунок 2.34 – Настройка IP-адресов на физических интерфейсах на CORE1

Аналогично, опираясь на составленную логическую схему, следует настроить второй коммутатор.

```
console(config)# interface port-channel 1
console(config-if)# ip address 172.16.0.6/30
console(config-if)# ip firewall disable
console(config-if)# exit
console(config)# interface gi1/0/9
console(config-if)# ip address 172.16.2.2/30
console(config-if)# ip firewall disable
console(config-if)# exit
console(config)# interface gi1/0/8
console(config-if)# ip address dhcp
console(config-if)# ip firewall disable
```

Рисунок 2.35 – Настройка IP-адресов на физических интерфейсах на CORE2

Чтобы ограничить возможности пользователей в сети на всех коммутаторах ядра надо прописать списки доступа, откуда и куда трафик не может ходить. Для того, чтобы запретить клиентский сети маршрутизироваться в сеть для управления и сеть администраторов (только в одну сторону), надо создать правило и применить его на необходимый VLAN-интерфейс.

```

console(config)# ip access-list extended ACL_V201
console(config-ip-al)# deny ip 10.0.0.0 0.0.1.255 192.168.100.0 0.0.0.255
console(config-ip-al)# deny ip 10.0.0.0 0.0.1.255 192.168.101.0 0.0.0.255
console(config-ip-al)# permit ip any any
console(config-ip-al)# exit
console(config)# ip access-list extended ACL_V102
console(config-ip-al)# permit ip 192.168.101.0 0.0.0.255 192.168.100.0 0.0.0.255
console(config-ip-al)# deny ip any any
console(config-ip-al)# exit
console(config)# interface vlan 201
console(config-if)# service-acl input ACL_V201
console(config-if)# exit
console(config)# interface vlan 102
console(config-if)# service-acl input ACL_V102
console(config-if)# exit

```

Рисунок 2.36 – Настройка ACL на коммутаторах ядра

Так как в сети существует второй провайдер, то надо реализовать и необходимые правила, чтобы использовать оба шлюза. Так как второй провайдер заходит сразу в ядро, то использоваться данный шлюз будет только в очень крайнем – случае, а именно в случае отказа межсетевого экрана. Для этого на всех коммутаторах ядра (если их больше, чем 2), не имеющих подключения к провайдеру надо указать плавающий маршрут – если отказывает NGFW, весь трафик перенаправлять на CORE2. На самом CORE2 аналогичный маршрут, но только на интерфейс, идущий к провайдеру.

```

console(config)# ip route 0.0.0.0 0.0.0.0 172.16.0.1
console(config)# ip route 0.0.0.0 0.0.0.0 172.16.2.2 10

```

Рисунок 2.37 – Настройка основного и резервного маршрута на CORE1

```

console(config)# ip route 0.0.0.0 0.0.0.0 172.16.0.5
console(config)# ip route 0.0.0.0 0.0.0.0 gigabitethernet 1/0/8 10

```

Рисунок 2.38 – Настройка основного и резервного маршрута на CORE2

Так как трафик в случае отказа будет маршрутизироваться через CORE2, на нем следует настроить сетевую трансляцию адресов. Для этого нужно создать зону безопасности и объекты, в которых указать адреса всех локальных сетей.


```

console(config)# security zone Internet
console(config-zone)# exit
console(config)# interface gi1/0/8
console(config-if)# security-zone Internet
console(config-if)# exit
console(config)# object-group network VLAN100
console(config-object-group-network)# ip address-range 192.168.100.1-192.168.100.254
console(config-object-group-network)# exit
console(config)# object-group network VLAN102
console(config-object-group-network)# ip address-range 192.168.101.1-192.168.101.254
console(config-object-group-network)# exit
console(config)# object-group network VLAN101
console(config-object-group-network)# ip address-range 10.0.2.1-10.0.2.254
console(config-object-group-network)# exit
console(config)# object-group network VLAN201
console(config-object-group-network)# ip address-range 10.0.0.1-10.0.1.254
console(config-object-group-network)# exit

```

Рисунок 2.39 – Создание зоны безопасности и групп объектов

После следует создать набор правил с любым названием и в нем определить правила, которые будут подменять IP-адрес ранее созданных локальных сетей на внешний.

```

console(config)# nat source
console(config-snat)# ruleset MASQUERADE
console(config-snat-ruleset)# to zone Internet
console(config-snat-ruleset)# rule 1
console(config-snat-rule)# description "masqueade"
console(config-snat-rule)# match source-address VLAN100
console(config-snat-rule)# action source-nat interface
console(config-snat-rule)# enable
console(config-snat-rule)# exit
console(config-snat-ruleset)# rule 2
console(config-snat-rule)# description "masqueade"
console(config-snat-rule)# match source-address VLAN101
console(config-snat-rule)# action source-nat interface
console(config-snat-rule)# enable
console(config-snat-rule)# exit
console(config-snat-ruleset)# rule 3
console(config-snat-rule)# description "masquerade"
console(config-snat-rule)# match source-address VLAN102
console(config-snat-rule)# action source-nat interface
console(config-snat-rule)# enable
console(config-snat-rule)# exit
console(config-snat-ruleset)# rule 4
console(config-snat-rule)# description "masquerade"
console(config-snat-rule)# match source-address VLAN201
console(config-snat-rule)# action source-nat interface
console(config-snat-rule)# enable
console(config-snat-rule)# exit

```

Рисунок 2.40 – Создание правил SNAT для каждой сети.

Далее на уровне ядра надо определить магистральные порты и разрешить VLAN, которые могут по ним проходить. Аналогичные настройки произвести на всём уровне ядра, в соответствии с L1 схемой.

```

console(config)# interface gi1/0/6-8
console(config-if)# switchport mode trunk
console(config-if)# switchport trunk allowed vlan add 201,101,100,102
console(config-if)# exit
console(config)# interface port-channel 2
console(config-if)# switchport mode trunk
console(config-if)# switchport trunk allowed vlan add 201,101,100,102
console(config-if)# exit

```

Рисунок 2.41 – Настройка магистральных портов на CORE1

Так как МСЭ еще не знает о локальных сетях, надо анонсировать ему маршруты. Аналогично требуется сделать на всех устройствах уровня ядра.

```

console#
console# configure
console(config)# router ospf 1
console(config-ospf)# network 192.168.100.0 0.0.0.255 area 172.16.0.0
console(config-ospf)# network 10.0.0.0 0.0.1.255 area 172.16.0.0
console(config-ospf)# network 10.0.2.0 0.0.0.255 area 172.16.0.0
console(config-ospf)# network 192.168.101.0 0.0.0.255 area 172.16.0.0
console(config-ospf)# network 172.16.0.0 0.0.0.3 area 172.16.0.0
console(config-ospf)# exit
console(config)#
console(config)#

```

Рисунок 2.42 – Настройка OSPF на CORE1

Последним шагом на коммутаторах ядра нужно настроить VRRP, чтобы избежать дополнительной точки отказа на NGFW. Ранее именно для этого указывались последние адреса из диапазонов, чтобы виртуальный адрес имел привычный формат.

```

console# configure
console(config)# interface vlan 100
console(config-if)# vrrp 1 ip 192.168.100.1
console(config-if)# vrrp 1 priority 150
console(config-if)# no vrrp 1 shutdown
console(config-if)#
console(config-if)# exit
console(config)#
console(config)#
console(config)# interface vlan 201
console(config-if)# vrrp 2 ip 10.0.0.1
console(config-if)# vrrp 2 priority 100
console(config-if)#
console(config-if)# no vrrp 2 shutdown
console(config-if)# exit
console(config)#
console(config)# interface vlan 101
console(config-if)# vrrp 3 ip 10.0.2.1
console(config-if)# vrrp 3 priority 150
console(config-if)#
console(config-if)# no vrrp 3 shutdown
console(config-if)# exit
console(config)#
console(config)# interface vlan 102
console(config-if)#
console(config-if)# vrrp 4 ip 192.168.101.1
console(config-if)# vrrp 4 priority 100
console(config-if)# no vrrp 4 shutdown
console(config-if)#

```

Рисунок 2.43 – Настройка VRRP на CORE1

Также можно заметить, что на разных устройствах указывается разный приоритет. Делается это для того, чтобы разбалансировать нагрузку в нормальном состоянии устройств.

Если из строя выйдет один – второй будет обрабатывать свои два VLAN, а после возьмет еще два от другого коммутатора.

```
console(config)# interface vlan 100
console(config-if)# vrrp 1 ip 192.168.100.1
console(config-if)# vrrp 1 priority 100
console(config-if)# no vrrp 1 shutdown
console(config-if)#
console(config-if)#
console(config-if)# exit
console(config)#
console(config)#
console(config)# interface vlan 201
console(config-if)# vrrp 2 ip 10.0.0.1
console(config-if)# vrrp 2 priority 150
console(config-if)#
console(config-if)# no vrrp 2 shutdown
console(config-if)#
console(config-if)# exit
console(config)#
console(config)# interface vlan 101
console(config-if)# vrrp 3 ip 10.0.2.1
console(config-if)# vrrp 3 priority 100
console(config-if)#
console(config-if)# no vrrp 3 shutdown
console(config-if)# exit
console(config)#
console(config)# interface vlan 102
console(config-if)#
console(config-if)# vrrp 4 ip 192.168.101.1
console(config-if)# vrrp 4 priority 150
console(config-if)# no vrrp 4 shutdown
console(config-if)#
```

Рисунок 2.44 – Настройка VRRP на CORE1

В последнюю очередь необходимо настроить коммутаторы доступа, которые первые обрабатывают трафик от пользователей и серверов. Процесс настройки коммутаторов доступа показан на рисунках 2.45–2.46. На каждом коммутаторе необходимо настроить uplink-порты так, чтобы они могли пропускать несколько VLAN. Остальные порты сделать портами доступа и распределить в соответствии с L3 схемой. Агрегацию на серверном коммутаторе необходимо настроить также, как на коммутаторах ядра.

```
console(config)# interface gi1/0/1-2
console(config-if)# switchport mode trunk
console(config-if)# switchport trunk allowed vlan add 201,101,100,102
console(config-if)# exit
console(config)# interface gi1/0/3-5
console(config-if)# switchport mode access
console(config-if)# switchport access vlan 201
console(config-if)# exit
```

Рисунок 2.45 – Настройка магистральных и портов доступа на access-sw-2

Заканчивается настройка сети назначением адресов для удаленного доступа к коммутаторам, в VLAN 102, задача которого – обрабатывать трафик, связанный с управлением

сетевого оборудования, серверов и если рассматривать физическую инфраструктуру, то и Out-Of-Band портов. Аналогично первому следует настроить остальной уровень доступа, включая серверный коммутатор.

```
console(config)# vlan 100,101,102,201
console(config-vlan)# exit
console(config)# interface vlan 102
console(config-if)# ip address 192.168.101.30/24
console(config-if)# ip firewall disable
console(config-if)# exit
```

Рисунок 2.46 – Настройка IP-адреса управления на access-sw-0

После выполнения всех вышеописанных шагов процесс настройки сетевых устройств закончен.

2.3 Настройка серверных ОС

На рисунках 2.47–2.60 показан процесс стандартной базовой настройки. Сначала следует полностью зайти в root-пользователя, так как в процессе настройки отключится sudo, из-за манипуляций с именем устройства. Нужно установить требуемое имя устройства, обновить текущую оболочку. Проверить настройки можно командой hostnamectl. Далее стоит отключить сервис управления сетевыми интерфейсами – NetworkManager, так как он конфликтует с основным во многих дистрибутивах, а именно ifupdown2 [8].

```
administrator@astra:~$ sudo -i
root@astra:~# hostnamectl set-hostname dc-1.cloud.consyst.local
root@astra:~# exec bash
root@dc-1:~# hostname
dc-1.cloud.consyst.local
root@dc-1:~# nano /etc/network/interfaces
root@dc-1:~# systemctl --now disable NetworkManager
Removed /etc/systemd/system/dbus-org.freedesktop.nm-dispatcher.service.
Removed /etc/systemd/system/multi-user.target.wants/NetworkManager.service.
Removed /etc/systemd/system/network-online.target.wants/NetworkManager-wait-online.service.
root@dc-1:~# systemctl restart networking
root@dc-1:~# ip --br a
lo                UNKNOWN        127.0.0.1/8 ::1/128
eth0              UP             10.0.2.10/24
eth1              UP             192.168.101.10/24
root@dc-1:~# █
```

Рисунок 2.47 – Команды настройки и проверки изменений

В командах выше редактируется файл /etc/network/interfaces, в котором стоит описать сетевые реквизиты и прочие настройки для сетевых интерфейсов.

```

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
address 10.0.2.10/24
gateway 10.0.2.1

auto eth1
iface eth1 inet static
address 192.168.101.10/24

```

Рисунок 2.48 – Файл сетевых настроек устройства

Также в процессе был отредактирован файл /etc/hosts, который как раз таки и нарушал работу sudo. Чтобы все привести к рабочему виду, нужно указать соответствие нового полного доменного и короткого имени устройства на внешний адрес.

```

GNU nano 3.2 /etc/hosts

127.0.0.1    localhost
#127.0.1.1  astra
10.0.2.10   dc-1.cloud.consynt.local dc-1

# The following lines are desirable for IPv6 capable hosts
::1        localhost ip6-localhost ip6-loopback
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters

```

Рисунок 2.49 – Исправленный файл /etc/hosts

Сетевые настройки на машинах, ответственных за хранение данных будут немного отличаться. Так как между ними добавлено подключение, для него тоже следует указать свои реквизиты [10].

```

iface eth0 inet static
address 10.0.2.15/24
gateway 10.0.2.1

auto eth1
iface eth2 inet static
address 192.158.101.15/24

auto eth2
iface eth2 inet static
address 172.16.1.2/30

```

Рисунок 2.50 – Сетевые реквизиты на машинах хранения данных

Все остальные устройства, работающие под управлением ОС Astra Linux 1.7 должны быть настроены аналогичным образом, но с учетом составленных физической и логической схем.

Следующим шагом стоит настроить сервера виртуализации. Для этого нужно просто отредактировать файл `/etc/network/interfaces`, в котором указать автозапуск всех физических интерфейсов, а также исправить адрес главного моста на адрес из VLAN 101 в соответствии со схемой.

```
GNU nano 7.2 /etc/network/interfaces
auto lo
iface lo inet loopback

auto ens4
iface ens4 inet manual
auto ens5
iface ens5 inet manual
auto ens6
iface ens6 inet manual

auto srvbr0
iface srvbr0 inet static
    address 10.0.2.12/24
    gateway 10.0.2.1
    bridge-ports ens4
    bridge-stp off
    bridge-fd 0

source /etc/network/interfaces.d/*
```

Рисунок 2.51 – Сетевые реквизиты на серверах виртуализации

После этого надо перезагрузить сервис управления сетью и проверить настройки командой `ip a`, но с флагом `--br` для сокращения вывода команды [7].

```
root@pve-office-node:~# systemctl restart networking.service
root@pve-office-node:~# ip --br a
lo                UNKNOWN          127.0.0.1/8 ::1/128
ens4              UP                fe80::e0c:e6ff:fec7:1/64
ens5              UP                fe80::e0c:e6ff:fec7:1/64
ens6              DOWN
srvbr0            UP                10.0.2.12/24 fe80::e0c:e6ff:fec7:0/64
root@pve-office-node:~# _
```

Рисунок 2.52 – Команды перезагрузки и диагностики сети

Теперь можно зайти в веб-интерфейс по настроенному адресу и указать старые учетные данные.

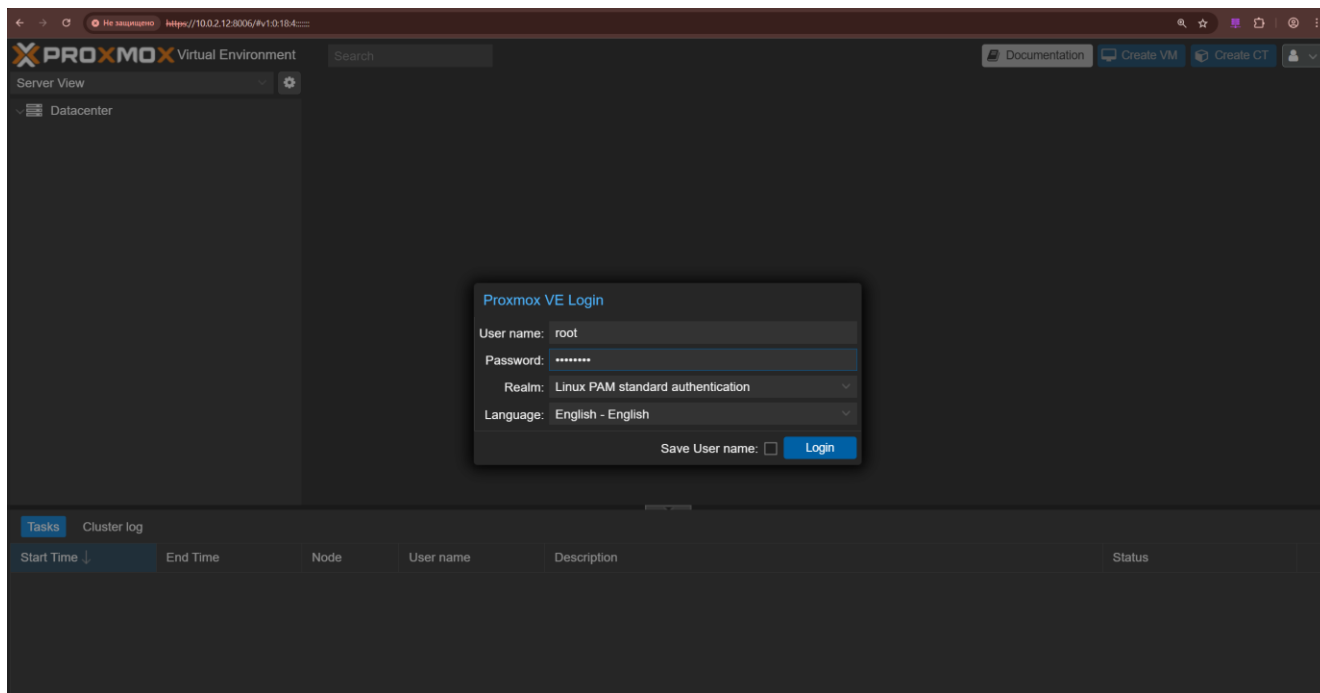


Рисунок 2.53 – Веб-интерфейс Proxmox

Продолжить настройки сети стоит в веб-интерфейсе, чтобы гарантировать отсутствие опечаток и ошибок в конфигурации. Нужно создать дополнительный мост, в котором будет интерфейс ens5, который находится в VLAN 102.

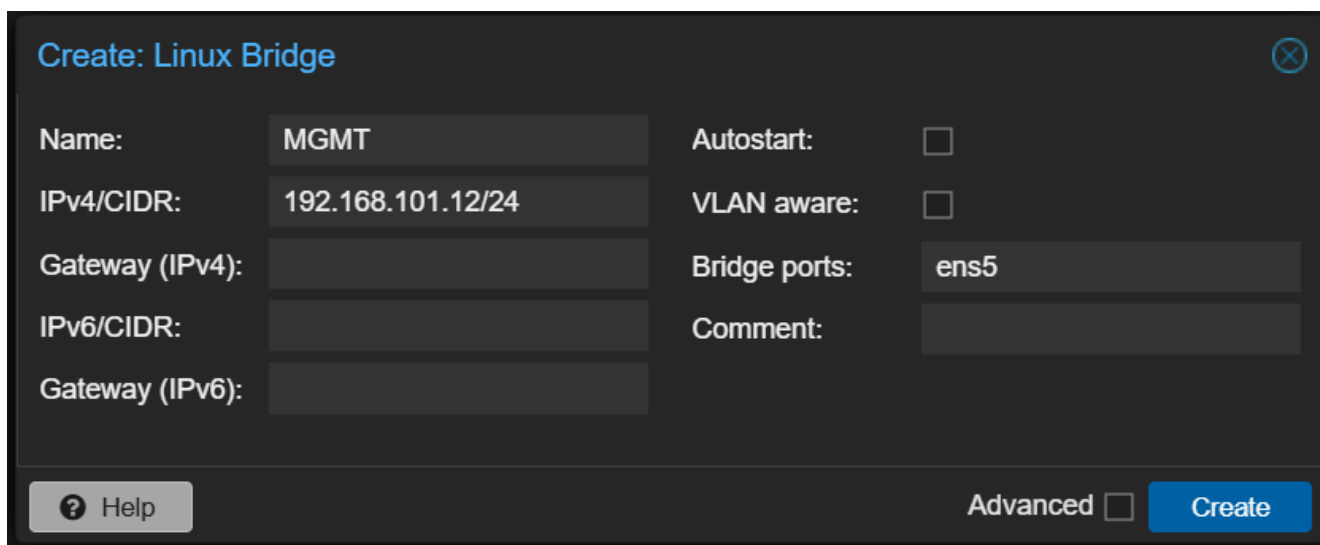


Рисунок 2.54 – Создание моста для MGMT управления

Proxmox после установки сам делает предварительную настройку системы, чтобы пользователь мог зайти и сразу начать работу. Хорошей практикой является удаление некоторых настроек, которые не несут особой пользы. Для начала стоит выключить том для хранения дисков виртуальных машин, чтобы избежать случайного переполнения системного диска.

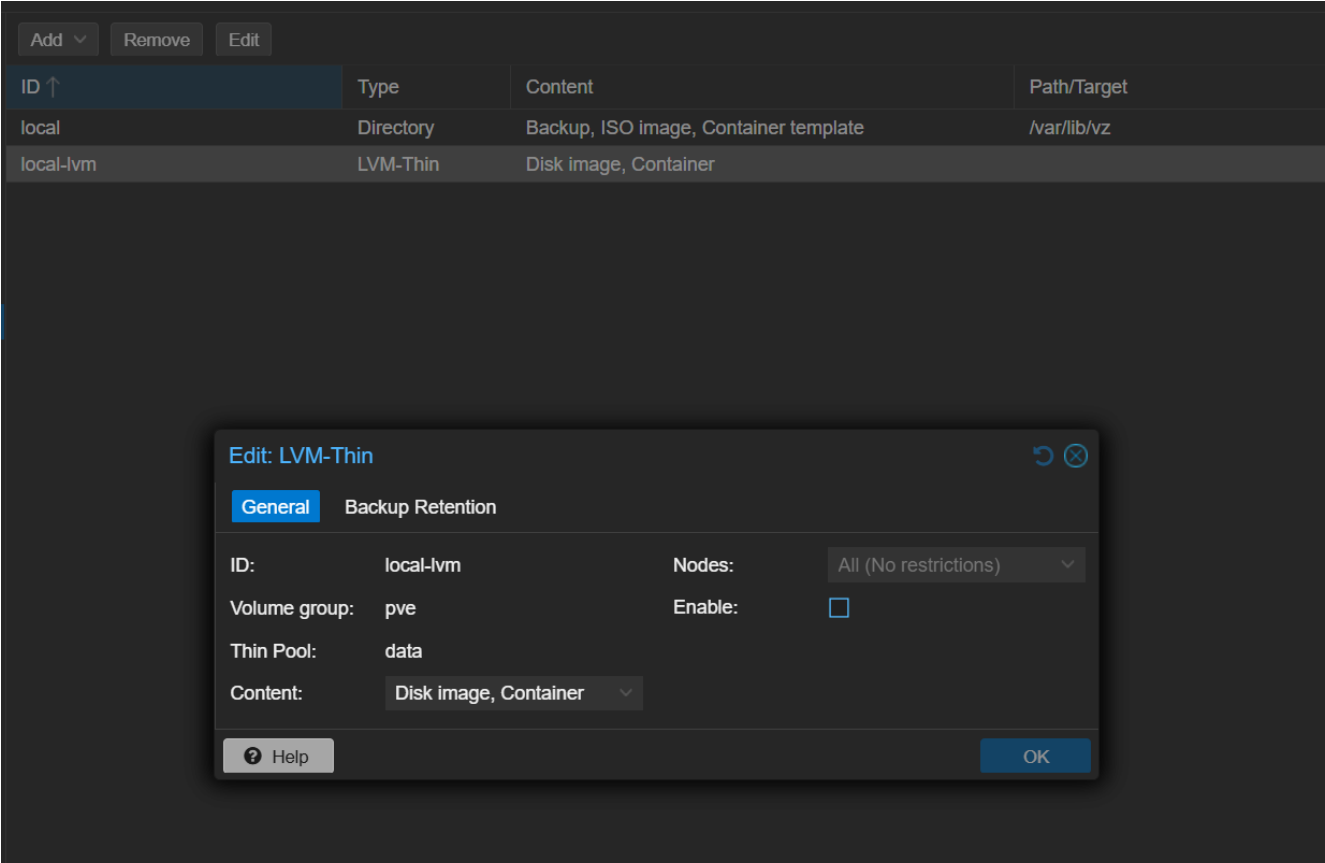


Рисунок 2.55 – Отключение дополнительного тома хранения

После выключить платные репозитории и добавить общедоступный, чтобы обеспечить своевременное обновление системы.

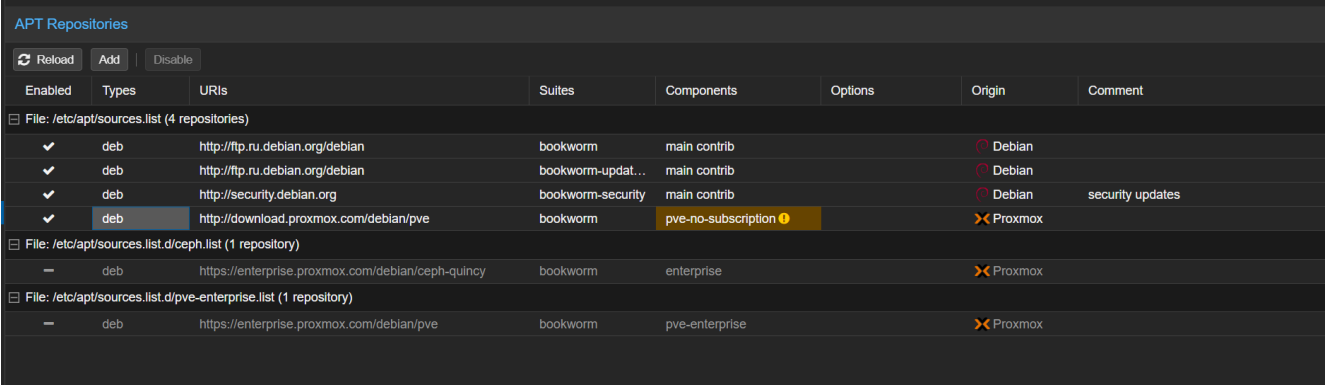


Рисунок 2.56 – Репозитории сервера

Теперь стоит создать кластер, чтобы в будущем обеспечить полную отказоустойчивость виртуальных машин на этих серверах. Для этого надо перейти в раздел «Cluster», и выбрать создание. Указать имя и подключения, которые будут использоваться для связи между серверами. По умолчанию будет использоваться VLAN 102, но в случае отказа взаимодействие между серверами будет происходить в 101 VLAN.

Рисунок 2.57 – Создание кластера

После нужно скопировать информацию для подключения к кластеру и вставить ее на остальных серверах.

Рисунок 2.58 – Информация для подключения к кластеру

В результате в веб-интерфейсе каждого должно появиться три машины.

Cluster Information

Create Cluster

Join Information

Join Cluster

Cluster Name:cluster-pves

Config Version:3

Number of Nodes:3

Cluster Nodes

Nodename	ID ↑	Votes	Link 0	Link 1
pve-1	1	1	192.168.101.12	10.0.2.12
pve-2	2	1	192.168.101.13	10.0.2.13
pve-3	3	1	192.168.101.14	10.0.2.14

Рисунок 2.59 – Созданный кластер

Дальнейшую настройку кластера необходимо продолжить позже, после создания NFS ресурсов и настройки файловых серверов.

В итоге останется только настроить удаленные подключения по нужному адресу. Для этого следует в файле `/etc/ssh/sshd_config` указать порт и адрес, на котором будет работать демон SSH. Необходимо указывать адрес интерфейса, который находится в VLAN 102. После

выполнения этого процесса на всех серверах, SSH будет доступен только в VLAN 102, который как раз таки и нужен для управления всеми устройствами. На серверах виртуализации этой настройки необходимо избежать из-за настройки кластера, которому необходим 22 порт и оба адреса.

```
Include /etc/ssh/sshd_config.d/*.conf

Port 3121
#AddressFamily any
ListenAddress 192.168.101.12
#ListenAddress ::
```

Рисунок 2.60 – Конфигурация SSH

На этом процесс предварительной настройки всех серверных ОС закончен и можно приступить к непосредственному развертыванию инфраструктурных сервисов на этих серверах.

2.4 Развертывание сервисов

Для функционирования серверов виртуализации и кластера следует развернуть файловые сервера, на которых будут храниться все образы виртуальных машин. Первым делом нужно установить пакет с NFS сервером. Процесс настройки NFS и резервного копирования показан на рисунках 2.61-2.74.

```
root@storage-2:~# apt install nfs-kernel-server
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Будут установлены следующие дополнительные пакеты:
  keyutils libevent-core-2.1-6 libnfsidmap1 libyaml-0-2 nfs-common python3-yaml rpcbind
Предлагаемые пакеты:
  open-iscsi watchdog
Следующие NOBBIE пакеты будут установлены:
  keyutils libevent-core-2.1-6 libnfsidmap1 libyaml-0-2 nfs-common nfs-kernel-server python3-yaml rpcbind
Обновлено 0 пакетов, установлено 8 новых пакетов, для удаления отмечено 0 пакетов, и 1412 пакетов не обновлено.
Необходимо скачать 853 kB архивов.
После данной операции объём занятого дискового пространства возрастёт на 3 291 kB.
Хотите продолжить? [Д/н]
```

Рисунок 2.61 – Установка NFS-сервера

После в конфигурации указать директорию и хостов, которым разрешено монтирование этого ресурса.

```
GNU nano 3.2 /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/vms 192.168.101.12(rw,sync,no_subtree_check) 192.168.101.13(rw,sync,no_subtree_check) 192.168.101.14(rw,sync,no_subtree_check)
```

Рисунок 2.62 – Настройки сетевого ресурса

Все ресурсы необходимо экспортировать, чтобы они стали доступны.

```
root@storage-1:~# exportfs -rav
exporting 192.168.101.12:/vms
exporting 192.168.101.13:/vms
exporting 192.168.101.14:/vms
root@storage-1:~#
```

Рисунок 2.63 – Подтверждение изменений

Без дополнительных инструментов одновременная работа двух серверов невозможна, поэтому второй будет использоваться как хранилище резервных копий. Для этого нужно сгенерировать ключи на первом сервере.

```
root@storage-1:~# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:VjNVnQ90TUnt5xryXEj3Y+KzkyeodShoxhfDxmEqEw4 root@storage-1.cloud.consyst.local
The key's randomart image is:
+---[RSA 3072]-----+
|          .o++B|
|         .++|
|        +.o.|
|       .o.=|
|      S.o.o+|
|     E o .*.+o+|
|    o...B.*++|
|   o=+.*=+|
|  o.o.o.o.=+|
+---[SHA256]-----+
root@storage-1:~#
```

Рисунок 2.64 – Генерация ключей для rsync

После генерации нужно добавить ключ на второй сервер, чтобы подключение по SSH происходило по паролю, того требует rsync.

```
root@storage-1:~# ssh-copy-id -i .ssh/id_rsa.pub administrator@172.16.1.1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ".ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are a
lready installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to in
stall the new keys
administrator@172.16.1.1's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'administrator@172.16.1.1'"
and check to make sure that only the key(s) you wanted were added.

root@storage-1:~# ssh-copy-id -i .ssh/id_rsa.pub root@172.16.1.1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ".ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are a
lready installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to in
stall the new keys
root@172.16.1.1's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'root@172.16.1.1'"
and check to make sure that only the key(s) you wanted were added.

root@storage-1:~#
```

Рисунок 2.65 – Копирование ключей на второй сервер

Настройка автоматических бэкапов будет произведена через скрипт, который сам добавляет необходимые задания в «cron». В скрипте необходимо только лишь указать сервер, имя хоста и директории, которые необходимо отдавать на второй сервер.

```
GNU nano 3.2                                rsync.sh

#!/bin/bash

# =====

# Autogenerate crontab tasks and ssh connect for rsync backup

# Rsync server
RSYNC_SERVER="root@172.16.1.2"
# For dir on server (/backup/srv-2)
CLIENT_NAME="storage-1"
# Dirs to backup
SRC_DIRS=("/etc" "/disk")
# Every 5m, for test
CRON_TIME="* */1 * * *"
# Place of public key
SSH_KEY="$HOME/.ssh/id_rsa"

# =====

if [[ ! -f "$SSH_KEY" || ! -f "$SSH_KEY.pub" ]]; then
    echo "Cant find keys, use: ssh-keygen"
    exit 1
fi
```

Рисунок 2.66 – Настройка данных скрипта для бэкапов на второй сервер

После можно запустить скрипт. Скрипт при пропуске шага с ключами сам попытается их добавить. После проверки подключения к серверу и возможности резервного копирования будут созданы необходимые задания.

```
root@storage-2:~# ./rsync.sh
Add ssh-key to root@172.16.1.2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
root@172.16.1.2's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'root@172.16.1.2'"
and check to make sure that only the key(s) you wanted were added.

Backup Completed: rsync -az --delete /etc /disk root@172.16.1.2:/disk/backup/storage-1/
root@storage-2:~#
```

Рисунок 2.67 – Успешная отработка скрипта

Задания отработают сразу после запуска скрипта и после каждый час все изменения будут синхронизироваться. Указанные директории должны появиться на втором сервере, с абсолютно неизменными данными и правами.

```

root@storage-1:~# ls -la /disk/backup/
итого 12
drwxr-xr-x 3 root root 4096 июн  6 23:29 .
drwxr-xr-x 4 root root 4096 июн  6 23:29 ..
drwxr-xr-x 4 root root 4096 июн  6 23:29 storage-1
root@storage-1:~# ls -la /disk/backup/storage-1/disk/
итого 16
drwxr-xr-x 4 root root 4096 июн  6 22:25 .
drwxr-xr-x 4 root root 4096 июн  6 23:29 ..
drwx----- 2 root root 4096 июн  6 22:24 lost+found
drwxrwxrwx 8 root root 4096 июн  6 22:32 vms
root@storage-1:~# ls -la /disk/backup/storage-1/disk/vms/images/
итого 12
drwxr-xr-x 3 nobody nogroup 4096 июн  6 23:01 .
drwxrwxrwx 8 root root 4096 июн  6 22:32 ..
drwxr----- 2 nobody nogroup 4096 июн  6 23:31 301
root@storage-1:~# █

```

Рисунок 2.68 – Успешная отработка скрипта

После резервного копирования стоит наконец-то проверить работу сетевых ресурсов. На одном из Proxmox-серверов произведем команду ручного монтирования. Из вывода всех примонтированных ресурсов можно увидеть и NFS.

```

root@pve-2:~# mkdir /vms
root@pve-2:~# mount.nfs4 192.168.101.16:/vms /vms
mount.nfs4: access denied by server while mounting 192.168.101.16:/vms
root@pve-2:~# mount.nfs4 192.168.101.16:/vms /vms
root@pve-2:~# df -h
Filesystem                Size      Used Avail Use% Mounted on
udev                      7.9G         0   7.9G   0% /dev
tmpfs                     1.6G       1.1M   1.6G   1% /run
/dev/mapper/pve-root      13G       3.0G   8.8G  26% /
tmpfs                     7.9G        60M   7.9G   1% /dev/shm
tmpfs                     5.0M         0   5.0M   0% /run/lock
tmpfs                     1.6G         0   1.6G   0% /run/user/0
/dev/fuse                 128M       32K   128M   1% /etc/pve
192.168.101.16:/vms       28G       5.4G   22G   21% /vms
root@pve-2:~#

```

Рисунок 2.69 – Проверка ограничений и монтирования сетевых ресурсов в системе

Из системы ресурс стоит убрать и перейти в веб-интерфейс в раздел «Storage». В нем необходимо добавить новый NFS-ресурс и указать нужные данные.

Рисунок 2.70 – Добавление NFS тома хранения

После создания этот ресурс автоматически появится на всех серверах кластера.

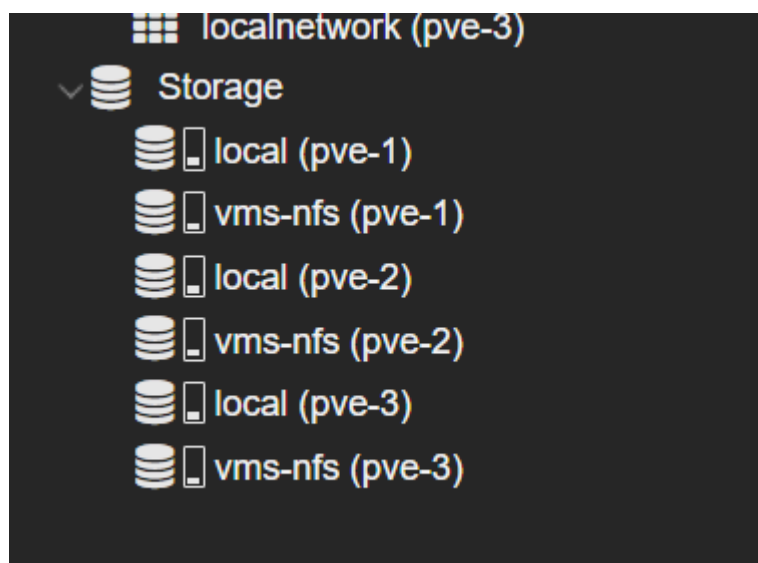


Рисунок 2.72 – Смонтированные ресурсы на серверах

В качестве шаблона и проверки работы ресурса нужно создать новый LXC-шаблон для Debian 12.

Рисунок 2.73 – Создание новой машины

В конце создания можно увидеть все настройки, с которыми будет создан новый LXC-контейнер.

Create: LXC Container

General

Template

Disks

CPU

Memory

Network

DNS

Confirm

Key ↑	Value
features	nesting=1
hostname	debian-lxc
memory	512
nameserver	192.168.122.1
net0	name=eth0,bridge=svrbr0,firewall=1,ip=10.0.2.20/24,gw=10.0.2.1
nodename	pve-1
ostemplate	vms-nfs:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst
pool	
rootfs	vms-nfs:10
ssh-public-keys	
swap	512
tags	template
unprivileged	1
vmid	301

☒ Start after created

Advanced ☒

Back

Finish

Рисунок 2.74 – Итоговые настройки машины

В случае успешного создания контейнера можно сказать о том, что новое хранилище и кластер успешно функционируют.

Так как в сети будут функционировать веб-сервисы, на которых предпочтительно использовать SSL, стоит развернуть центр сертификации. Разворачиваться ЦС будет из скрипта, на отдельной виртуальной машине. Процесс создания машины и инфраструктурных сервисов показан на рисунках 2.75-2.91.

Сначала нужно создать клон шаблона, на котором будут реализованы базовые сервисы.

Clone CT 301 (debian-lxc)

Target node:

pve-1

Target Storage:

Same as source

CT ID:

100

Hostname:

Kanboard

Resource Pool:

Help

Clone

Рисунок 2.75 – Настройки клона контейнера

После успешного клонирования следует перепроверить сетевые настройки. Необходимо наличие двух подключений в мост для VLAN 101 и для VLAN 102.

Add	Remove	Edit							
ID ↑	Name	Bridge	Firewall	VLAN Tag	MAC address	IP address	Gateway	MTU	Disconnected
net0	eth0	srvbr0	Yes		BC:24:11:49:...	10.0.2.21/24	10.0.2.1		No
net1	eth1	MGMT	Yes		BC:24:11:C2:...	192.168.101.21/24	192.168.101.1		No

Рисунок 2.76 – Настройки сети клона

Как будет настроена сеть, нужно залогиниться в машину под созданными в шаблоне учетными данными и провести базовую настройку, как и для всех серверных машин. После этой настройки нужно установить систему контейнеризации, в которой будут развернуты некоторые сервисы, указанные в требованиях [11].

```
root@Services:~# sh get-docker.sh
# Executing docker install script, commit: 53a22f61c0628e58e1d6680b49e82993d304b449
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install ca-certificates curl >/dev/null
+ sh -c install -m 0755 -d /etc/apt/keyrings
+ sh -c curl -fsSL "https://download.docker.com/linux/debian/gpg" -o /etc/apt/keyrings/docker.asc
+ sh -c chmod a+r /etc/apt/keyrings/docker.asc
+ sh -c echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian bookworm stable" > /etc/apt/sources.list.d/docker.lis
t
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io docker-compose-plugin docker-ce-rootless-extras docker-buildx-pl
ugin >/dev/null
```

Рисунок 2.77 – Установка системы контейнеризации

Пока что момент контейнеров стоит опустить и приступить к центру сертификации, без которых сервисы в контейнерах не будут нормально функционировать. Можно развернуть центр вручную или через скрипты. Для второго варианта необходимо локально скопировать репозиторий, в котором содержатся скрипты.

```
[root@Services ~]# git clone https://github.com/dhxcg/ls-la
Cloning into 'ls-la'...
remote: Enumerating objects: 1194, done.
remote: Counting objects: 100% (529/529), done.
remote: Compressing objects: 100% (358/358), done.
remote: Total 1194 (delta 266), reused 316 (delta 153), pack-reused 665 (from 2)
Receiving objects: 100% (1194/1194), 9.94 MiB | 6.46 MiB/s, done.
Resolving deltas: 100% (446/446), done.
[root@Services ~]# chmod +x ls-la/alt/scripts/openssl-*
[root@Services ~]#
```

Рисунок 2.78 – Клонирование репозитория

Как репозиторий будет скопирован, следует аналогично предыдущим требованиям назначить права. Исходный код скрипта представлен на рисунке 2.89.


```
# VARS
DIR="/ca"
CN="Root CA Consyst-Cloud-Infrastructure"
DAYS=3650
CONF="/var/lib/ssl/openssl.cnf"

# Install OpenSSL and backup config
apt-get update && apt-get install openssl curl -y
cp ${CONF}(.bak)

curl https://raw.githubusercontent.com/dhxcg/ls-la/refs/heads/main/storage-configs/openssl.cnf > ${CONF}

# CA structure
mkdir -p ${DIR}/{certs,newcerts,crl,private}
touch ${DIR}/index.txt
echo '00' > ${DIR}/serial

# Generate private key and CSR
openssl req -new -nodes \
  -newkey rsa-4096 \
  -keyout ${DIR}/private/cakey.pem \
  -out ${DIR}/cacert.csr \
  -subj "/C=RU/CN=${CN}" \
  -config ${CONF} \
  -extensions v3_ca

# Self-sign the root certificate
openssl ca -selfsign \
  -config ${CONF} \
  -in ${DIR}/cacert.csr \
  -out ${DIR}/cacert.pem \
  -extensions v3_ca \
  -keyfile ${DIR}/private/cakey.pem \
  -cert ${DIR}/cacert.pem \
  -days ${DAYS} \


```

Рисунок 2.79 – Код скрипта

В разделе «VARs» нужно задать свои данные, с которыми будет развернут ЦС. После запуска скрипта в результате должен выводиться корневой сертификат.

```
Using configuration from /var/lib/ssl/openssl.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
  Serial Number: 0 (0x0)
  Validity
    Not Before: Jun  3 15:18:59 2025 GMT
    Not After : Jun  1 15:18:59 2035 GMT
  Subject:
    countryName      = RU
    commonName       = Root CA Consyst-Cloud-Infrastructure
  X509v3 extensions:
    X509v3 Subject Key Identifier:
      67:74:19:15:4B:6E:66:25:D1:1A:9E:3E:BD:8C:21:BD:04:5A:6E:31
    X509v3 Authority Key Identifier:
      keyid:67:74:19:15:4B:6E:66:25:D1:1A:9E:3E:BD:8C:21:BD:04:5A:6E:31

    X509v3 Basic Constraints:
      CA:TRUE
Certificate is to be certified until Jun  1 15:18:59 2035 GMT (3650 days)
Sign the certificate? [y/n]:y

1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
[root@CA scripts]#
```

Рисунок 2.80 – Результат выполнения скрипта

Центр нужен для подписи сертификатов для сервисов. Из сервисов будут развернуты:

- Kanboard, который нужен для организации процесса работы всех разработчиков в команде;

- Zabbix, для мониторинга сетевых устройств и серверного сегмента;
- Bookstack, для ведения документации к разрабатываем продуктам.

При запуске скрипта необходимо указать доменное имя и директорию, где будут храниться выпущенные сертификаты. Этот процесс необходимо повторить для каждого сервиса.

```
[root@Services scripts]# ./openssl-enroll.sh zabbix.cloud.consyst.local zabbix
1. Generating client private key: zabbix/zabbix.cloud.consyst.local.key
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
2. Creating CSR with CN=zabbix.cloud.consyst.local: zabbix/zabbix.cloud.consyst.local.csr
3. Signing CSR with root CA
Using configuration from /var/lib/ssl/openssl.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
  Serial Number: 1 (0x1)
  Validity
    Not Before: Jun  7 17:23:57 2025 GMT
    Not After : Jun  7 17:23:57 2026 GMT
  Subject:
    commonName                = zabbix.cloud.consyst.local
  X509v3 extensions:
    X509v3 Basic Constraints:
      CA:FALSE
    Netscape Comment:
      OpenSSL Generated Certificate
    X509v3 Subject Key Identifier:
      E0:46:3D:8A:A4:4B:55:D2:5F:D0:95:76:83:A2:6B:56:32:A9:EC:1B
    X509v3 Authority Key Identifier:
      keyid:A3:48:2B:53:6F:2D:D5:7D:1A:2F:77:2D:A9:83:2B:60:5E:B9:03:1F

Certificate is to be certified until Jun  7 17:23:57 2026 GMT (365 days)

Write out database with 1 new entries
Data Base Updated
4. Creating PKCS#12 package: zabbix/zabbix.cloud.consyst.local.p12
Enter Export Password:
Verifying - Enter Export Password:
Done! Files saved in zabbix:
- Private key: zabbix/zabbix.cloud.consyst.local.key
- CSR: zabbix/zabbix.cloud.consyst.local.csr
- Certificate: zabbix/zabbix.cloud.consyst.local.crt
- PKCS#12 bundle: zabbix/zabbix.cloud.consyst.local.p12
[root@Services scripts]#
```

Рисунок 2.81 – Выпуск сертификата

Для того, чтобы централизованно управлять доступом ко всем развертываемым сервисам следует развернуть Nginx в качестве обратного прокси-сервера [4]. Именно в нем будут реализованы виртуальные хосты и к ним добавлены сертификаты.

```
[root@Services bookstack]# apt-get install nginx
Reading Package Lists... Done
Building Dependency Tree... Done
The following NEW packages will be installed:
  nginx
0 upgraded, 1 newly installed, 0 removed and 20 not upgraded.
Need to get 688kB of archives.
After unpacking 2636kB of additional disk space will be used.
Get:1 http://ftp.altlinux.org p10/branch/x86_64/classic nginx 1.26.3-alt1:p10+375326.100.2.1@1740132320 [688kB]
Fetched 688kB in 0s (1065kB/s)
Committing changes...
Preparing...
Installing...
1: nginx-1.26.3-alt1
Done.
[root@Services bookstack]#
```

Рисунок 2.82 – Установка nginx

В каждом виртуальном хосте следует описать доменное имя сервиса, его сертификаты и путь, куда перенаправлять запросы, которые попали на этот виртуальный хост.

```

GNU nano 7.2 /etc/nginx/sites-available.d/proxy-bkcst.conf
server {
    listen 443 ssl;
    server_name bookstack.cloud.consyst.local;

    ssl_certificate /certs/bookstack/bookstack.cloud.consyst.local.crt;
    ssl_certificate_key /certs/bookstack/bookstack.cloud.consyst.local.key;

    proxy_ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
    proxy_ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://localhost:3003/;
    }
}

```

Рисунок 2.83 – Конфигурация виртуального хоста в роли обратного прокси

Как будут созданы все виртуальные хосты, стоит перезапустить Nginx проверить, слушает ли он необходимые порты.

```

[root@services bookstack]# ss -tlnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
Process
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:*
users: (("chronyd",pid=2094,fd=5))
udp UNCONN 0 0 [::]:323 [::]:*
users: (("chronyd",pid=2094,fd=6))
tcp LISTEN 0 511 0.0.0.0:80 0.0.0.0:*
users: (("nginx",pid=5831,fd=7), ("nginx",pid=5830,fd=7), ("nginx",pid=5829,fd=7), ("nginx",pid=5828,fd=7), ("nginx",pid=5827,fd=7), ("nginx",pid=5826,fd=7), ("nginx",pid=5825,fd=7), ("nginx",pid=5824,fd=7), ("nginx",pid=5823,fd=7), ("nginx",pid=5822,fd=7), ("nginx",pid=5821,fd=7))
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
users: (("sshd",pid=2869,fd=3))
tcp LISTEN 0 511 0.0.0.0:443 0.0.0.0:*
users: (("nginx",pid=5831,fd=6), ("nginx",pid=5830,fd=6), ("nginx",pid=5829,fd=6), ("nginx",pid=5828,fd=6), ("nginx",pid=5827,fd=6), ("nginx",pid=5826,fd=6), ("nginx",pid=5825,fd=6), ("nginx",pid=5824,fd=6), ("nginx",pid=5823,fd=6), ("nginx",pid=5822,fd=6), ("nginx",pid=5821,fd=6))
tcp LISTEN 0 128 [::]:22 [::]:*
users: (("sshd",pid=2869,fd=4))

```

Рисунок 2.84 – Проверка портов у nginx

Если развертывание Nginx было выполнено без ошибок, стоит приступить к развертыванию конечных сервисов. Для этого нужно создать директорию и в ней compose-файл. В нем описать параметры и образ развертываемого сервиса.

```

GNU nano 7.2
services:
  kanboard:
    image: kanboard/kanboard:latest
    volumes:
      - ./kanboard_data:/var/www/app/data
      - ./kanboard_plugins:/var/www/app/plugins
      - ./config.php:/var/www/app/config.php:ro
    ports:
      - 127.0.0.1:3002:80
    environment:
      PLUGIN_INSTALLER: true

```

Рисунок 2.85 – Конфигурация compose-файла для Kanboard

После написания compose-файла следует запустить проект. Указанные образы автоматически скачаются и добавятся в систему.

```
[root@Services kanboard]# docker compose down
[+] Running 2/2
✓ Container kanboard-kanboard-1 Removed
✓ Network kanboard_default Removed
[root@Services kanboard]# docker compose up -d
[+] Running 2/2
✓ Network kanboard_default Created
✓ Container kanboard-kanboard-1 Started
[root@Services kanboard]# docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
1037bb4f25a6   kanboard/kanboard:latest  "/usr/local/bin/entr..."  2 seconds ago  Up 2 seconds  443/tcp, 127.0.0.1:3002->80/tcp    kanboard-kanboard-1
[root@Services kanboard]#
```

Рисунок 2.86 – Запуск Kanboard

Далее можно заходить в веб-интерфейс по указанному адресу. Посмотрев сертификат можно убедиться, что и обратный прокси, и сам проект работают.

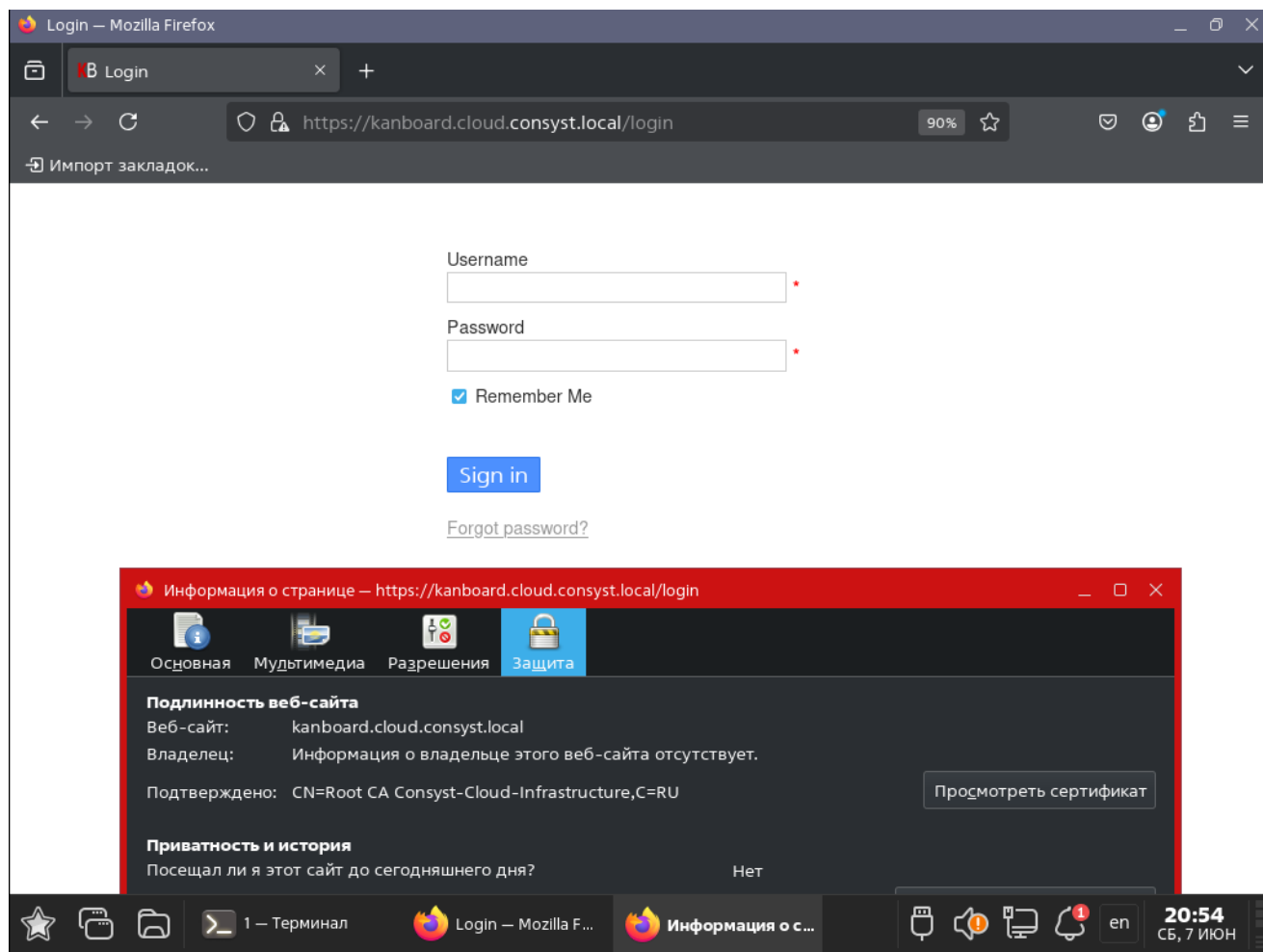


Рисунок 2.87 – Проверка работы обратного прокси, SSL и Kanboard

Следующим сервисом следует развернуть Zabbix. Для этого точно также необходимо написать файл для сборки проекта, а после запустить его. Проект для Zabbix выглядит чуть сложнее, чем остальные. Так как сервис требует и базы данных, и веб-сервера, и самого Zabbix-сервера, эти задачи разделяют между контейнерами. Также добавляются зависимости, что контейнеры запускались в строго установленном порядке и сам веб-сервер не мог запуститься раньше, чем сервер БД и Zabbix-сервер [12].

```

GNU nano 7.2                                                                 docker-compose.yml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: "zabbix"
      POSTGRES_USER: "zabbix"
      POSTGRES_PASSWORD: "zabbix"
    restart: always

  zabbix-server:
    image: zabbix/zabbix-server-pgsql:ubuntu-latest
    environment:
      DB_SERVER_HOST: "db"
      POSTGRES_USER: "zabbix"
      POSTGRES_PASSWORD: "zabbix"
      POSTGRES_DB: "zabbix"
    depends_on:
      - db
    restart: always

  zabbixweb:
    image: zabbix/zabbix-web-nginx-pgsql:ubuntu-latest
    environment:
      DB_SERVER_HOST: "db"
      POSTGRES_USER: "zabbix"
      POSTGRES_PASSWORD: "zabbix"
      POSTGRES_DB: "zabbix"
      ZBX_SERVER_HOST: "zabbix-server"
      PHP_TZ: "Europe/Moscow"
    depends_on:
      - db
      - zabbix-server
    ports:
      - "127.0.0.1:3001:8080"
    restart: always

```

Рисунок 2.88 – Содержимое compose-файла для Zabbix

После запуска проекта можно переходить в браузер и проверять работу виртуального хоста и самого Zabbix.

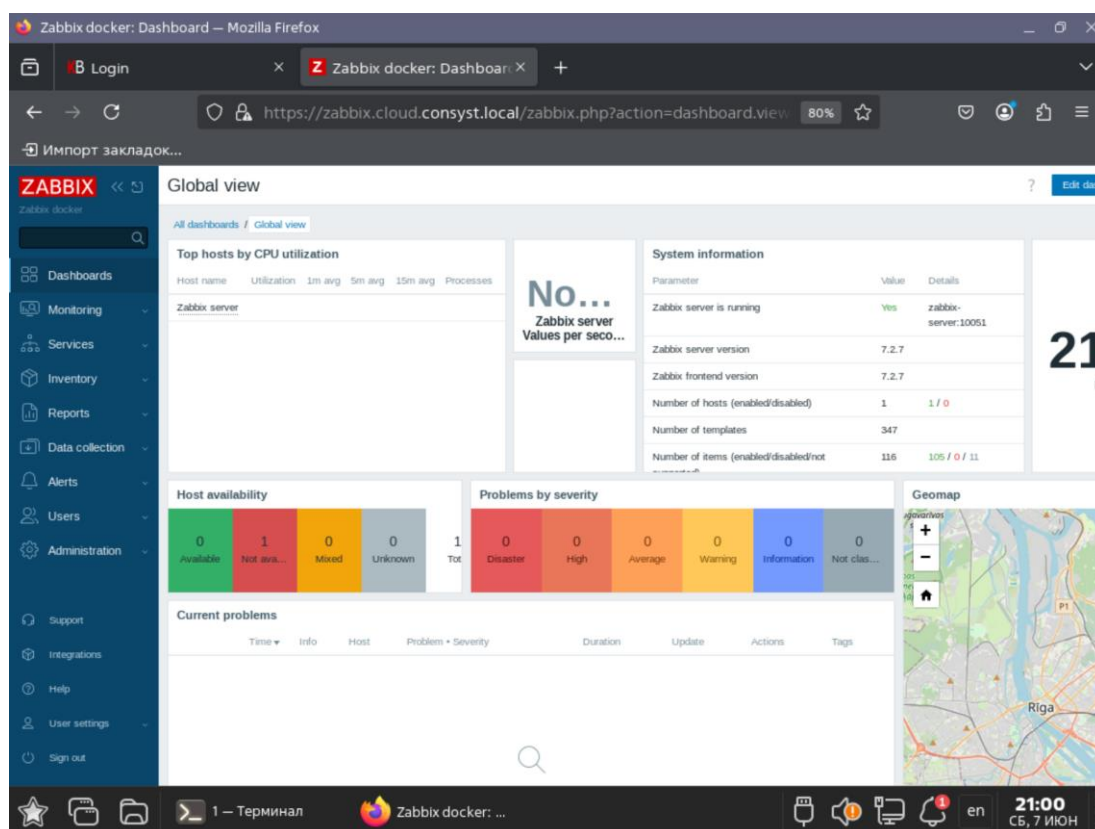


Рисунок 2.89 – Проверка работы обратного прокси, SSL и Zabbix

Последним сервисом станет Bookstack. Уже стандартно надо написать файл проекта. Требуется БД, которую можно развернуть или на отдельном сервере, или сразу в отдельном контейнере.

```

GNU nano 7.2                                     docker-compose.yml
services:
  bookstack:
    image: lscr.io/linuxserver/bookstack:latest
    container_name: bookstack
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Etc/UTC
      - APP_URL='https://bookstack.cloud.consyst.local'
      - APP_KEY='base64:HBMw9jHlr8h8TGcLKTco7ZJa0e8JDvBu1X5CXvJ5+Pw='
      - DB_HOST=bookstack-db
      - DB_PORT=3306
      - DB_USERNAME=kk
      - DB_PASSWORD=kk
      - DB_DATABASE=kk
    volumes:
      - ./bookstack/config:/config
    ports:
      - 127.0.0.1:3003:80
    restart: unless-stopped
    depends_on:
      - mysql

  mysql:
    container_name: bookstack-db
    image: mysql:5.7
    environment:
      - MYSQL_ROOT_PASSWORD=kk
      - MYSQL_USER=kk
      - MYSQL_PASSWORD=kk
      - MYSQL_DATABASE=kk
    volumes:
      - ./db:/var/lib/mysql
  
```

Рисунок 2.90 – Содержимое compose-файла для Bookstack

После запуска можно заходить в браузер и проверять работу Bookstack.

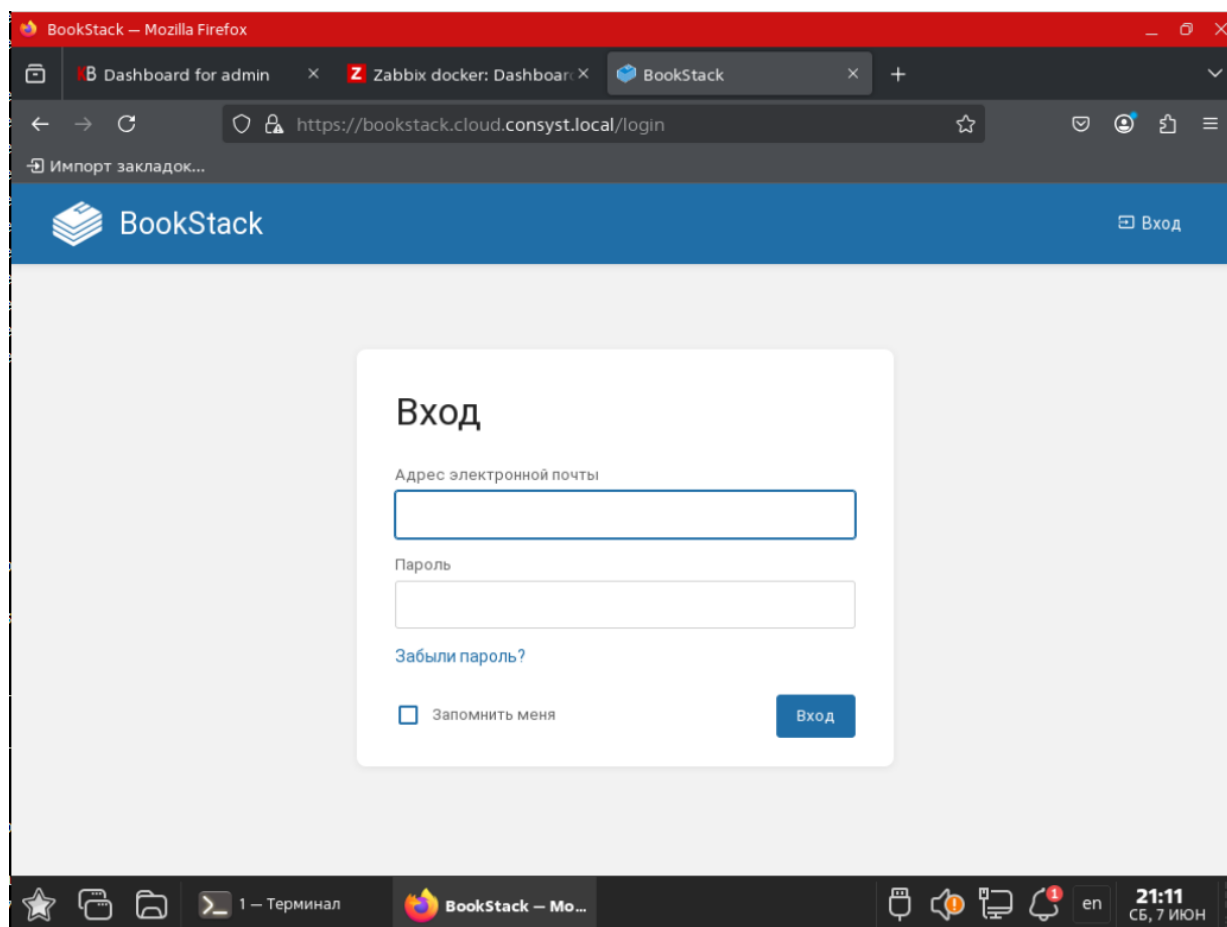


Рисунок 2.91 – Проверка веб-интерфейса, SSL и обратного прокси

Теперь можно приступить к настройке доменной инфраструктуры под управлением ALD PRO. Первым делом стоит переключить уровень защищенности на максимальный «Смоленск» и включить МКЦ. Процесс развертывания и настройки ALD Pro показан на рисунках 2.92-2.110.

```
root@dc-1:~# astra-modeswitch get
0
root@dc-1:~# astra-modeswitch set 2
update-initramfs: Generating /boot/initrd.img-5.4.0-162-generic
root@dc-1:~# astra-modeswitch get
2
root@dc-1:~# █
```

Рисунок 2.92 – Настройка уровня защищенности

Посмотреть статус МКЦ можно через команду «astra-mic-control status», так как он имеет свойство не включаться с первого раза.

```
administrator@dc-1:~$ sudo astra-mic-control status
АКТИВНО
administrator@dc-1:~$ █
```

Рисунок 2.93 – Проверка МКЦ

После следует выставить репозитории. В Astra Linux имеется соответствие версий системы и версий ALD PRO, их необходимо учитывать. Для версии ALD 2.4.0 нужно выставить одну из последних версий Astra Linux - 1.7.6.

```
GNU nano 3.2 /etc/apt/sources.list

# Astra Linux repository description https://wiki.astralinux.ru/x/0oLiC

# Новые репозитории
deb https://dl.astralinux.ru/astra/frozen/1.7_x86-64/1.7.6/repository-base/ 1.7_x86-64 main contrib non-free
deb https://dl.astralinux.ru/astra/frozen/1.7_x86-64/1.7.6/repository-extended/ 1.7_x86-64 main contrib non-free

#deb cdrom:[OS Astra Linux 1.7.0 1.7_x86-64 DVD 1/ 1.7_x86-64 contrib main non-free
#deb https://download.astralinux.ru/astra/stable/1.7_x86-64/repository-main/ 1.7_x86-64 main contrib non-free
#deb https://download.astralinux.ru/astra/stable/1.7_x86-64/repository-update/ 1.7_x86-64 main contrib non-free
#deb https://download.astralinux.ru/astra/stable/1.7_x86-64/repository-base/ 1.7_x86-64 main contrib non-free
#deb https://download.astralinux.ru/astra/stable/1.7_x86-64/repository-extended/ 1.7_x86-64 main contrib non-free

#deb https://download.astralinux.ru/astra/stable/1.7_x86-64/uu/last/repository-update/ 1.7_x86-64 main contrib non-free
```

Рисунок 2.94 – Установка новых репозитория

При обновлении можно увидеть только два прописанных репозитория и отсутствие ошибок. Если вывод аналогичный – то все отработало успешно.

```
root@dc-1:~# apt update
Сущ:1 https://dl.astralinux.ru/astra/frozen/1.7_x86-64/1.7.6/repository-base 1.7_x86-64 InRelease
Сущ:2 https://dl.astralinux.ru/astra/frozen/1.7_x86-64/1.7.6/repository-extended 1.7_x86-64 InRelease
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Все пакеты имеют последние версии.
root@dc-1:~# █
```

Рисунок 2.95 – Обновление списка пакетов

После надо обновить все имеющиеся в системе пакеты для соответствия версии 1.7.6. В Astra Linux простое обновление через «upgrade» напроочь ломает систему, поэтому необходимо использовать команду «sudo apt dist-upgrade -y -o Dpkg::Options::=--force-confold», чтобы система могла автоматически удалять пакеты старых версий. После долгого обновления необходимо перезагрузить машину [9].

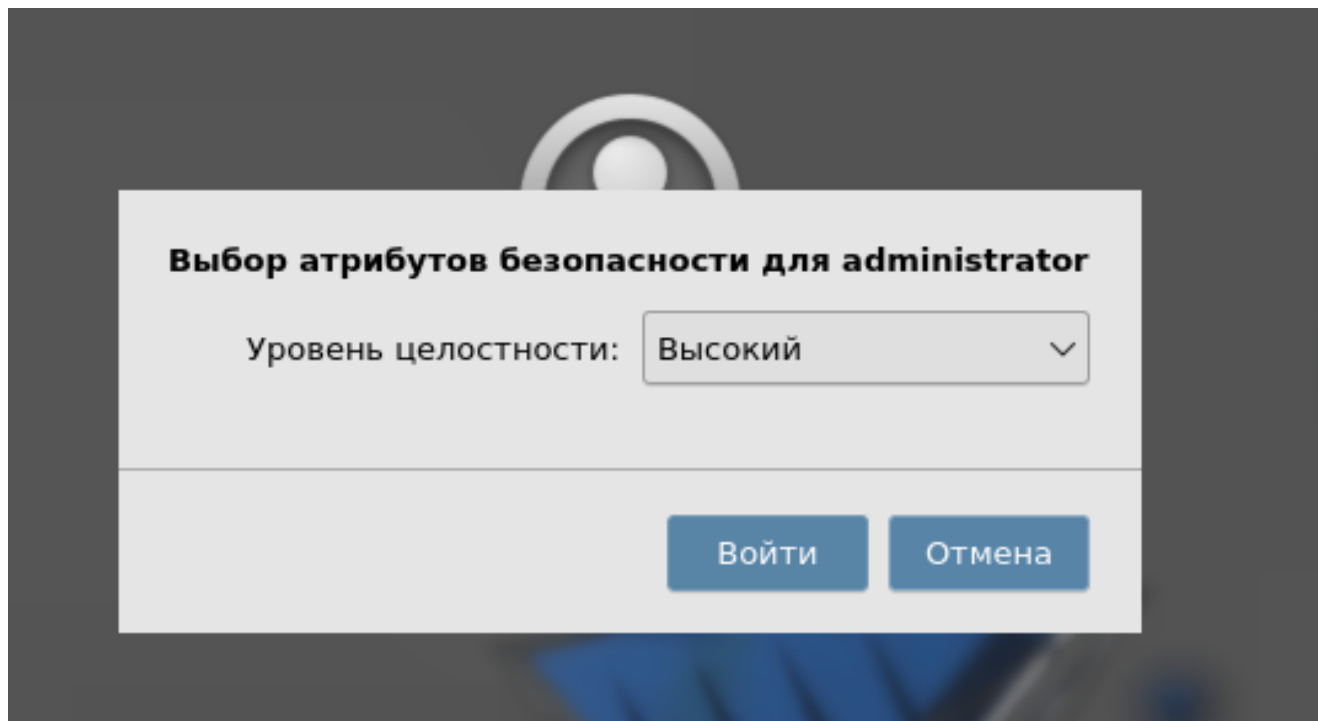


Рисунок 2.96 – Выбор уровня целостности

После обновления версия системы должна была обновиться в соответствии с установленными репозиториями.

```
root@dc-1:~# cat /etc/os-release
PRETTY_NAME="Astra Linux"
NAME="Astra Linux"
ID=astra
ID_LIKE=debian
ANSI_COLOR="1;31"
HOME_URL="https://astralinux.ru"
SUPPORT_URL="https://astralinux.ru/support"
LOGO=astra
VERSION_ID=1.7_x86-64
VERSION_CODENAME=1.7_x86-64
root@dc-1:~# cat /etc/astra_version
1.7.6
root@dc-1:~# cat /etc/astra
astra/          astra_license          astra-safepolicy.conf  astra_version
root@dc-1:~# cat /etc/astra/*
1.7.6.11
1.7.6.ext2.1
1.7.6.ext2
root@dc-1:~# █
```

Рисунок 2.97 – Проверка версии ОС

Теперь надо снова установить репозиторий, но уже для ALD. Далее обновить список пакетов.

```
administrator@dc-1:~$ sudo cat /etc/apt/sources.list.d/aldpro.list
deb https://dl.astralinux.ru/aldpro/frozen/01/2.4.0 1.7_x86-64 main base
administrator@dc-1:~$ sudo apt update
Сущ:1 https://dl.astralinux.ru/astra/frozen/1.7_x86-64/1.7.6/repository-base 1.7_
x86-64 InRelease
Сущ:2 https://dl.astralinux.ru/astra/frozen/1.7_x86-64/1.7.6/repository-extended
1.7_x86-64 InRelease
Сущ:3 https://dl.astralinux.ru/aldpro/frozen/01/2.4.0 1.7_x86-64 InRelease
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Все пакеты имеют последние версии.
administrator@dc-1:~$
```

Рисунок 2.98 – Обновление с новыми репозиториями

После необходимо установить пакеты для ALD PRO. Также стоит отметить, что «aldpro-gc» и «aldpro-syncer» устанавливаются для дальнейшей настройки глобального каталога.

```
administrator@dc-1:~$ sudo DEBIAN_FRONTEND=noninteractive \
> apt-get install -y -q aldpro-mp aldpro-gc aldpro-syncer
Чтение списков пакетов...
Построение дерева зависимостей...
Чтение информации о состоянии...
Следующие пакеты устанавливались автоматически и больше не требуются:
 kgamma5 liblivemedia64 libopts25 libplymouth4 libsnmp30 libwpd-1.0-1
 libwpbackend-fdo-1.0-1 libxcb-util0
```

Рисунок 2.99 – установка необходимых пакетов

Процесс установки должен закончиться настройкой пакетов самого ALD PRO. В случае ошибок необходимо проверять, как происходило обновление системы.

```
Настраивается пакет aldpro-mp-ui-audit (2.4.0-18) ...
httpd not running, trying to start
Настраивается пакет aldpro-mp-ui-automation (2.4.0-17) ...
Настраивается пакет aldpro-mp-ui-monitoring (2.4.0-20) ...
Настраивается пакет aldpro-mp-ui-domain-management (2.4.0-90) ...
Настраивается пакет aldpro-mp-ui-software-management (2.4.0-28) ...
Настраивается пакет aldpro-mp-ui-services (2.4.0-49) ...
Настраивается пакет aldpro-mp-ui-users-and-computers (2.4.0-59) ...
Настраивается пакет aldpro-mp-ui-group-policies (2.4.0-36) ...
Настраивается пакет aldpro-mp (2.4.0-28) ...
Created symlink /etc/systemd/system/timers.target.wants/dirsrv-watcher.timer → /l
ib/systemd/system/dirsrv-watcher.timer.
dirsrv-watcher.service is a disabled or a static unit, not starting it.
Настраивается пакет aldpro-gc (2.4.0-58) ...
Настраивается пакет aldpro-help-center (2.4.0-4) ...
Обрабатываются триггеры для xserver-xorg-core (2:1.17-1ubuntu4astra.se28) ...
update exec ids due to /usr/bin changed
Обрабатываются триггеры для dbus (1.12.24-0+deb11u1.astra.se8+ci5) ...
Обрабатываются триггеры для libgost-astra (1.1.1+ci1+b1) ...
Finded reference to active OpenSSL config section [default_conf]
```

Рисунок 2.100 – Успешная установка серверной части

Конечное развертывание производится командой «aldpro-server-install». В параметрах указываются данные, с которыми будет развертываться домен.

```
administrator@dc-1:~$ sudo aldpro-server-install -n dc-1 -d cloud.consyst.local -p 'P@ssw0rd' \> --ip 10.0.2.10 --setup_gc --setup_syncer --no-reboot
```

Рисунок 2.101 – Команда развертывания контроллера домена

Процесс развертывания очень долгий. В конце установки пользователя должно вернуть в терминал.

```
-----
      ID: send_config_to_ldap
Function: module.run
  Result: True
Comment: aldpro_subsystems.send_config_to_ldap: True
Started: 18:13:28.700184
Duration: 36.047 ms
Changes:
-----
      aldpro_subsystems.send_config_to_ldap:
      True

Summary for local
-----
Succeeded: 5 (changed=5)
Failed:    0
-----
Total states run:    5
Total run time: 378.304 ms
administrator@dc-1:~$
administrator@dc-1:~$
administrator@dc-1:~$
administrator@dc-1:~$
```

Рисунок 2.102 – Успешное окончание развертывания

После развертывания нужно перезагрузить машину и указать вход в домен, а не в локального пользователя. Учетные данные указывать те же, что и при развертывании КД.

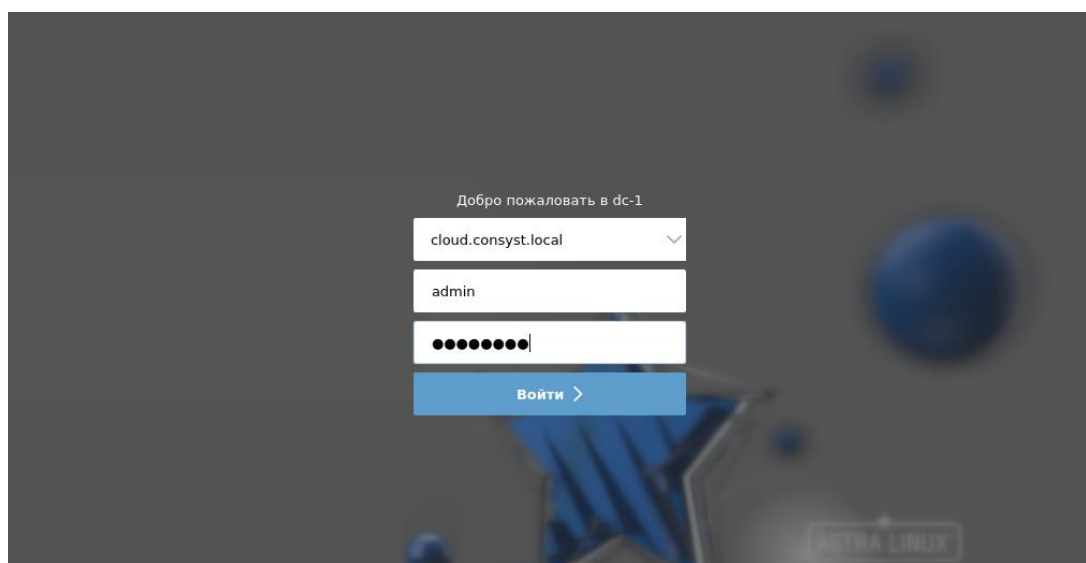


Рисунок 2.103 – Окно входа после перезагрузки

После входа стоит проверить id текущего пользователя и Kerberos-билет. Если такой имеется – домен успешно развернут и функционирует.

```
admin@dc-1:~$ id
uid=712600000(admin) gid=712600000(admins) gpyuny=712600000(admins),1001(astra-admin),712600003(1
padmin),1005611117(ald trust admin)
admin@dc-1:~$ klist
Ticket cache: KEYRING:persistent:712600000:krb_ccache_2f1s0c0
Default principal: admin@CLOUD.CONSYST.LOCAL

Valid starting      Expires            Service principal
06.06.2025 18:39:30  07.06.2025 18:11:09  ldap/dc-1.cloud.consynt.local@CLOUD.CONSYST.LOCAL
06.06.2025 18:39:25  07.06.2025 18:11:09  krbtgt/CLOUD.CONSYST.LOCAL@CLOUD.CONSYST.LOCAL
admin@dc-1:~$
```

Рисунок 2.104 – Проверка id и Kerberos-билетов

У ALD PRO имеется веб-интерфейс, через который предпочтительно настраивать домен. На данный момент в домене имеется наш контроллер домена.

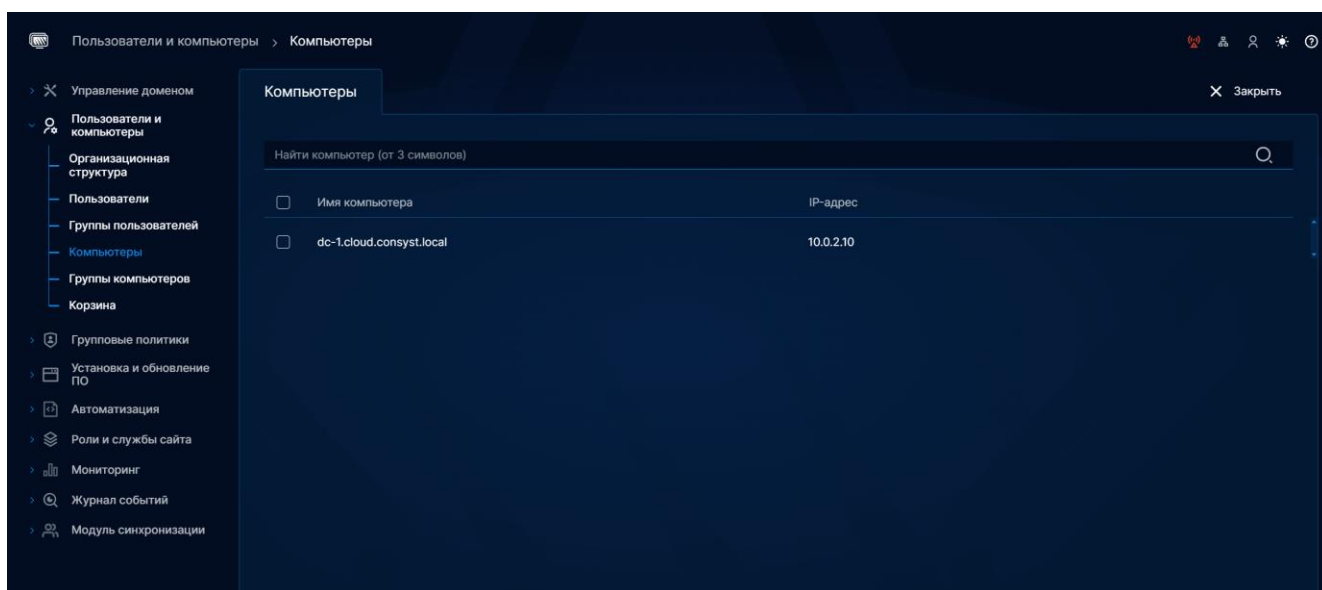


Рисунок 2.105 – Веб-интерфейс ALD Pro

Так как это будет единственный DNS-сервер в отделе, необходимо разрешить пересылку запросов к вышестоящим серверам, чтобы пользователи имели доступ в интернет.

```
/* dnssec-enable is obsolete and 'yes' by default */
dnssec-validation no;
allow-recursion { any; };
allow-query-cache { any; };
```

Рисунок 2.106 – Настройка DNS-сервера домена

После необходимо перезагрузить домен FreeIPA и удостовериться, что все корректно запустилось.

```
root@dc-1:~# nano /etc/bind/ipa-options-ext.conf
root@dc-1:~# ipactl restart
Restarting Directory Service
Restarting krb5kdc Service
Restarting kadmind Service
Restarting named Service
Restarting httpd Service
Restarting ipa-custodia Service
Restarting smb Service
Restarting winbind Service
Restarting ipa-otpd Service
Restarting ipa-dnskeysyncd Service
ipa: INFO: The ipactl command was successful
root@dc-1:~# ipactl stat
```

Рисунок 2.107 – Перезагрузка домена

За всю DNS-архитектуру в данный момент будет отвечать только ALD PRO, ввиду небольших масштабов. Предварительно предстоит создать обратные зоны для всех подсетей отдела.

Служба разрешения имён > Новая зона DNS

Основное

Имя зоны

обязательно

192.168.101.0/24

Тип зоны

☐ Прямой

☒ Обратный

Рисунок 2.108 – Создание DNS-зоны

Ниже показан список всех зон, которые должны быть созданы в ALD PRO.

Зоны DNS		Перенаправление запросов	Глобальная конфигурация DNS
Найти			
☐	Имя зоны	▼ Состояние	
☐	0.10.in-addr.arpa.	Включено	
☐	100.168.192.in-addr.arpa.	Включено	
☐	101.168.192.in-addr.arpa.	Включено	
☐	2.0.10.in-addr.arpa.	Включено	
☐	cloud.consyst.local.	Включено	

Рисунок 2.109 – Все зоны отдела

Так как для всех машин в домене записи создаются автоматически, останется создать записи для машин, которые нельзя полноценно ввести в домен, по типу серверов виртуализации.

Служба разрешения имён > Зона DNS: cloud.consyst.local. > Новая запись DNS

Основное

Имя записи обязательно

Тип записи обязательно

IP-адрес обязательно

Рисунок 2.110 – Процесс создания записи

После настройки DNS-записей базовая настройка всего серверного сегмента – закончена.

2.6 Настройка пользовательских ОС

Для того, чтобы удаленные клиенты могли иметь доступ к рабочим ресурсам отдела, необходимо настроить удаленный доступ. Изначально имеется два варианта – подключаться к VPN, который нужен для связи отделов или подключаться на Ideco NGFW напрямую, по внешнему адресу. В данном случае рассматривается именно второй вариант. Для этого на ПК удаленного работника необходимо скачать скрипт, который предоставляется компанией Ideco,

дать права и запустить. Процесс настройки клиентских машин, их ввода в домен, а также настройки подключения удаленных клиентов показан на рисунках 2.111–2.124.

```
root@astra:/home/administrator/Зарпузки# ls -al
итого 94536
drwxr-xr-x  2 administrator administrator   4096 июн  5 21:34 .
drwx----- 15 administrator administrator   4096 июн  5 21:33 ..
-rw-r--r--  1 administrator administrator   118 дек 17 14:45 .directory
-rw-r--r--  1 administrator administrator 50311168 июн  5 21:34 IdecoAgent.msi
-rw-r--r--  1 administrator administrator 46478914 июн  5 21:34 IdecoAgent.sh
root@astra:/home/administrator/Зарпузки# chmod +x IdecoAgent.sh
root@astra:/home/administrator/Зарпузки# nano IdecoAgent.sh
root@astra:/home/administrator/Зарпузки# nano IdecoAgent.sh
root@astra:/home/administrator/Зарпузки# ./IdecoAgent.sh
Installing Ideco Client 19.3.439.0 to /usr/local/Ideco/Agent ...
Detected OS: Astra Linux
Unpacking files ...
Installing systemd service: IdecoService.service ...
Created symlink /etc/systemd/system/multi-user.target.wants/IdecoService.service - /usr/local/lib/systemd/system/IdecoService.service.
Installing GUI application desktop shortcut ...
Removing other versions ...
Installation completed successfully.
root@astra:/home/administrator/Зарпузки#
```

Рисунок 2.111 – Установка Ideco Client

После этого установка быстро завершится и теперь следует добавить корневой сертификат МСЭ в директорию для доверенных сертификатов на клиенте.

```
root@astra:/home/administrator/Зарпузки# cp root_ca.crt /usr/share/ca-certificates/
root@astra:/home/administrator/Зарпузки# dpkg-reconfigure ca-certificates
ca-certificates ca-certificates-local
root@astra:/home/administrator/Зарпузки# dpkg-reconfigure ca-certificates
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Обрабатываются триггеры для ca-certificates (20230311+b1) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
```

Рисунок 2.112 – Копирование сертификата в хранилище и перенастройка пакета

После команды `dpkg-reconfigure` скрипты, написанные при сборке пакеты начнут обрабатывать заново и в терминале появится интерактивное меню. В нем необходимо найти скопированный сертификат и выбрать его.

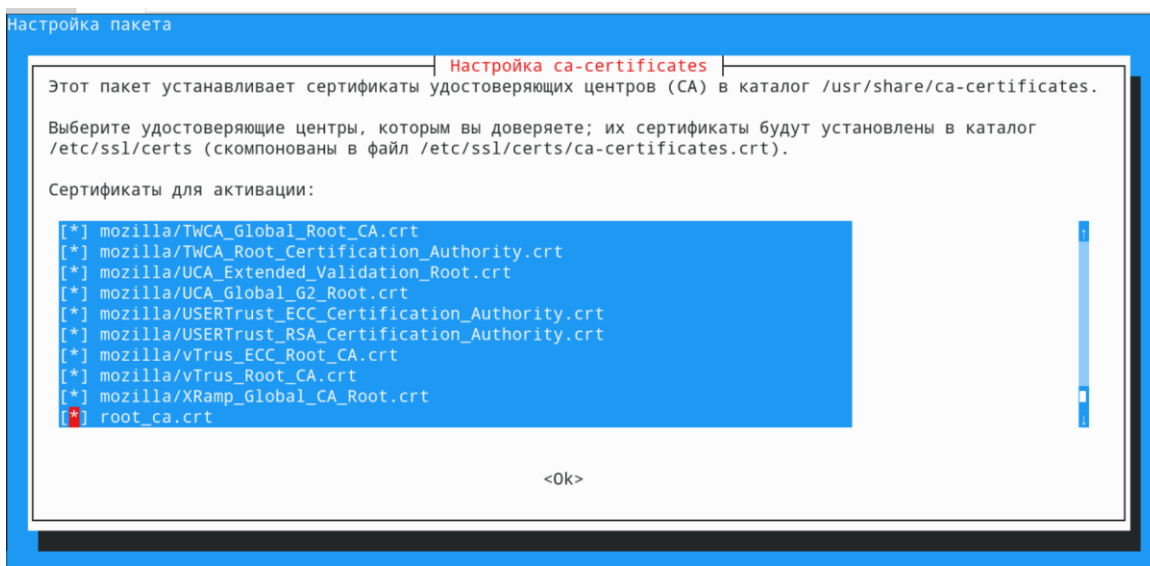


Рисунок 2.113 – Добавление нового сертификата

После этого можно запускать графическое приложение из меню запуска. Потребуется указать учетные данные клиента или администратора, созданного на МСЭ. После подключение успешно установится.

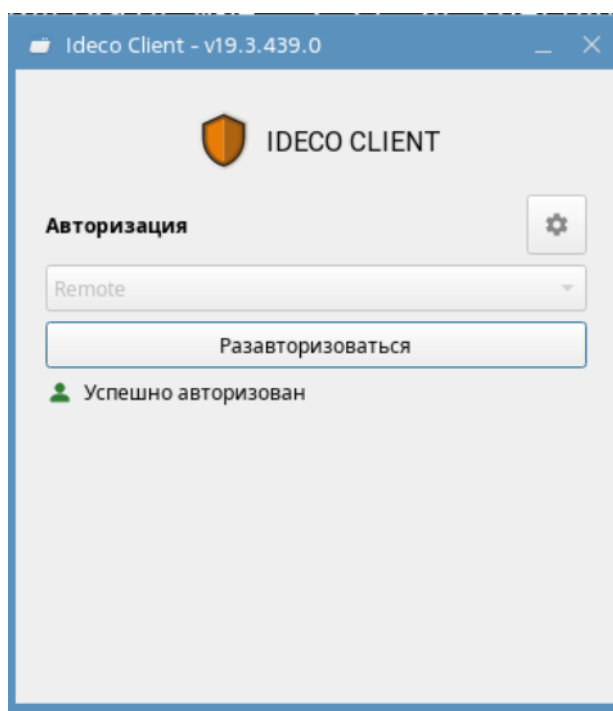


Рисунок 2.114 – Ideco Client и успешная авторизация

Также необходимо добавить сертификат от ЦС, который был развернут ранее. Процесс останется неизменным.

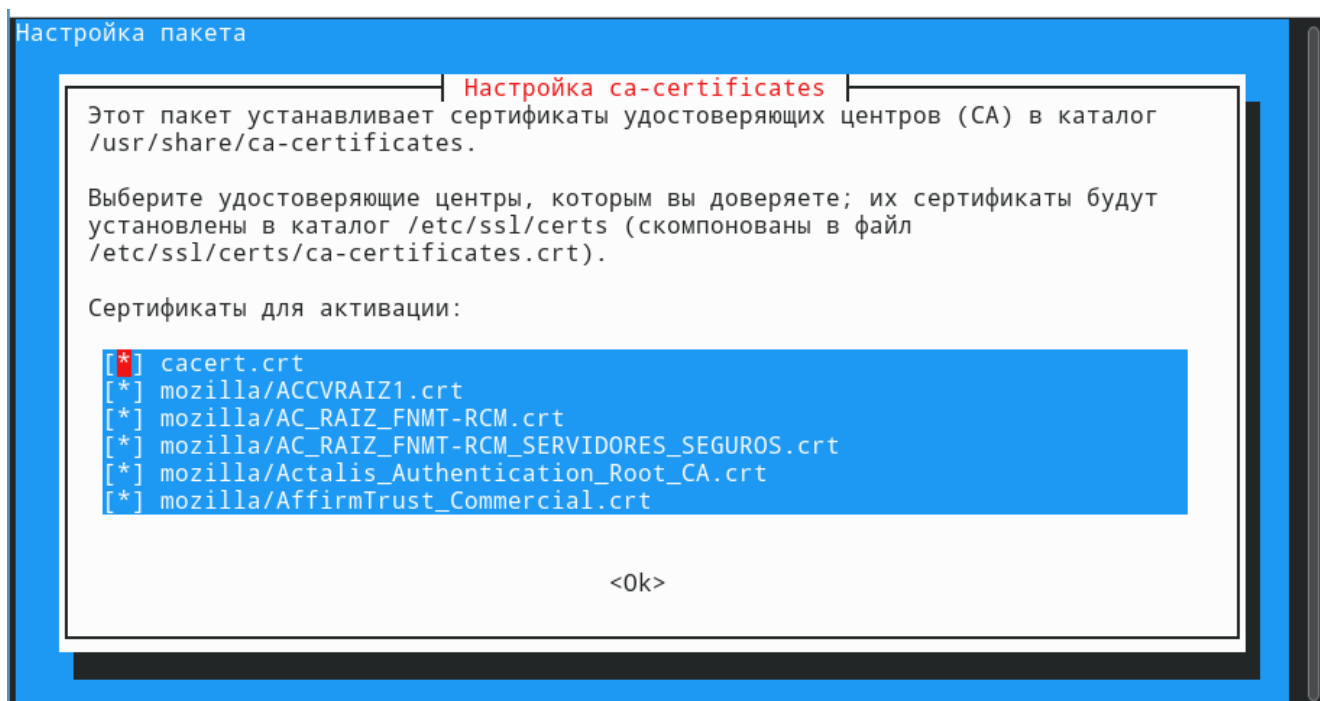


Рисунок 2.115 – Добавление центра сертификации в доверенные

Кроме системы сертификат также надо добавить в доверенные в браузере. Некоторые браузеры используют локальную базу сертификатов, а некоторым, по типу Firefox, их необходимо настраивать отдельно.

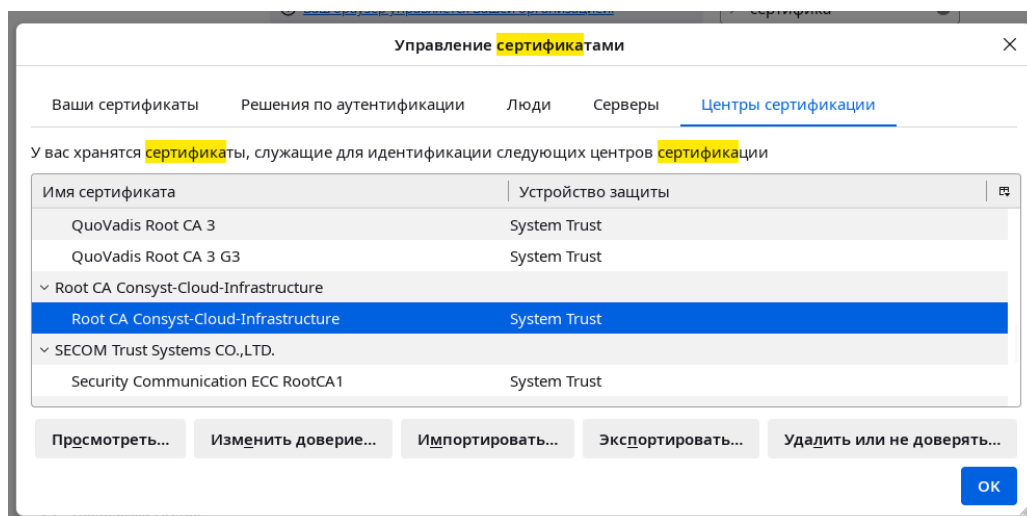


Рисунок 2.116 – Установка доверия в браузере

После установки сертификатов проверить можно командой «trust list». Она покажет абсолютно все сертификаты и для системы, и для браузера. Этот процесс необходимо проделать на каждом клиентском рабочем месте.

```
pkcs11:id=%A3%48%2B%53%6F%2D%D5%7D%1A%2F%77%2D%A9%83%2B%60%5E%B9%03%1F;type=cert
type: certificate
label: Root CA Consyst-Cloud-Infrastructure
trust: anchor
category: authority
```

Рисунок 2.117 – Проверка доверия в системе

Далее машины администраторов и клиентов необходимо ввести в домен. Чтобы ввести клиентские машины в домен ALD PRO, необходимо провести стандартный процесс с некоторыми изменениями. В первую очередь, также отключить NetworkManager. После настроить сетевые реквизиты и выставить в качестве DNS-сервера контроллер домена. Также необходимо выставить имя компьютера, чтобы КД смог добавить необходимую А и PTR запись.

```
root@astra:~# systemctl --now disable NetworkManager
Removed "/etc/systemd/system/multi-user.target.wants/NetworkManager.service".
Removed "/etc/systemd/system/dbus-org.freedesktop.nm-dispatcher.service".
Removed "/etc/systemd/system/network-online.target.wants/NetworkManager-wait-online.service".
root@astra:~# nano /etc/network/interfaces
root@astra:~# nano /etc/resolv.conf
root@astra:~# hostnamectl hostname pc-1.cloud.consyst.local
root@astra:~# nano /etc/hosts
root@astra:~#
```

Рисунок 2.118 – Предварительная настройка клиентской машины

Так как клиентские машины используют Astra Linux 1.8, репозитории будут отличаться.


```

GNU nano 7.2 /etc/apt/sources.list
#deb cdrom:[OS Astra Linux 1.8.1.6 1.8_x86-64 DVD ]/ 1.8_x86-64 contrib main non-free non-free-firmware
deb https://download.astralinux.ru/astra/stable/1.8_x86-64/repository-main/ 1.8_x86-64 main contrib non-free
#deb https://download.astralinux.ru/astra/stable/1.8_x86-64/repository-devel/ 1.8_x86-64 main contrib non-free
deb https://download.astralinux.ru/astra/stable/1.8_x86-64/repository-extended/ 1.8_x86-64 main contrib non-free
deb https://dl.astralinux.ru/aldpro/frozen/01/2.4.0 1.8_x86-64 main base

```

Рисунок 2.119 – Установка репозитория для Astra Linux 1.8.1

После обновления и установки репозитория необходимо установить клиентский набор пакетов, который позволит настроить машину.

```

root@pc-1:~# sudo DEBIAN_FRONTEND=noninteractive \
  apt-get install -y -q aldpro-client
Чтение списков пакетов...
Построение дерева зависимостей...
Чтение информации о состоянии...
Следующие пакеты устанавливались автоматически и больше не требуются:
 fonts-terminus-otb libxatracker2 libxvnc1 mtdev-tools python3-gpg samba-dsdb-modules x11-apps
 x11-session-utils xfonts-bolkhov-75dpi xfonts-bolkhov-isocyr-75dpi xfonts-bolkhov-isocyr-misc
 xfonts-bolkhov-misc xfonts-terminus xfonts-terminus-oblique xinit xkbset xorg
Для их удаления используйте «sudo apt autoremove».
Будут установлены следующие рекомендуемые пакеты:

```

Рисунок 2.120 – Установка клиентского ПО для ALD PRO

Команда ввода в домен похожа на развертывание КД, но флаги будут немного отличаться.

```

root@pc-1:~# sudo /opt/rbta/aldpro/client/bin/aldpro-client-installer --host pc-1 \
  --domain cloud.consyst.local --account admin --password 'P@ssw0rd' \
  --gui --force
systemctl stop aldpro-client-service-discovery.service
/usr/bin/astra-freeipa-client -d 'cloud.consyst.local' -u 'admin' -p 'P@ssw0rd' -y --par '--hostname=pc-1.cloud.consyst.local --force-join'
dpkg-query: пакет «ntpsec» не установлен, информация о нём недоступна
Для проверки файлов архивов используйте команду dpkg --info (dpkg-deb --info).
dpkg-query: пакет «ntp» не установлен, информация о нём недоступна
Для проверки файлов архивов используйте команду dpkg --info (dpkg-deb --info).
/usr/bin/astra-freeipa-client: строка 226: userpass: Нет такого файла или каталога
Discovery was successful!
Client hostname: pc-1.cloud.consyst.local
Realm: CLOUD.CONSYST.LOCAL
DNS Domain: cloud.consyst.local
IPA Server: dc-1.cloud.consyst.local
BaseDN: dc=cloud,dc=consyst,dc=local
Skipping chrony configuration
Successfully retrieved CA cert
  Subject: CN=CA Signing Certificate
  Issuer: CN=CA Signing Certificate
  Valid From: 2025-06-06 14:55:44
  Valid Until: 2045-06-01 14:55:44
Enrolled in IPA realm CLOUD.CONSYST.LOCAL
Created /etc/ipa/default.conf
Configured sudoers in /etc/nsswitch.conf

```

Рисунок 2.121 – Ввод в домен машины на Astra Linux 1.8.1

После выполнения вышеуказанных шагов необходимо перезагрузить машину. В окне авторизации появится домен. Нужно ввести доменные учетные данные.

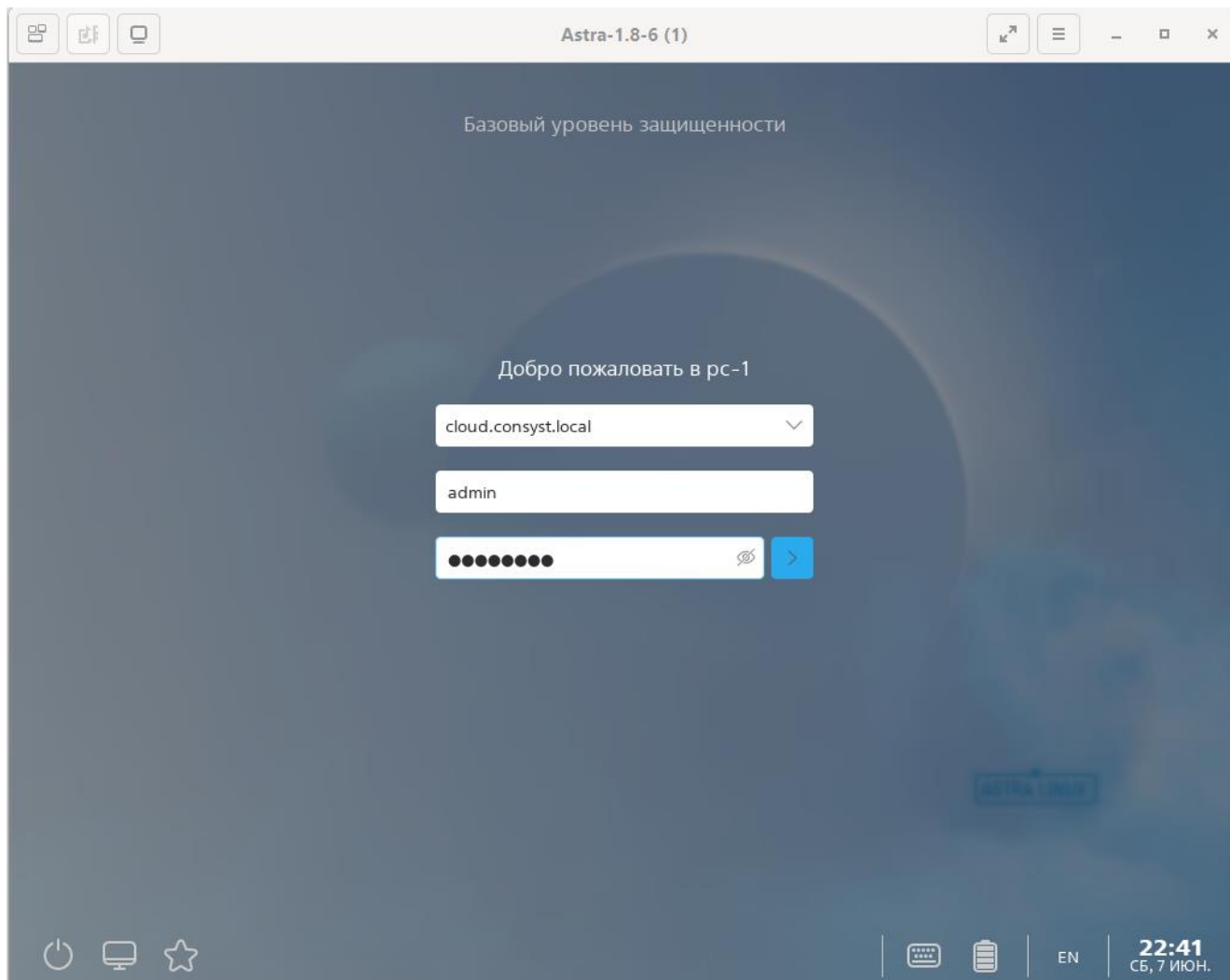


Рисунок 2.122 – Окно авторизации в доменного пользователя

После входа необходимо проделать стандартный процесс проверки UID пользователя и его Kerberos-билетов. В случае корректного отображения и отсутствия ошибок – ввод выполнен успешно. Таким образом необходимо ввести каждую машину администраторов и клиентов в домен.

```
admin@cloud.consyst.local@pc-1:~$ id
uid=712600000(admin@cloud.consyst.local) gid=712600000(admins@cloud.consyst.local) rpyнны=712600000(admins@cloud.consyst.local),1001(astra-admin),712600003(lpadmin@cloud.consyst.local),1005611117(ald trust admin@cloud.consyst.local)
admin@cloud.consyst.local@pc-1:~$ klist
Ticket cache: KEYRING:persistent:712600000:krb_ccache_X1od4m0
Default principal: admin@CLOUD.CONSYST.LOCAL

Valid starting    Expires          Service principal
07.06.2025 22:41:40  08.06.2025 22:38:43  ldap/dc-1.cloud.consyst.local@CLOUD.CONSYST.LOCAL
07.06.2025 22:41:38  08.06.2025 22:38:43  krbtgt/CLOUD.CONSYST.LOCAL@CLOUD.CONSYST.LOCAL
admin@cloud.consyst.local@pc-1:~$
```

Рисунок 2.123 – Проверка пользователя и Kerberos-билетов

Последним шагом останется настроить машины администраторов. Нужно добавить публичные ключи на все машины, относящиеся к серверному сегменту. В случае с КД – этого не

требуется, так как при вводе в домен используются SSHFP записи, которые сами определяют, может ли тот или иной ПК подключаться к другим машинам в домене.

```
admin@cloud.consyst.local@pc-1:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/admin/.ssh/id_ed25519):
Created directory '/home/admin/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/admin/.ssh/id_ed25519
Your public key has been saved in /home/admin/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:tUDd924rThhAuoDfYwhv/cxIFDDCe4r5K+9wQmpz+hw admin@cloud.consyst.local@pc-1.cloud.consyst.local
The key's randomart image is:
+--[ED25519 256]--+
|..o...o..|
|.o..+...|
|o..o...|
|..*..o..|
|.o.o=BS..|
|oo..o*o..o|
|..E..+...|
|..O..|
|..*..|
+-----[SHA256]-----+
admin@cloud.consyst.local@pc-1:~$ ssh-copy-id -i /home/admin/.ssh/id_ed25519.pub root@services
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/admin/.ssh/id_ed25519.pub"
The authenticity of host 'services (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:FDlektHGFVJD1JMupAueHqbFuP0OUdwj6qnK40c0LzU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
root@services's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'root@services'"
and check to make sure that only the key(s) you wanted were added.

admin@cloud.consyst.local@pc-1:~$ ssh root@services
Last login: Sat Jun  7 19:51:05 2025
[root@Services ~]#
```

Рисунок 2.124 – Процесс копирования ключей для беспарольного доступа

После выполнения шагов с ключами на всех машинах администраторов, всю инфраструктуру и указанные требования можно считать выполненными.

2.7 Проверка соответствия представленным и законодательным требованиям

После завершения проектирования и развёртывания макета необходимо провести проверку выполнения всех требований, представленных на начальном этапе. Виртуальная инфраструктура развёрнута в изолированной среде, без привязки к внешним сервисам и системам организации.

Все базовые сервисы, необходимые исключительно для нужд отдела, развёрнуты локально. В процессе настройки исключается любая интеграция с основной корпоративной инфраструктурой. Для повышения надёжности хранения данных были реализованы резервные копии как обычных файлов, так и конфигурационных. Средства виртуализации были объединены в кластер, а виртуальная машина с инфраструктурными сервисами была добавлена в НА-кластер, для автоматической миграции. В качестве службы единого пользовательского пространства используется ALD PRO – она интегрирована с остальными компонентами среды и задействуется при проверке доступа.

Удалённый доступ организован через Idec NGFW, по внешнему адресу. Подключение только к инфраструктуре отдела, в этом случае, позволяет четко контролировать всех удаленных сотрудников. Подключение проверено на Astra Linux, включая основные дистрибутивы, используемые в организации (Windows, Alt Linux). Ограничения на пересечение контуров также были реализованы, подключение работает стабильно.

Для фильтрации клиентского трафика используется NGFW. Созданы группы категорий контента, настроены базовые политики доступа. Основой правил фильтрации выбран подход "по умолчанию запрещено". В ходе проверки проводятся выборочные попытки доступа к различным категориям ресурсов – все нежелательные запросы корректно блокируются.

Сетевая архитектура реализована по упрощённой трёхуровневой модели без уровня распределения. Сеть разделена на четыре сегмента: клиентский, административный, репликационный и серверный. Виртуальные машины отнесены к серверному сегменту, доступ между зонами ограничен в соответствии с назначением. Сегментация выполнена через логическое разделение и настройку маршрутов. Оборудование, задействованное в макете, подбирается с учётом необходимого запаса портов.

Состав макета также был оптимизирован – число виртуальных машин сокращено до разумного минимума без моделирования всех рабочих мест. Физическая схема размещения оборудования в макете не учитывается. Также был развёрнут центр сертификации, проверена возможность выпуска и отзыва сертификатов. Дополнительно развёрнуты сервисы для внутренней документации, с учётом потребностей сотрудников, занятых в разработке.

На текущем этапе можно сказать, что все представленные требования реализованы в полном объёме. Однако все также важно учитывать, что макет создаётся исключительно в демонстрационных и тестовых целях. Его структура не отражает финальное физическое размещение оборудования и не учитывает ряд эксплуатационных нюансов. Основная задача – протестировать логические решения, оценить взаимодействие компонентов и подтвердить применимость выбранных подходов в пределах поставленной задачи.

3 Охрана труда, ТБ и производственная санитария при эксплуатации инфраструктуры

Проектирование, развёртывание и сопровождение ИТ-инфраструктуры сопровождаются рядом производственных рисков, требующих системного подхода к вопросам охраны труда. Для безопасной и устойчивой работы с техническими средствами необходимо заранее предусмотреть организационные, санитарно-гигиенические и инженерные меры защиты персонала.

Работы, связанные с подключением и обслуживанием серверного, телекоммуникационного и вспомогательного оборудования, отнесены к категории технически сложных и выполняются исключительно квалифицированными специалистами. Все сотрудники, допущенные к эксплуатации инфраструктуры, обязаны пройти предварительное обучение, инструктаж по технике безопасности и ознакомиться с нормативными документами, действующими в организации. Учет прохождения инструктажей ведётся в специализированных журналах.

Помещения, предназначенные для размещения оборудования, должны соответствовать ряду требований: наличие принудительной вентиляции, контролируемый температурный режим, защита от статического электричества, соблюдение норм влажности и пылеудаления. Допускается установка климатических систем с резервированием для обеспечения круглосуточной стабильной работы аппаратных средств.

Особое внимание уделяется электробезопасности. Все устройства подключаются через заземлённые розетки, при этом нагрузка на сеть рассчитывается заранее с учётом номинальной мощности. Для повышения надёжности применяются источники бесперебойного питания, обеспечивающие корректное завершение работы в случае отключения основного электроснабжения. В распределительных шкафах устанавливаются автоматические выключатели и устройства аварийного отключения. Работа под напряжением категорически запрещена.

При выполнении работ в стойках и коммуникационных узлах запрещается использовать нестандартные инструменты и несертифицированные расходные материалы. Кабельные трассы прокладываются в соответствии с регламентом, с учётом радиусных изгибов, допустимой нагрузки и необходимой маркировки. Крепление коммутационного оборудования осуществляется с учётом веса, центра тяжести и устойчивости стоек.

Рабочие места персонала, взаимодействующего с системами управления, должны быть оборудованы с соблюдением требований эргономики. Это включает регулируемую мебель, специальные экраны, достаточную площадь для работы с документацией и периферийными устройствами. Освещённость обеспечивается комбинированным способом: общее освещение

дополняется индивидуальными светильниками в зонах интенсивной работы. Периодически проводятся замеры освещённости и шума.

Санитарные нормы обязывают соблюдать регулярный график проветривания и влажной уборки помещений, особенно в зонах с интенсивной работой серверного оборудования. Поскольку источники шума – вентиляторы, источники питания, коммутационное оборудование – могут превышать допустимые значения, допускается использование шумозащитных панелей или индивидуальных средств защиты слуха.

Все работы по замене комплектующих, прокладке новых линий, диагностике и обновлению системного программного обеспечения проводятся по утверждённому регламенту. При этом должна быть обеспечена фиксация каждого этапа – от отключения питания до восстановления штатной работы оборудования. Не допускается нарушение процедур журналирования и эксплуатации оборудования с известными неисправностями.

Для снижения риска возгорания помещения оборудуются автоматическими системами пожаротушения, предпочтительно с использованием газового или аэрозольного состава, не наносящего вред электронике. Дополнительно устанавливаются системы раннего обнаружения задымления и сигнализации. Пожарные выходы, средства эвакуации и огнетушители должны быть доступны, промаркированы и регулярно проверяться.

Мероприятия по охране труда дополняются периодическими аудитами состояния оборудования, условий труда и соблюдения норм. Ответственные лица проводят внутренние проверки, оценивают соблюдение инструкций, актуальность технической документации и соответствие фактических условий нормативным требованиям. Все выявленные отклонения фиксируются и устраняются в установленный срок.

Заключение

В рамках дипломного проекта была выполнена разработка макета ИТ-инфраструктуры, отражающей ключевые архитектурные и технические решения, предъявляемые к современным корпоративным системам. На всех этапах работы соблюдались представленные требования, начиная от организации сетевого взаимодействия и фильтрации трафика, заканчивая развёртыванием базовых сервисов и средств обеспечения отказоустойчивости.

Проект реализован в виде изолированной виртуальной среды, в которой смоделированы основные компоненты внутренней инфраструктуры отдела. Настроена сетевая архитектура с логической сегментацией, развёрнуты сервисы управления, удалённого доступа и документооборота, внедрены механизмы централизованной аутентификации и резервного копирования. Для проверки выполнения требований были проведены отдельные сценарии взаимодействия, включая проверку фильтрации трафика, отказоустойчивости, корректности доступа и функциональности инфраструктурных сервисов.

Следует отметить, что разработанный макет не претендует на полноту и масштаб конечной инфраструктуры. Его задача заключается не в создании полноценной производственной среды, а в демонстрации возможных решений, их взаимодействия и пригодности для последующей адаптации под реальные условия. В рамках макета не учитываются эксплуатационные ограничения, экономические факторы и физическое размещение компонентов.

Подводя итог, можно сказать, что проект выполняет свою основную функцию – позволяет в безопасной и контролируемой среде протестировать принятые архитектурные подходы и убедиться в их работоспособности. Макет может быть использован как основа для последующего тиражирования, либо как инструмент принятия проектных решений на следующих этапах внедрения.

Перечень сокращений и условных обозначений

ГБ – гигабайт

ГГц – гигагерц

АСУ ТП – автоматизированная система управления технологическим процессом

ВМ – виртуальная машина

ГБ – гигабайт

ЕПП – единое пользовательское пространство

ИБ – информационная безопасность

ИП – индивидуальный предприниматель

ИС – информационная система

ИСПДн – информационная система персональных данных

КИИ – критическая информационная инфраструктура

МСЭ – межсетевой экран

МЭ – межсетевой экран

ОС – операционная система

ОЗУ – оперативное запоминающее устройство

ОСРВ – операционная система реального времени

ПК – персональный компьютер

ПД – персональные данные

ПО – программное обеспечение

САПР – система автоматизированного проектирования

СУБД – система управления базами данных

ФЗ – федеральный закон

ЦОД – центр обработки данных

ACL – Access Control List (список управления доступом)

AD – Active Directory (служба каталогов Microsoft)

ALD – Astra Linux Directory (служба каталогов Astra Linux)

BGP – Border Gateway Protocol (протокол граничного шлюза)

CAD – Computer-Aided Design (система автоматизированного проектирования)

CRC – Cyclic Redundancy Check (циклический избыточный код)

CPU – Central Processing Unit (центральный процессор)

CRM – Customer Relationship Management (система управления взаимоотношениями с клиентами)

DDNS – Dynamic Domain Name System (динамическая система доменных имён)

DHCP – Dynamic Host Configuration Protocol (протокол динамической настройки хостов)

DNS – Domain Name System (система доменных имён)

DNSSEC – Domain Name System Security Extensions (расширения безопасности DNS)

EDR – Endpoint Detection and Response (обнаружение и реагирование на угрозы на конечных устройствах)

ECMP – Equal-Cost Multi-Path (маршрутизация с равной стоимостью путей)

GPU – Graphics Processing Unit (графический процессор)

HA – High Availability (высокая доступность)

HSRP – Hot Standby Router Protocol (протокол горячего резервирования маршрутизаторов)

ICMP – Internet Control Message Protocol (протокол управляющих сообщений интернета)

IDS – Intrusion Detection System (система обнаружения вторжений)

IPS – Intrusion Prevention System (система предотвращения вторжений)

IP – Internet Protocol (сетевой протокол IP)

iSCSI – Internet Small Computer System Interface (протокол передачи SCSI-команд по IP-сетям)

LAG – Link Aggregation Group (группа агрегации каналов)

LDAP – Lightweight Directory Access Protocol (протокол лёгкого доступа к каталогам)

MAC – Media Access Control (уровень управления доступом к среде передачи)

MSTP – Multiple Spanning Tree Protocol (расширение STP для VLAN)

NAS – Network Attached Storage (сетевое хранилище данных)

NAT – Network Address Translation (трансляция сетевых адресов)

NGFW – Next Generation Firewall (межсетевой экран нового поколения)

OSPF – Open Shortest Path First (протокол маршрутизации с открытым кратчайшим путём)

QoS – Quality of Service (качество обслуживания)

RAID – Redundant Array of Independent Disks (избыточный массив независимых дисков)

RAM – Random Access Memory (оперативная память)

RSTP – Rapid Spanning Tree Protocol (быстрый протокол дерева расширений)

SAN – Storage Area Network (выделенная сеть хранения данных)

SDN – Software-Defined Networking (сети с программно-определяемой маршрутизацией)

SFP – Small Form-factor Pluggable (оптический/медный трансивер малого форм-фактора)

SIEM – Security Information and Event Management (система управления событиями и информацией безопасности)

SSH – Secure Shell (протокол защищённого удалённого доступа)

SSL – Secure Sockets Layer (протокол защищённого соединения)

STP – Spanning Tree Protocol (протокол построения дерева)

TLS – Transport Layer Security (протокол безопасности транспортного уровня)

URL – Uniform Resource Locator (унифицированный указатель ресурса)

VLAN – Virtual Local Area Network (виртуальная локальная сеть)

VPN – Virtual Private Network (виртуальная частная сеть)

VRP – Virtual Routing Platform (виртуализированная маршрутизирующая платформа)

VRRP – Virtual Router Redundancy Protocol (протокол резервирования маршрутизаторов)

VXLAN – Virtual Extensible LAN (расширяемая виртуальная локальная сеть поверх IP)

Список используемых источников

1 ГОСТ 2.301–68 Единая система конструкторской документации. Форматы. – М.: СтандартИнформ, 2007.

2 ГОСТ 2.104–2006 Единая система конструкторской документации. Основные надписи. – М.: СтандартИнформ, 2007. – 14 с.

3 ГОСТ 2.105–2019 Единая система конструкторской документации. Общие требования к текстовым документам. – М.: СтандартИнформ, 2024. – 35 с.

4 Клиент-серверная архитектура. – URL: <https://servergate.ru/articles/klient-servernaya-arkhitektura> (дата обращения: 12.05.2025)

5 Безопасность сервера. – URL: https://help.ubuntu.ru/wiki/безопасность_сервера (дата обращения: 02.06.2025)

6 Средства сетевой диагностики. – URL: <https://docs.carbonsoft.ru/pages/viewpage.action?pageId=7241735> (дата обращения: 13.05.2025)

7 Мониторинг и анализ производительности сетевых интерфейсов в Linux: ethtool, iperf, nload. – URL: <https://fileenergy.com/linux/monitoring-i-analiz-proizvoditelnosti-setevykh-interfejsov-v-linux-ethtool-iperf-nload> (дата обращения: 14.05.2025)

8 Диагностика Linux. – URL: <https://garden.struchkov.dev/ru/dev/linux/Диагностика-Linux> (дата обращения: 12.12.2012)

9 Развертывание ALD. – URL: https://www.aldpro.ru/professional/ALD_Pro_Module_02/ (дата обращения: 03.06.2025)

10 Сеть и стек TCP/IP, управление сетевыми интерфейсами. – URL: <https://www.aldpro.ru/professional/ALSE-Module-20/> (дата обращения: 10.06.2025)

11 Установка Docker. – URL: <https://docs.docker.com/get-started/get-docker/> (дата обращения: 10.06.2025)

12 Zabbix. Установка в Docker. – URL: https://www.zabbix.com/container_images (дата обращения: 10.06.2025)

13 Трудовой кодекс Российской Федерации: от 30.12.2001 № 197-ФЗ (ред. от 07.04.2025) // Собрание законодательства РФ. – 07.01.2002. – № 1 (ч. 1). – Ст. 3.

14 ГОСТ Р 50949–2001 Средства отображения информации индивидуального пользования. Методы измерений и оценки эргономических параметров и параметров безопасности. – М.: СтандартИнформ, 2008. – 23 с.

Приложение А
(обязательное)

Прайс-лист с ценами

Таблица А.1 – Прайс-лист с ценами

№ п/п	Наименование	Кол-во, шт.	Цена за единицу, руб., с НДС	Сумма, руб., с НДС
1	Программный комплекс ALD Pro, лицензия на 1 контроллер домена, на 8 управляемых устройств и ОС ALSE	1	343 600,00	343 600,00
2	vESR BASIC на 1 устройство	5	13 230,00	66 150,00
3	Astra Linux Special Edition 1.7, x86-64 «Орел», сроком на 12 мес.	12	6 150,00	73 800,00
Итого				483 550,00

Приложение Б
(обязательное)



КОНСИСТ-ОС
РОСАТОМ

ОРГАНИЗАЦИЯ АО «КОНЦЕРН РОСЭНЕРГОАТОМ»

Акционерное общество
«КОНСИСТ – ОПЕРАТОР СВЯЗИ» (АО «КОНСИСТ-ОС»)

АКТ АПРОБАЦИИ

Наименование дипломного проекта: РАЗРАБОТКА МАКЕТА ИТ-ИНФРАСТРУКТУРЫ СОВРЕМЕННОЙ КОМПАНИИ

Описание: в период с 13.02.2025 по 12.05.2025 Куваев Тимофей Иванович, в рамках программы стажировки провёл работы по разработке макета тестирования инфраструктуры, предназначенной для повышения эффективности управления ресурсами и обеспечения безопасности данных компании. В рамках апробации была проведена комплексная проверка работоспособности всех компонентов системы, включая серверное оборудование, средства виртуализации. Благодаря проведенной апробации удалось выявить потенциальные риски и слабые места в проектируемой инфраструктуре, что позволило своевременно внести необходимые корректировки и оптимизации. Сотрудники, участвующие в тестировании, отметили улучшение удобства использования инфраструктуры и повышение гибкости в управлении средствами компании. Общий уровень удовлетворённости от внедрения макета был оценен как высокий, что подтверждает успешность проведенной апробации. На основе полученных данных и отзывов сотрудников АО «КОНСИСТ-ОС» дипломный проект был признан успешно апробированным и рекомендованным к дальнейшему внедрению на практике.

Директор департамента развития ИТ-компетенций и экспертной поддержки сервисов

 М.А. Афанасьев

					РК 09.02.06 401 09 ПЗ	Лист
						102
Изм	Лист	№ докум.	Подп.	Дата		